TESCHA
TECNOLÓGICO DE ESTUDIOS SUPERIORES
CHALCO

INGENIERIA EN SISTEMAS COMPUTACIONALES

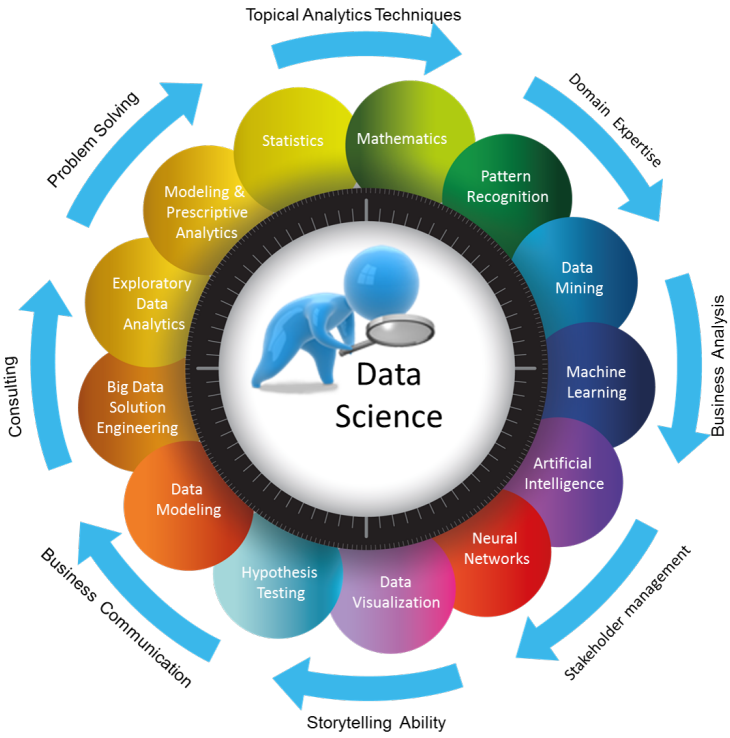
DESARROLLO DE APLICACIONES PARA LA CIENCIA DE DATOS.

Alumno: *José Antonio Pareja Pérez*

Carrera: *Ingeniería en Sistemas Computacionales*

Fecha de entrega: *Viernes 23 de octubre de 2026*

Reporte: *Ejercicio CRUD — Biblioteca Lumbre*



TESCHA
TECNOLÓGICO DE ESTUDIOS SUPERIORES
CHALCO

INGENIERIA EN
SISTEMAS
COMPUTACIONALES

ÍNDICE

1. ESPECIFICACIÓN Y DISEÑO SDD. 1

1.1. REQUERIMIENTOS Y REGLAS DE NEGOCIO. 2

1.2. ESQUEMAS Y RELACIÓN ENTRE COLECCIONES. 3

2. CONTRATO COMÚN DE LA API. 4

2.1. ENDPOINTS DE GÉNEROS. 5

2.2. ENDPOINTS DE LIBROS. 6

2.3. AUTENTICACIÓN Y CONSULTAS EXTERNAS. 7

3. ARQUITECTURA POR CAPAS. 8

3.1. CONTROLADOR Y SERVICIO: CÓDIGO. 9

3.2. INTEGRIDAD Y OPERACIONES DE MONGOOSE. 10

4. SERVICIOS Y CONSUMO DE OPEN LIBRARY. 11

4.1. DETALLE, ERRORES E INTEGRACIÓN. 12

5. LOGIN Y GENERACIÓN DEL JWT. 13

5.1. VERIFICACIÓN Y MIDDLEWARE JWT. 14

6. CONFIGURACIÓN Y EJECUCIÓN. 15

6.1. DEPENDENCIAS, CONEXIÓN Y ERRORES. 16

6.2. GUÍA DE ARCHIVOS Y FUNCIONES DEL CRUD. 17

6.3. GUÍA DE LOS DEMÁS COMPONENTES. 18

7. PREPARACIÓN DE POSTMAN. 19

7.1. MÉTODO, URL, PARAMS Y SEND. 20

7.2. BODY Y RESPUESTA DE ERROR. 21

ÍNDICE (CONTINUACIÓN)

7.3. HEADERS Y TIPO DE CONTENIDO. 22

7.4. AUTHORIZATION Y ASERCIONES. 23

7.5. MÉTODO DE EVIDENCIA Y AISLAMIENTO. 24

7.6. PRUEBA 01: CONSULTAR LOS RECURSOS DE LA API. 25

7.7. PRUEBA 02: RECHAZAR UNA CONTRASEÑA INCORRECTA. 26

7.8. PRUEBA 03: OBTENER EL TOKEN JWT DE ACCESO. 27

7.9. PRUEBA 04: RECHAZAR UNA PETICIÓN SIN TOKEN. 28

7.10. PRUEBA 05: RECHAZAR UN TOKEN VENCIDO. 29

7.11. PRUEBA 06: CREAR UN GÉNERO LITERARIO. 30

7.12. PRUEBA 07: CONSULTAR TODOS LOS GÉNEROS. 31

7.13. PRUEBA 08: CONSULTAR UN GÉNERO POR ID. 32

7.14. PRUEBA 09: ACTUALIZAR UN GÉNERO. 33

7.15. PRUEBA 10: CREAR UN LIBRO VINCULADO AL GÉNERO. 34

7.16. PRUEBA 11: CONSULTAR EL CATÁLOGO DE LIBROS. 35

7.17. PRUEBA 12: CONSULTAR UN LIBRO POR ID. 36

7.18. PRUEBA 13: ACTUALIZAR ESTANTE Y EJEMPLARES. 37

7.19. PRUEBA 14: PROTEGER UN GÉNERO CON LIBROS ASOCIADOS. 38

7.20. PRUEBA 15: RECHAZAR EJEMPLARES NEGATIVOS. 39

7.21. PRUEBA 16: RECHAZAR UN ID CON FORMATO INVÁLIDO. 40

7.22. PRUEBA 17: ELIMINAR UN LIBRO. 41

7.23. PRUEBA 18: COMPROBAR QUE EL LIBRO YA NO EXISTE. 42

ÍNDICE (CONTINUACIÓN)

7.24. PRUEBA 19: ELIMINAR EL GÉNERO SIN LIBROS ASOCIADOS. 43

7.25. PRUEBA 20: COMPROBAR EL CATÁLOGO VACÍO. 44

7.26. PRUEBA 21: COMPROBAR LA ELIMINACIÓN DEL GÉNERO. 45

7.27. PRUEBA 22: BUSCAR OBRAS: OPEN LIBRARY SIMULADO. 46

7.28. PRUEBA 23: CONSULTAR FICHA: PROVEEDOR SIMULADO. 47

7.29. PRUEBA 24: OBRA INEXISTENTE: PROVEEDOR SIMULADO. 48

7.30. PRUEBA 25: FALLA DEL PROVEEDOR: RESPUESTA 502. 49

7.31. PRUEBA 26: RENOVAR ACCESO PARA PRUEBAS NEGATIVAS. 50

7.32. PRUEBA 27: LOGIN SIN CAMPOS REQUERIDOS. 51

7.33. PRUEBA 28: TOKEN INVÁLIDO EN CONSULTA GENERAL. 52

7.34. PRUEBA 29: ALTA DE GÉNERO SIN NOMBRE. 53

7.35. PRUEBA 30: CREAR GÉNERO PARA VALIDAR CONFLICTOS. 54

7.36. PRUEBA 31: ALTA DE GÉNERO DUPLICADO. 55

7.37. PRUEBA 32: CAMBIO DE GÉNERO CON NOMBRE VACÍO. 56

7.38. PRUEBA 33: CREAR SEGUNDO GÉNERO. 57

7.39. PRUEBA 34: RENOMBRADO CON NOMBRE DUPLICADO. 58

7.40. PRUEBA 35: CONSULTA DE GÉNERO CON ID INVÁLIDO. 59

7.41. PRUEBA 36: CAMBIO DE GÉNERO INEXISTENTE. 60

7.42. PRUEBA 37: BAJA DE GÉNERO INEXISTENTE. 61

7.43. PRUEBA 38: ALTA DE LIBRO SIN AUTOR. 62

7.44. PRUEBA 39: ALTA DE LIBRO CON GÉNERO INEXISTENTE. 63

ÍNDICE (CONTINUACIÓN)

7.45. PRUEBA 40: ALTA DE LIBRO CON EJEMPLARES FRACCIONARIOS. 64

7.46. PRUEBA 41: CAMBIO DE LIBRO INEXISTENTE. 65

7.47. PRUEBA 42: CAMBIO DE LIBRO CON CAMPOS NO PERMITIDOS. 66

7.48. PRUEBA 43: BAJA DE LIBRO INEXISTENTE. 67

7.49. PRUEBA 44: BAJA DE LIBRO CON ID INVÁLIDO. 68

7.50. PRUEBA 45: ELIMINAR GÉNERO DE CONTROL. 69

7.51. PRUEBA 46: ELIMINAR GÉNERO DE PRUEBAS NEGATIVAS. 70

7.52. PRUEBA 47: CONFIRMAR GÉNEROS VACÍOS. 71

7.53. PRUEBA 48: BÚSQUEDA EXTERNA CON PARÁMETRO VACÍO. 72

7.54. PRUEBA 49: DETALLE EXTERNO CON ID INVÁLIDO. 73

7.55. PRUEBA 50: BUSCAR PEDRO PÁRAMO EN OPEN LIBRARY REAL. 74

7.56. PRUEBA 51: CONSULTAR OBRA EN OPEN LIBRARY REAL. 75

8. RESULTADOS Y EVIDENCIAS. 76

8. RESULTADOS Y EVIDENCIAS (CONTINUACIÓN). 77

8. RESULTADOS Y EVIDENCIAS (CONTINUACIÓN). 78

8.1. TRAZABILIDAD Y VALIDACIÓN FINAL. 79

8.2. CONCLUSIONES Y LÍMITES. 80

8.3. REPRODUCCIÓN Y FUENTES. 81

1. ESPECIFICACIÓN Y DISEÑO SDD.

Objetivo y dominio.

Desarrollar y validar la API REST de Biblioteca Lumbre para administrar libros y géneros literarios. La base lumbre_biblioteca contiene las colecciones generos y libros, relacionadas de forma coherente: cada libro pertenece a un género. La entrega incluye código ejecutable, especificaciones, colección de Postman y evidencias de solicitudes HTTP.

Punto de partida y alcance temporal.

La aplicación inicial se adaptó de empresa. No existe evidencia de una especificación escrita antes de esa implementación. Para esta entrega se realizó una nueva iteración SDD: primero se guardó docs/ESPECIFICACION-SDD.md; después se reorganizaron los servicios y controladores y se reforzó JWT; finalmente se ejecutaron pruebas contra esos criterios. Esta secuencia evita atribuir al trabajo anterior una planificación que no está documentada.

Etapas aplicadas.

1. Definir requerimientos RF01–RF08, reglas RN01–RN06, datos y contratos HTTP.

2. Implementar la delegación routes → controllers → services → models y verificar exclusivamente JWT HS256 con claims de sesión completos.

3. Ejecutar pruebas positivas y negativas, comparar código HTTP y contenido con el contrato y conservar evidencias.

4. Revisar la matriz de trazabilidad y documentar resultados, decisiones y límites.

La portada conserva el viernes 23 de octubre de 2026 indicado para entrega. Las evidencias conservan sus fechas reales de ejecución posterior. El reporte se concentra en el backend y las herramientas de prueba.

1.1. REQUERIMIENTOS Y REGLAS DE NEGOCIO.

Elemento	Definición / responsabilidad
RF01–RF02 · Acceso	Login emite JWT por una hora. Todo CRUD requiere Bearer válido. Rechazar credenciales incorrectas, tokens inválidos y expirados.
RF03 · Géneros	Alta, consulta general, consulta por ID, cambio y baja. Nombre requerido y único; descripción opcional.
RF04 · Libros	Las cinco operaciones CRUD. Nombre, autor, género, ubicación y ejemplares obligatorios.
RF05–RF06 · Relación	Género existente para guardar libro; poblar referencias al consultar; impedir eliminación de un género ocupado.
RF07 · Proveedor externo	Consultar búsqueda y obra de Open Library desde Axios; adaptar datos y controlar 404/502.
RF08 · Validación	Rechazar tipos incorrectos, ID inválido, cambios vacíos y campos de actualización no permitidos.

RN01: nombre del género con trim y unicidad sensible a mayúsculas. RN02: el género de un libro puede recibirse como nombre existente o ObjectId. RN03: solo campos editables pueden cambiar, conservando _id y fechaCreacion. RN04: borrado físico; repetirlo devuelve 404. RN05: listado vacío responde 200 con datos: [].

RN06: práctica local con un administrador. La comprobación de relaciones es secuencial, sin transacciones; ante escritores concurrentes se necesita una estrategia transaccional antes de usar el proyecto en producción. No se implementan préstamos, cuentas múltiples ni roles porque no forman parte de este alcance.

1.2. ESQUEMAS Y RELACIÓN ENTRE COLECCIONES.

Elemento	Definición / responsabilidad
generos._id	ObjectId generado por MongoDB; clave del género.
generos.nombre	String requerido, trim, índice único. Ejemplo: Narrativa.
generos.descripcion	String opcional, trim, valor predeterminado vacío.
generos.fechaCreacion	Date, asignada por Date.now al crear.
libros._id	ObjectId generado por MongoDB.
libros.nombre / autor	String requeridos, trim. Ejemplo: Pedro Páramo / Juan Rulfo.
libros.genero	ObjectId requerido, ref: Genero. Referencia persistida.
libros.ubicacion	String requerido, trim. Ejemplo: A-01.
libros.ejemplares	Number requerido; entero mayor o igual a cero.
libros.fechaCreacion	Date automática. __v en ambos modelos es metadato de Mongoose.

Cardinalidad: Genero (1) → Libro (N). El virtual Genero.libros no se almacena como un segundo arreglo: Mongoose lo obtiene haciendo coincidir _id del género con genero de los libros. Así se evita mantener duplicadas ambas direcciones de la relación.

populate("genero") sustituye la referencia por el documento relacionado en la respuesta del libro. populate("libros") incorpora el arreglo inverso en las consultas de géneros. La API comprueba la existencia del género; una referencia de Mongoose por sí sola no impone una llave foránea de SQL.

2. CONTRATO COMÚN DE LA API.

Direcciones y formato.

Base habitual: http://127.0.0.1:8021. Base de pruebas: http://127.0.0.1:8031. La instancia 8032 usa Open Library simulado. Las respuestas son JSON. POST y PUT con cuerpo usan Content-Type: application/json; el CRUD añade Authorization: Bearer <token>. GET y DELETE no requieren cuerpo.

Parámetros y respuestas.

En rutas con :id, sustituirlo por un ObjectId de 24 caracteres hexadecimales. Un formato incorrecto produce 400; un identificador válido sin documento produce 404. Los listados no reciben parámetros, filtros ni paginación. Las consultas externas usan q en query o un ID de obra en path.

```
Éxito CRUD: { "mensaje": "...", "datos": {...} }
Listado: { "mensaje": "...", "datos": [] }
Error: { "mensaje": "Descripción del rechazo" }
```

Elemento	Definición / responsabilidad
201 / 200	Creación / consulta, cambio o borrado correctos.
400 / 409	Datos inválidos o género ocupado / conflicto de nombre único.
401 / 403	Credenciales o token inválidos / token expirado.
404	Documento u obra inexistente.
500 / 503	Fallo interno / MongoDB o configuración de login no disponibles.
502	Fallo de Open Library traducido por el controlador externo.

Se conservan los sufijos del ejercicio por compatibilidad. PUT /baja/:id actualiza parcialmente los campos enviados; DELETE /eliminar/:id elimina. Esta semántica se declara de forma expresa. En un contrato nuevo se preferirían rutas de recurso y PATCH para cambios parciales.

2.1. ENDPOINTS DE GÉNEROS.

Elemento	Definición / responsabilidad
POST /generos/falta	Alta. Body: nombre:string requerido; descripcion:string opcional. 201 devuelve el género; 400 nombre ausente o duplicado detectado previamente; 409 conflicto del índice.
GET /generos/obtenerVarios	Consulta general sin Body. 200 devuelve datos:Genero[], incluyendo libros relacionados. Una lista vacía es válida.
GET /generos/obtenerUno/:id	Consulta específica. id en path obligatorio. 200 género con libros; 400 ID mal formado; 404 inexistente.
PUT /generos/baja/:id	Cambio. Body no vacío con nombre y/o descripcion. 200 documento actualizado; 400 campos inválidos; 404 inexistente; 409 nombre duplicado.
DELETE /generos/eliminar/:id	Baja física sin Body. 200 documento eliminado; 400 ID inválido o género con libros; 404 inexistente.

Ejemplos de solicitudes.

```
POST: {"nombre":"Narrativa","descripcion":"Novelas"}
PUT: {"nombre":"Literatura"}
```

Los cinco endpoints requieren Bearer y pueden devolver 401, 403, 503 o 500 según las condiciones comunes. La respuesta de consulta incorpora libros; el alta y la modificación no garantizan ese virtual poblado. No se debe interpretar un virtual sin poblar como una consulta de libros.

Pruebas de referencia: 6–9 y 19 cubren éxito; 14 comprueba relación; 21, 29, 31–37 cubren nombres, conflictos, IDs y documentos inexistentes.

2.2. ENDPOINTS DE LIBROS.

Elemento	Definición / responsabilidad
POST /libros/falta	Alta. Body requerido: nombre, autor, genero y ubicacion de texto; ejemplares entero >=0. 201 devuelve libro con género poblado; 400 datos o género inexistente.
GET /libros/obtenerVarios	Consulta general sin Body. 200 datos:Libro[] con cada género poblado.
GET /libros/obtenerUno/:id	Consulta específica. id obligatorio en path. 200 libro poblado; 400 ID inválido; 404 documento inexistente.
PUT /libros/baja/:id	Cambio parcial. Al menos uno: nombre, autor, genero, ubicacion, ejemplares. 200 documento actualizado poblado; 400 datos inválidos; 404 inexistente.
DELETE /libros/eliminar/:id	Baja física sin Body. 200 documento eliminado, cuya referencia genero conserva el ObjectId; 400 ID inválido; 404 inexistente.

```
{ "nombre": "Pedro Páramo", "autor": "Juan Rulfo",
"genero": "Narrativa", "ubicacion": "A-01",
"ejemplares": 3 }
```

genero admite el nombre exacto de un género existente o su ObjectId. El servicio resuelve ambos al mismo ObjectId antes de guardar. Un número fraccionario, negativo, nulo o recibido como texto no satisface el contrato de ejemplares.

Headers comunes: Authorization: Bearer <token>; Content-Type: application/json al enviar Body. Pruebas 10–13 y 17: éxito. Pruebas 15–16, 18 y 38–44: datos, referencias, consultas, cambios y bajas inválidas.

2.3. AUTENTICACIÓN Y CONSULTAS EXTERNAS.

Elemento	Definición / responsabilidad
GET /api	Público, sin Body: 200 con sistema, relación y rutas disponibles. Comprueba Express; no sustituye una prueba de persistencia.
POST /auth/login	Público. JSON usuario:string y contrasena:string requeridos. 200 entrega token, usuarioAutenticado y duracion. 400 faltan campos, 401 credenciales incorrectas, 503 falta configuración.
POST /auth/token	Alias compatible de login: mismo cuerpo y respuestas. La colección usa /auth/login.
GET /servicios/libros	Público, query q opcional; default literatura; 1–200 caracteres no vacíos. 200 {datos:[{id,nombre,autor,anio}],total}. 400 query inválida; 502 proveedor no disponible.
GET /servicios/libros/:id	Público, id en path con patrón OL seguido de dígitos y W. 200 {datos:{id,nombre,resumen}}. 400 formato, 404 obra inexistente, 502 fallo externo.

No se envía Body en las consultas externas. Axios envía Accept: application/json y usa HTTPS por defecto. Un resultado externo es información bibliográfica consultada; no modifica las existencias locales ni crea libros automáticamente.

Los contratos de esta sección, los esquemas y los criterios de aceptación se conservan en docs/ESPECIFICACION-SDD.md. Las solicitudes ejecutables, variables, ejemplos de respuesta y aserciones están en reporte/Lumbre-CRUD.postman_collection.json.

3. ARQUITECTURA POR CAPAS.

Elemento	Definición / responsabilidad
routers/ · routes	Asocia método y URL. libroRouter y generoRouter aplican verificarToken antes de llamar al controlador. authRouter expone login y alias; servicioRouter expone búsqueda y detalle.
control/ · controllers	libroControl y generoControl extraen body/params, invocan el servicio y devuelven estado y JSON. servicioControl valida query/path externos y traduce errores del proveedor.
service/ · services	catalogoService centraliza reglas, validaciones, relación y operaciones de Mongoose. libroService mantiene el cliente Axios y adapta el contrato de Open Library.
modelo/ · models	Libro y Genero definen tipos, obligatorios, validaciones e índice único; representan las colecciones persistentes.
middleware/	verificarToken protege rutas; generarToken contiene el manejador de login y el firmador; manejarError convierte excepciones en respuestas HTTP.
app.js / config/db.js	Composición de Express, JSON, rutas, control de conexión y errores; conexión a MongoDB con nombre separado.

Flujo de alta: Postman → express.json → comprobación de conexión → router → verificarToken → crearLibro del controlador → crearLibro del servicio → resolverGenero → modelos → MongoDB → JSON 201. El servicio no conoce req ni res, por lo que las reglas no dependen de HTTP.

Express 5 propaga el rechazo de un controlador async al middleware de errores. Se evita repetir try/catch en cada operación. Los rechazos del dominio conservan su estado; un error interno no expone stack ni datos de conexión.

3.1. CONTROLADOR Y SERVICIO: CÓDIGO.

Controlador de libros: frontera HTTP.

```
async function crearLibro(req, res) {
const datos = await servicio.crearLibro(req.body);
res.status(201).json({ mensaje: 'Libro creado exitosamente', datos });
}
```

req.body contiene el JSON ya interpretado. await espera validaciones y persistencia. Solo después del éxito se responde 201. Si el servicio rechaza la operación, Express envía el error al middleware, sin responder dos veces.

Servicio: alta con relación existente.

```
async function crearLibro(datos = {}) {
validarLibro(datos, true);
const { nombre, autor, ubicacion, ejemplares } = datos;
const genero = await resolverGenero(datos.genero);
const libro = await Libro.create({ nombre, autor, ubicacion, ejemplares, genero });
return libro.populate('genero');
}
```

validarLibro comprueba tipos y obligatorios; resolverGenero busca el género por ID o nombre; Libro.create valida el esquema y persiste; populate añade los datos del género a la respuesta. El destructuring selecciona los campos permitidos en el alta.

Responsabilidades y decisión.

Los controladores dejaron de contener consultas MongoDB. Centralizar el dominio en catalogoService permite aplicar la misma validación al crear y al actualizar. Se mantiene libroService para la integración externa porque su contrato y errores pertenecen a otro proveedor.

3.2. INTEGRIDAD Y OPERACIONES DE MONGOOSE.

Bloqueo de borrado con referencias.

```
async function eliminarGenero(id) {
validarId(id);
// shortcut: comprobación secuencial local, usar transacciones antes de admitir
escritores concurrentes.
const cantidad = await Libro.countDocuments({ genero: id });
if (cantidad) rechazar(400, `No se puede eliminar el género porque tiene ${cantidad}
libro(s) asociado(s)`);
return encontrado(await Genero.findByIdAndDelete(id), 'Género');
}
```

countDocuments verifica la relación antes del borrado. encontrado convierte null en 404. Esta comprobación permite demostrar el orden correcto de baja en la práctica; no es una transacción atómica entre colecciones bajo concurrencia.

Actualización y validaciones.

```
const cambios = { ...datos };
if ('genero' in cambios) {
cambios.genero = await resolverGenero(cambios.genero);
}
return encontrado(await Libro.findByIdAndUpdate(id,
{ $set: cambios }, { new: true, runValidators: true }
).populate('genero'), 'Libro');
```

$set cambia únicamente los campos aprobados. new:true devuelve el estado actualizado y runValidators:true aplica el esquema en la modificación. findById consulta una clave; find devuelve una lista; findOne resuelve un nombre; exists comprueba duplicados; findByIdAndDelete devuelve el documento borrado o null.

La defensa de entrada se hace antes de Mongoose para evitar que su conversión automática acepte números enviados como texto. El esquema vuelve a validar límites y requeridos para proteger la persistencia.

4. SERVICIOS Y CONSUMO DE OPEN LIBRARY.

Solicitud real desde el servicio.

```
const axios = require('axios');
const cliente = axios.create({
baseURL: process.env.OPENLIBRARY_API_URL || 'https://openlibrary.org',
timeout: 8000,
headers: { Accept: 'application/json' }
});
const buscarLibros = async (termino = 'literatura') => {
const { data } = await cliente.get('/search.json', {
params: { q: termino, limit: 12, fields: 'key,title,author_name,first_publish_year' }
});
return (data.docs || []).map(libro => ({
id: libro.key.replace('/works/', ''), nombre: libro.title,
autor: (libro.author_name || []).join(', ') || 'Autor sin registrar',
anio: libro.first_publish_year || null
}));
};
```

El cliente se configura una vez. OPENLIBRARY_API_URL permite apuntar al proveedor público o a la simulación. El tiempo máximo de 8000 ms evita dejar una consulta esperando indefinidamente. La búsqueda envía q, limit=12 y fields para solicitar solamente key, title, author_name y first_publish_year.

El resultado externo docs se transforma al contrato local: key sin /works/ → id; title → nombre; author_name unido → autor; first_publish_year → anio. Si no hay autores se usa Autor sin registrar y si no hay año se devuelve null. total es el tamaño devuelto, no el total global de coincidencias.

Fuente del contrato: documentación oficial de Search API de Open Library, https://openlibrary.org/dev/docs/api/search. La consulta de búsqueda real está en la evidencia 50; permite comprobar que se ejecutó Axios a través del backend con el proveedor público.

4.1. DETALLE, ERRORES E INTEGRACIÓN.

```
const obtenerDetalleLibro = async (id) => {
const { data } = await cliente.get(`/works/${id}.json`);
return {
id, nombre: data.title,
resumen: typeof data.description === 'string' ? data.description :
data.description?.value || 'Esta obra no tiene una descripción disponible.'
};
};
```

La ficha usa GET /works/{id}.json. description puede ser texto o un objeto con value. Si no existe, se entrega un mensaje de ausencia de descripción; no se trata como falla del proveedor. Esta adaptación permite un contrato local estable.

Elemento	Definición / responsabilidad
Entrada inválida	servicioControl rechaza q vacío, duplicado o mayor de 200 caracteres; ID externo mal formado: 400 antes de Axios.
Obra inexistente	Si Axios recibe 404 en detalle, el controlador responde 404 con Obra no encontrada.
Proveedor caído / error	Otros fallos, incluidos errores HTTP, red o timeout, producen 502. No se expone la respuesta interna del proveedor.
Uso en Lumbre	Las respuestas bibliográficas se consultan desde /servicios/libros. El catálogo local se administra separadamente con el CRUD; no hay importación automática.

Pruebas 22–25: proveedor simulado, backend y Axios reales; cubren búsqueda, descripción ausente, 404 y 503 transformado a 502. Pruebas 50–51: búsqueda y ficha del proveedor real. La disponibilidad futura depende de Open Library; no se simulan esas dos últimas respuestas.

Contrato oficial de obras: https://openlibrary.org/dev/docs/api/books. Los ejemplos de búsqueda y obra se conservan en las exportaciones completas de respuesta.

5. LOGIN Y GENERACIÓN DEL JWT.

Código de firma.

```
const crearToken = (usuario) => {
if (!process.env.JWT_SECRET) throw new Error('Configura JWT_SECRET en .env');
const ahora = Math.floor(Date.now() / 1000);
return jwt.encode({ usuario, iat: ahora, exp: ahora + 3600 }, process.env.JWT_SECRET,
'HS256');
};
```

El login exige usuario y contraseña de texto, verifica configuración y los compara con AUTH_USER y AUTH_PASSWORD. Solo al coincidir llama a crearToken. Las credenciales incorrectas devuelven 401; las incompletas 400; una configuración faltante 503.

JWT contiene tres segmentos Base64URL: header, payload y firma. El header declara HS256; el payload contiene usuario, iat (emisión) y exp (vencimiento) en segundos Unix. HS256 es HMAC-SHA256 con JWT_SECRET. La firma permite detectar modificaciones; el payload es legible y no está cifrado, por eso nunca incluye la contraseña.

```
POST /auth/login
Content-Type: application/json
{"usuario":"practica","contrasena":"PracticaCRUD2026!"}
Authorization: Bearer <token devuelto>
```

Esta cuenta pertenece únicamente a las bases temporales del reporte. La configuración habitual tiene su propia cuenta en .env. No hay tabla de usuarios: es un login de administrador único para una práctica local. En un despliegue multiusuario se requerirían contraseñas con hash, roles, HTTPS y gestión de sesiones.

5.1. VERIFICACIÓN Y MIDDLEWARE JWT.

Separación de responsabilidades.

Generar el token corresponde al proceso de login. verificarToken no crea tokens ni comprueba la contraseña: extrae Bearer, verifica firma y algoritmo, revisa los claims y la vigencia, asigna req.usuario y llama next solo si la sesión es válida.

```
const coincidencia = /^Bearer\s+(\S+)$/i.exec(
req.get('Authorization') || ''
);
const payload = jwt.decode(coincidencia[1],
process.env.JWT_SECRET, false, 'HS256');
```

false indica que no se omita la verificación. HS256 explícito impide aceptar otro algoritmo. Además se exige usuario de texto no vacío, iat y exp enteros, y exp posterior a iat. Un token incompleto o con firma inválida responde 401; un token expirado responde 403. jwt-simple puede detectar la expiración durante decode y el catch conserva el mismo rechazo.

Protección y reutilización.

```
router.use(verificarToken);
router.post('/falta', crearLibro);
```

Al instalar el middleware antes de las rutas del CRUD, altas, bajas, cambios y consultas pasan por la misma validación. El login y las consultas bibliográficas públicas no requieren Bearer. La colección guarda el token en una variable y lo reutiliza; no se pega una clave JWT_SECRET en Postman.

Las pruebas Node también rechazan un token sin exp, exp de texto, HS512 y un token sin el prefijo Bearer. Estas pruebas comprueban el cambio de seguridad definido antes de implementarlo en esta iteración SDD.

6. CONFIGURACIÓN Y EJECUCIÓN.

1. Instalar Node.js y npm; en Ejercicio CRUD ejecutar npm ci para reproducir package-lock.json.

2. Copiar .env.example a .env; generar JWT_SECRET aleatorio; conservarlo fuera de la entrega compartida.

3. Iniciar MongoDB: docker compose up -d. Ejecutar npm run seed si se desean datos de ejemplo.

4. Ejecutar npm start. Confirmar GET http://127.0.0.1:8021/api; después autenticar y consultar el CRUD.

Elemento	Definición / responsabilidad
PORT	8021 para la app habitual; servidor de pruebas usa REPORTE_PORT o 8031.
MONGODB_URI	mongodb://127.0.0.1:27018. config/db.js fija dbName=lumbre_biblioteca.
AUTH_USER / AUTH_PASSWORD	Credenciales del administrador local; las pruebas sustituyen sus valores.
JWT_SECRET	Secreto aleatorio para firmar y verificar; nunca enviarlo al cliente.
OPENLIBRARY_API_URL	https://openlibrary.org; pruebas de error: http://127.0.0.1:4901/openlibrary.
TEST_MONGODB_URI	Opcional en npm test; señala servidor MongoDB de pruebas.
OPENLIBRARY_TEST_URL	Activa prueba automatizada de Axios contra simulación.

```
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
```

Guardar la salida aleatoria en JWT_SECRET. .env no se incluye en el paquete de entrega; .env.example documenta los nombres. Docker conserva datos en volumen y solo expone 27018 a 127.0.0.1. Ctrl+C detiene Node; docker compose stop detiene MongoDB conservando el catálogo.

6.1. DEPENDENCIAS, CONEXIÓN Y ERRORES.

Elemento	Definición / responsabilidad
Node.js / npm	JavaScript del servidor; Node v26.12.1 en esta ejecución. npm ci instala versiones del lock. npm start ejecuta app.js; npm test utiliza node:test.
Express ^5.2.1	Rutas, JSON y propagación de errores async.
Mongoose ^9.11.0	Esquemas, modelos, referencias, índices y consultas MongoDB.
jwt-simple ^0.5.6	Codificación y verificación de tokens HS256.
Axios ^1.20.0	Cliente HTTP con parámetros, timeout y errores externos.
dotenv ^18.0.6	Carga .env sin sobrescribir los valores ya definidos en el proceso.
MongoDB 8 / Docker Compose	Base documental local con volumen persistente y puerto 27018.
Postman CLI 1.74.0 / Desktop 12.32.3	Recorrido automatizado y capturas del editor HTTP.

config/db.js verifica MONGODB_URI, conecta con timeout de selección de 5000 ms y fija la base separada. app.js revisa mongoose.connection.readyState antes del CRUD; si no hay conexión devuelve 503. Express puede seguir mostrando /api aunque la base no esté disponible.

manejarError transforma 11000 en 409; ValidationError y CastError en 400; JSON mal formado en 400; errores del dominio conservan su estado; los demás producen 500 genérico. No se devuelven stack, secreto ni cadena de conexión. La colección documenta errores del cliente; el estado de desconexión se trata como condición operativa.

6.2. GUÍA DE ARCHIVOS Y FUNCIONES DEL CRUD.

Elemento	Definición / responsabilidad
control/libroControl.js	crearLibro: body → servicio → 201; consultarLibros: listado 200; consultarLibroPorId: params.id → documento; actualizarLibro: id+body → cambio; eliminarLibro: id → baja.
control/generoControl.js	crearGenero, consultarGeneros, consultarGeneroPorId, actualizarGenero y eliminarGenero realizan la misma traducción HTTP para géneros, sin lógica de persistencia propia.
service/catalogoService.js	Diez funciones públicas homónimas ejecutan operaciones del dominio con Mongoose. Las consultas pueblan relaciones; altas validan datos; cambios aplican campos permitidos; bajas comprueban existencia y relación.
rechazar / validarId / encontrado	rechazar lanza Error con status; validarId exige ObjectId; encontrado convierte ausencia de documento en 404.
validarCambios / validarTexto	Restringen campos de actualización y validan strings no vacíos; obligatorio distingue alta de cambio parcial.
validarGenero / validarLibro	Reglas específicas: descripción textual; campos del libro y ejemplares enteros >=0.
resolverGenero	Resuelve nombre o ObjectId a una referencia existente; rechaza con 400 si no encuentra el género.

Los nombres de archivo conservan las carpetas del ejercicio: control equivale a controllers y modelo a models. Tener nombres en español no cambia la separación de responsabilidades. Los servicios reciben valores JavaScript y devuelven documentos/promesas; solo controladores y middleware usan req/res.

6.3. GUÍA DE LOS DEMÁS COMPONENTES.

Elemento	Definición / responsabilidad
modelo/Genero.js / Libro.js	Crean Schema y exportan mongoose.model. El virtual libros resuelve relación inversa; el validador Number.isInteger protege ejemplares.
routers/*.js	Declaran cinco operaciones por recurso, dos alias de login y dos consultas externas. Exportan un Router montado por app.js.
control/servicioControl.js	buscar valida q y devuelve datos,total; detalle valida ID de obra y mapea 404/502.
service/libroService.js	buscarLibros envía search.json y adapta docs; obtenerDetalleLibro consulta works/id.json y normaliza description.
middleware/generarToken.js	crearToken firma claims; generarTokenMiddleware es el manejador final de login, aunque el archivo conserve el nombre middleware.
middleware/verificarToken.js	Autenticación de cada solicitud CRUD y asignación de req.usuario. No genera tokens.
middleware/manejarError.js	Una salida HTTP común para excepciones. Si headersSent, delega a next para no escribir dos respuestas.
seed.js / compose.yaml	Carga datos iniciales que falten y define MongoDB persistente. No es necesario sembrar datos para las bases temporales del reporte.
test/*.test.js	Verificación con assert y node:test. Bases temporales independientes; cierre y limpieza al finalizar.
reporte/scripts/	servidor-pruebas.cjs aísla bases; capturar-pruebas.py ejecuta Postman; entrega.py compone contenido y colección; generar-reporte.py exporta PDF y Word.

package.json declara scripts y dependencias; package-lock.json fija el árbol de instalación; README.md y LEEME.txt documentan ejecución. postman/mocks y simulations describen el proveedor simulado; los JSON de datos y PNG de evidencias permiten auditar cada resultado.

7. PREPARACIÓN DE POSTMAN.

1. Iniciar MongoDB, la API temporal 8031, la simulación 4901 y la API 8032 con el comando de reporte/LEEME.txt. Ambas instancias importan app.js real y usan bases nuevas.

2. Importar reporte/Lumbre-CRUD.postman_collection.json mediante Import en Postman. La colección contiene solicitudes numeradas; conservar el orden.

3. Revisar variables baseUrl, baseExterna, usuario y contrasena. token, idGenero, idLibro e idOtroGenero se rellenan al ejecutar las solicitudes que los generan.

4. Asignar tokenVencido al JWT temporal de /tmp/lumbre-reporte-expirado-8031.txt para la prueba de expiración. Este token nunca otorga acceso.

5. Ejecutar Send para revisar una solicitud o Run collection para todo el recorrido. No ejecutar dos recorridos simultáneos contra la misma base temporal.

Colección es el conjunto reutilizable de peticiones y scripts. Solicitud define método, URL, parámetros, headers, autorización y Body. Variables permiten cambiar el destino y reutilizar identificadores sin editar cada ruta. La colección entregada es un archivo local importable, no requiere publicar documentación en un workspace.

Almacenamiento del token en Scripts → Post-response.

```
pm.collectionVariables.set('token',
pm.response.json().token);
pm.collectionVariables.set('idGenero',
pm.response.json().datos._id);
```

El primer fragmento se ejecuta tras login; el segundo tras alta de género. Las siguientes peticiones referencian {{token}} y {{idGenero}}. No se guarda un token activo dentro del archivo de entrega.

7.1. MÉTODO, URL, PARAMS Y SEND.

En la captura se envió GET a la API local /servicios/libros?q=Pedro%20Paramo. El selector de método determina la operación. La URL contiene protocolo, host, puerto y ruta. Params muestra q; %20 representa un espacio. Send ejecuta la petición. La API consultó Open Library real y respondió JSON.

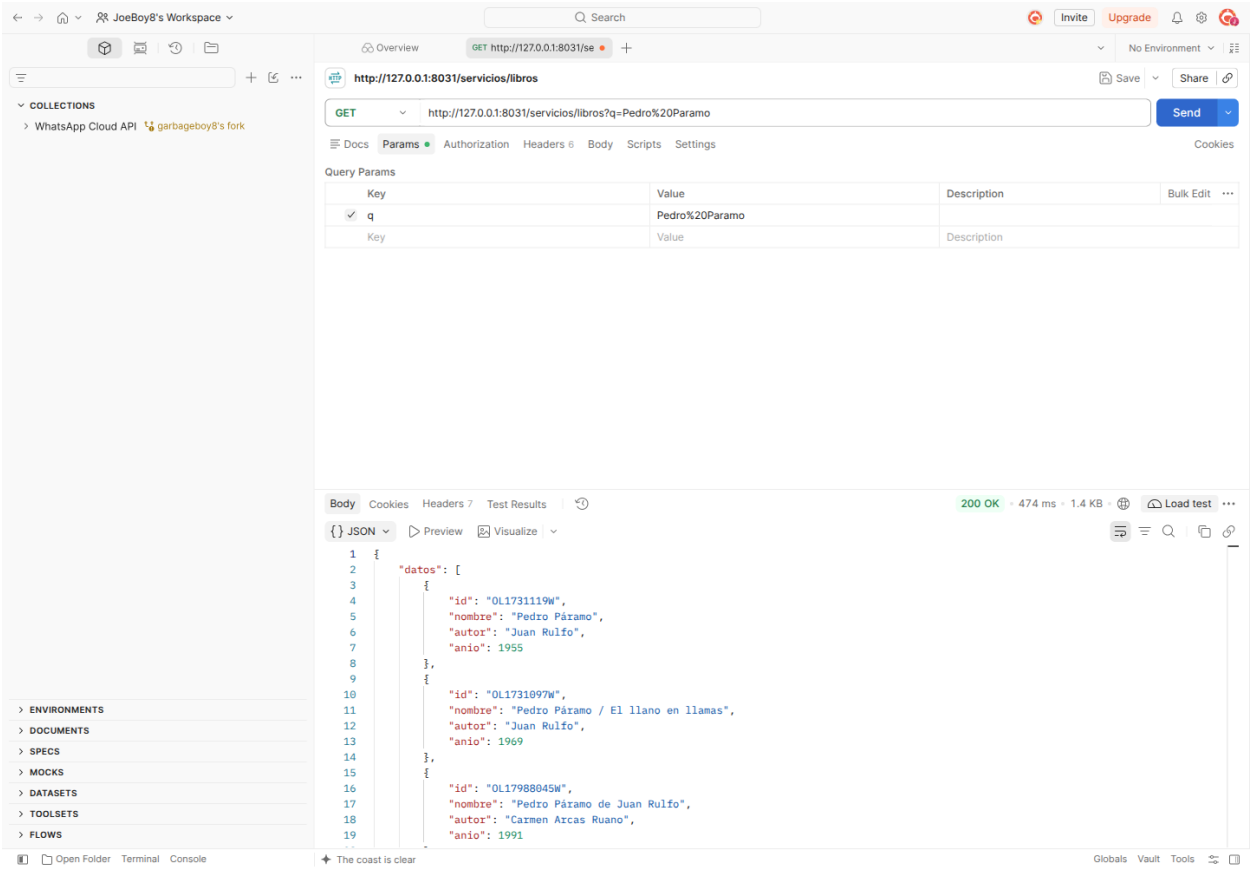


Figura P1. Postman Desktop: búsqueda real, parámetro q, Send y respuesta HTTP 200.

La zona inferior muestra el estado 200 OK, tiempo de respuesta, tamaño y Body. El tiempo es el observado en esa ejecución, no una garantía de rendimiento. El arreglo datos contiene obras adaptadas, con id, nombre, autor y anio.

7.2. BODY Y RESPUESTA DE ERROR.

Se seleccionó POST /auth/login, Body → raw → JSON. Se envió {"usuario":"practica","contrasena":"incorrecta"}. El cuerpo describe las credenciales; no es un parámetro de URL. Tras pulsar Send se obtuvo 401, porque la contraseña es intencionalmente incorrecta.

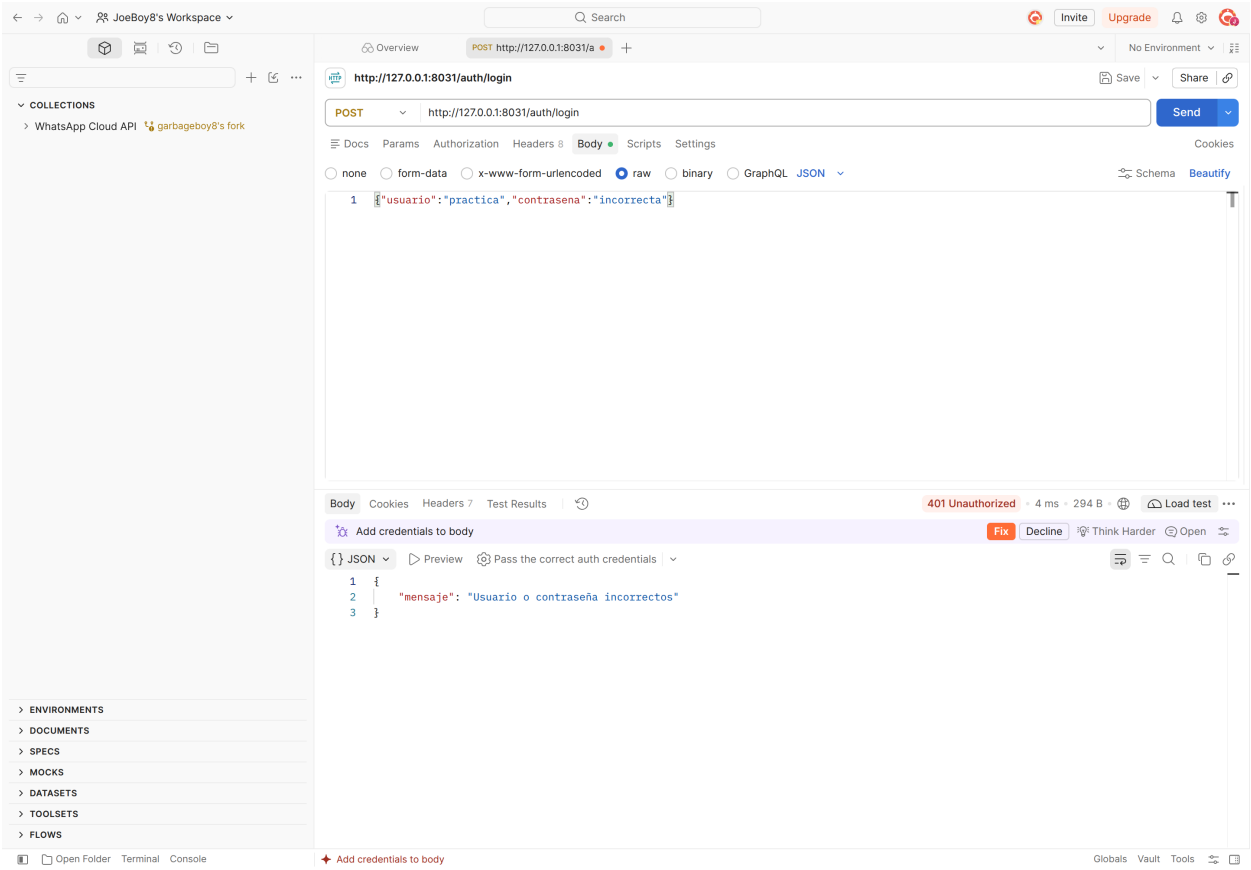


Figura P2. Postman Desktop: JSON de login y respuesta 401.

Los errores esperados también son evidencia válida: 401 demuestra que no se otorgó un token a las credenciales enviadas. Las altas y modificaciones usan el mismo editor raw JSON para sus campos de negocio. La captura utiliza una contraseña de prueba inválida.

7.3. HEADERS Y TIPO DE CONTENIDO.

Headers muestra las cabeceras que acompañan al mensaje. Content-Type: application/json permite interpretar el Body como JSON; Postman lo añade al elegir raw → JSON. Authorization debe contener Bearer y el token cuando se prueba el CRUD. Evitar duplicar manualmente un header que ya genera la pestaña Authorization.

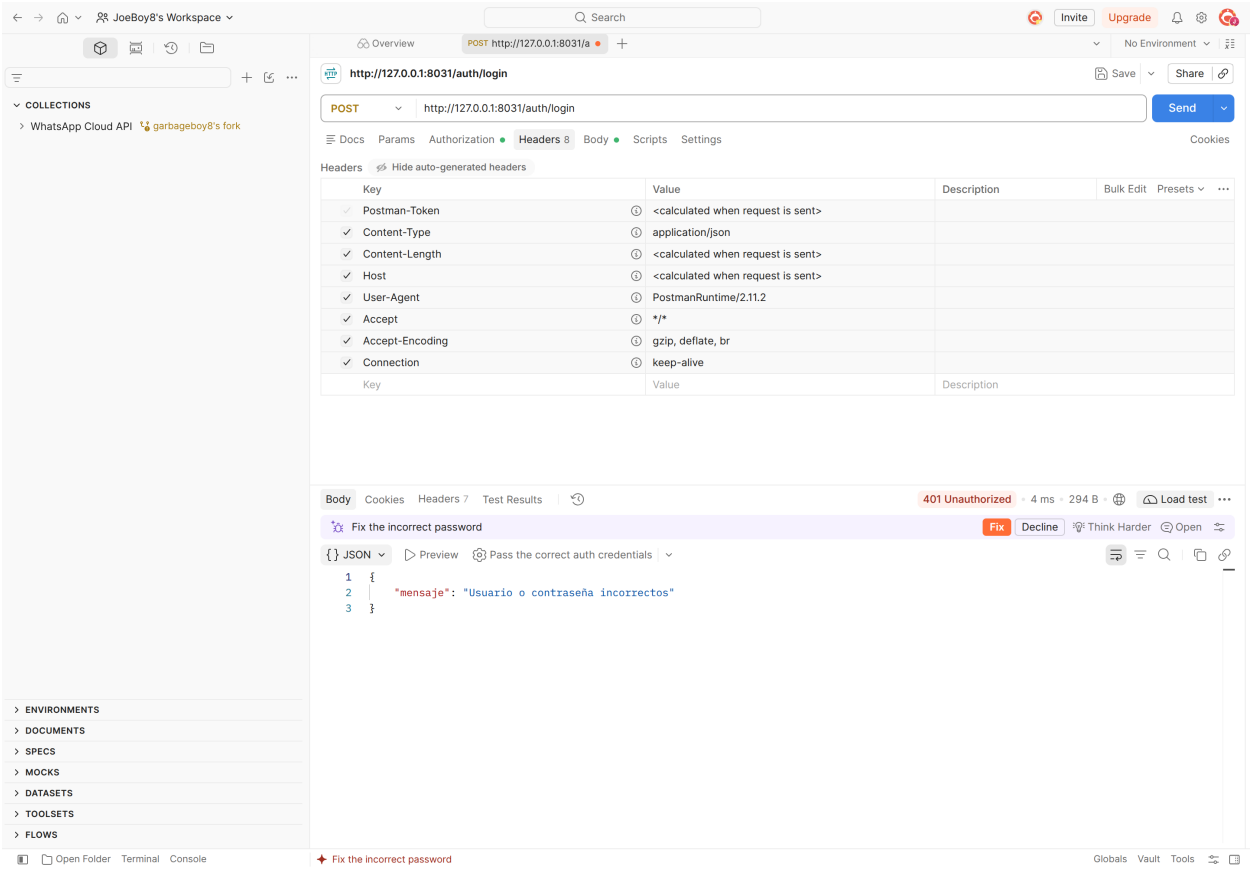


Figura P3. Postman Desktop: cabeceras de la petición JSON.

En Response también existe una pestaña Headers: contiene los encabezados devueltos por Express. Content-Type de respuesta identifica el formato recibido. Los headers de solicitud y de respuesta tienen propósitos distintos y deben revisarse en sus paneles correspondientes.

7.4. AUTHORIZATION Y ASERCIONES.

En Authorization seleccionar Bearer Token y usar {{token}}. Postman construye Authorization: Bearer <valor>. En la evidencia se utilizó el texto invalido para comprobar 401 sin mostrar un token real. Login utiliza No Auth; las peticiones de géneros y libros utilizan Bearer.

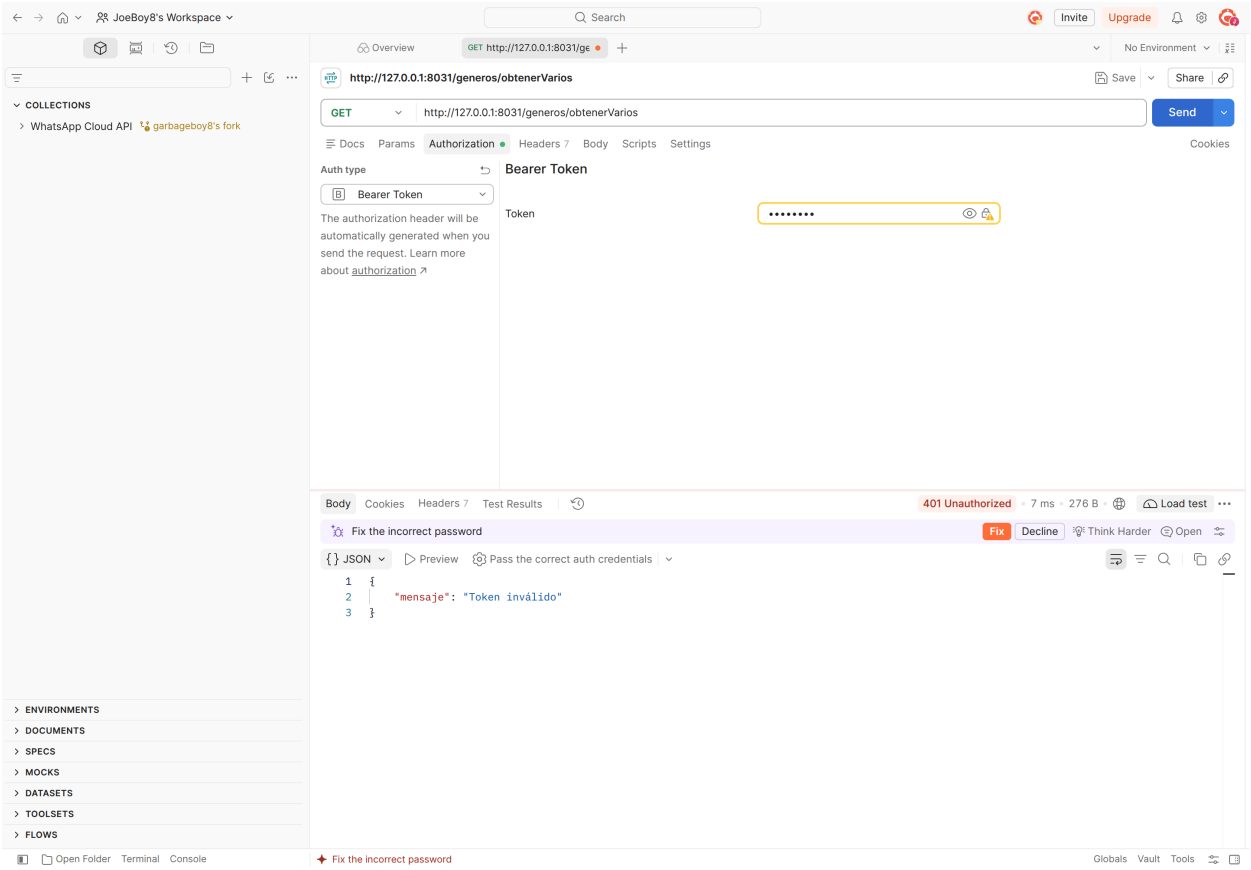


Figura P4. Postman Desktop: Bearer Token y rechazo 401.

Scripts → Post-response ejecuta pm.test para el código y el contenido esperado. Test Results muestra las aserciones. La colección añade verificaciones de campos, relaciones y listas; los casos negativos esperan expresamente 400, 401, 403, 404, 409 o 502.

7.5. MÉTODO DE EVIDENCIA Y AISLAMIENTO.

Las siguientes 51 capturas muestran una terminal Kitty real con solicitudes ejecutadas por Postman CLI. El script presenta el JSON exportado por Postman, el estado HTTP, el tiempo y el número de aserciones; no inventa respuestas. Los tokens se ocultan. Los cuerpos largos del proveedor real se abrevian solo en pantalla y se conservan completos en datos/50-resultado.json y 51-resultado.json.

Las 47 peticiones de CRUD, autenticación, validación y proveedor público se ejecutan contra la API 8031. Las cuatro peticiones 22–25 usan la API 8032 y un proveedor simulado en 4901. MongoDB y el código Express son reales en ambas. Se distingue la simulación de la disponibilidad comprobada del proveedor público.

Cada caso incluye objetivo, preparación, solicitud y resultado. Los códigos negativos son correctos cuando corresponden al contrato. El CLI puede terminar con salida 1 ante HTTP 4xx esperado; por eso el script comprueba tanto el HTTP esperado como failed=0. El recorrido de colección completo se comprueba por separado.

Archivo de respuesta y repetición.

```
reporte/datos/NN-resultado.json
reporte/datos/resultados.json
reporte/evidencias/NN.png
reporte/Lumbre-CRUD.postman_collection.json
```

Las bases temporales se llaman lumbre_reporte_ más un identificador aleatorio. Al detener cada servidor de pruebas con Ctrl+C se elimina exclusivamente su base y se cierra la conexión. La biblioteca habitual permanece separada. Los pasos de reproducción se entregan en reporte/LEEME.txt.

7.6. PRUEBA 01: CONSULTAR LOS RECURSOS DE LA API.

1. Envié GET a /api, sin cuerpo ni token, para comprobar que Express estaba disponible.

2. Revisé el nombre del sistema y las rutas de autenticación, libros, géneros y servicios que devolvió el servidor.

3. Verifiqué el estado 200 y la cabecera Content-Type con formato JSON.

Resultados:

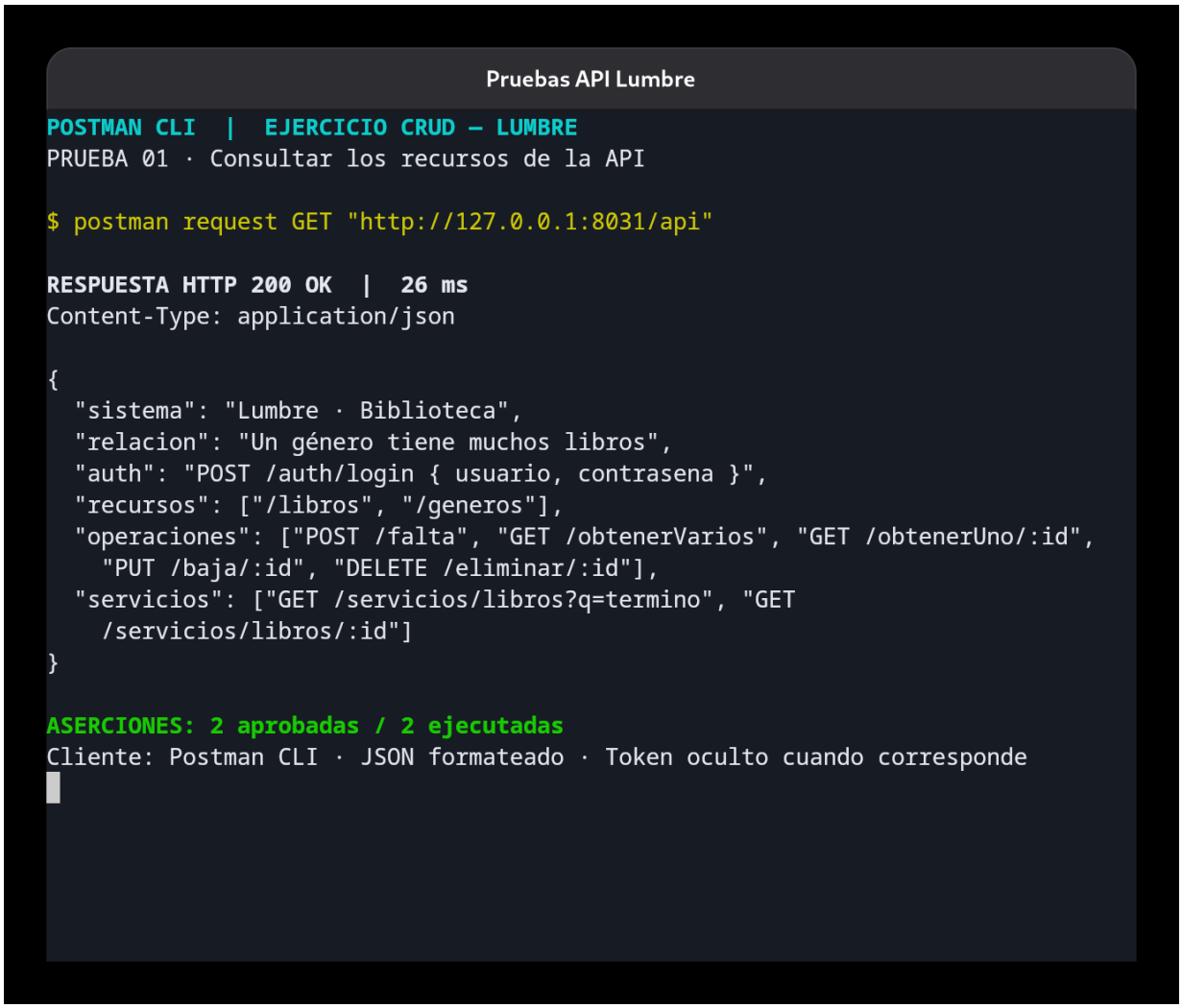


Figura 01. HTTP 200; 26 ms; 2 aserciones aprobadas.

La API respondió 200 OK y mostró el resumen de rutas. Esta prueba confirma que la instancia de Express atendió la petición; las operaciones posteriores comprueban el acceso a MongoDB.

7.7. PRUEBA 02: RECHAZAR UNA CONTRASEÑA INCORRECTA.

1. Envié POST a /auth/login con el usuario practica y una contraseña incorrecta.

2. El middleware comparó los valores recibidos con las credenciales configuradas para las pruebas.

3. Comprobé que la respuesta fuera 401 y que no se entregara un token para continuar.

Resultados:

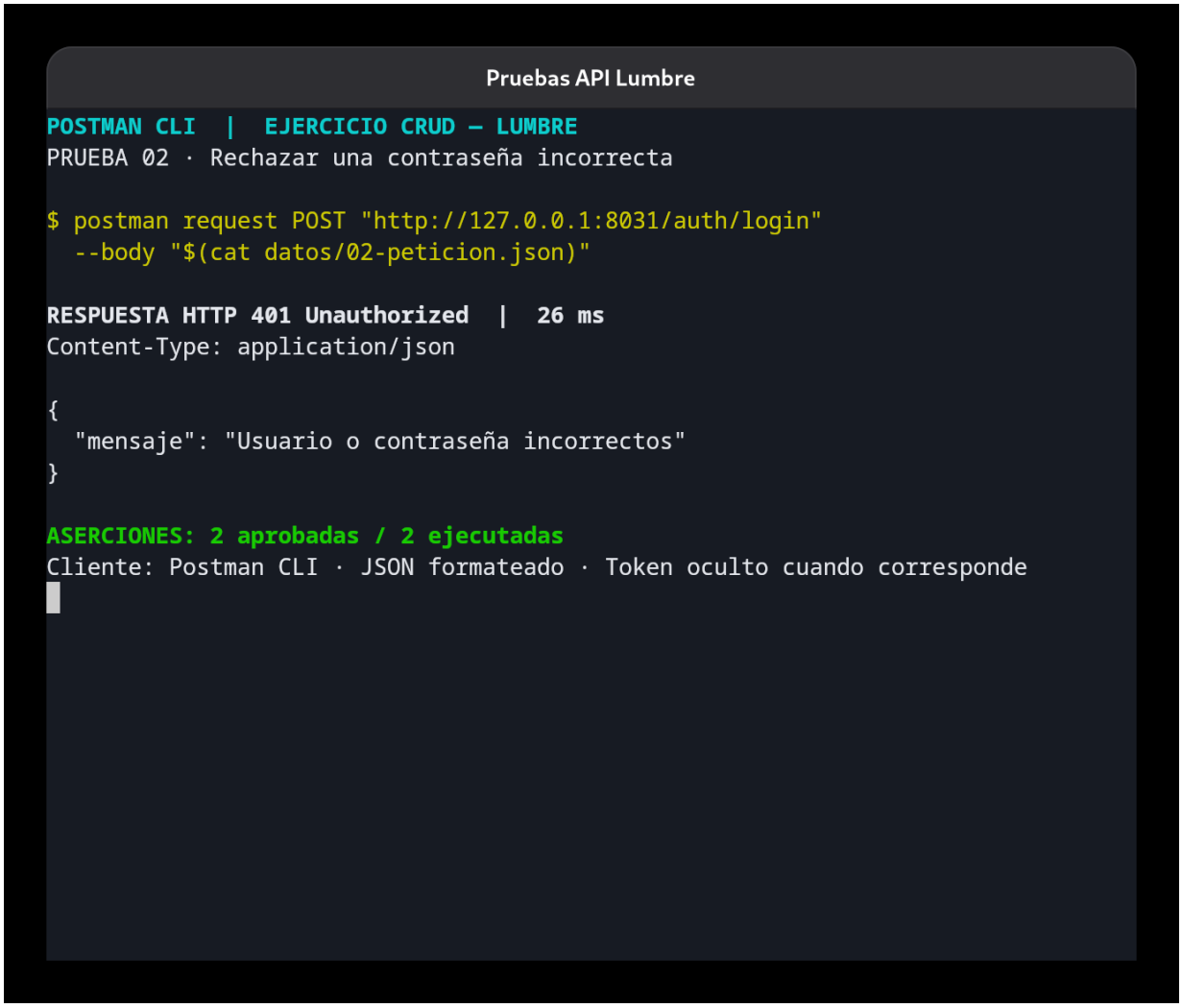


Figura 02. HTTP 401; 26 ms; 2 aserciones aprobadas.

Se obtuvo 401 Unauthorized con el mensaje Usuario o contraseña incorrectos. El rechazo es el resultado esperado de esta prueba negativa.

7.8. PRUEBA 03: OBTENER EL TOKEN JWT DE ACCESO.

1. Envié las credenciales válidas mediante POST a /auth/login.

2. El servidor generó un token firmado que contiene usuario, iat y exp. La expiración se establece una hora después de la emisión; la contraseña no se incluye en su contenido.

3. Guardé el token para las siguientes peticiones y comprobé que la respuesta incluyera una cadena en el campo token.

Resultados:

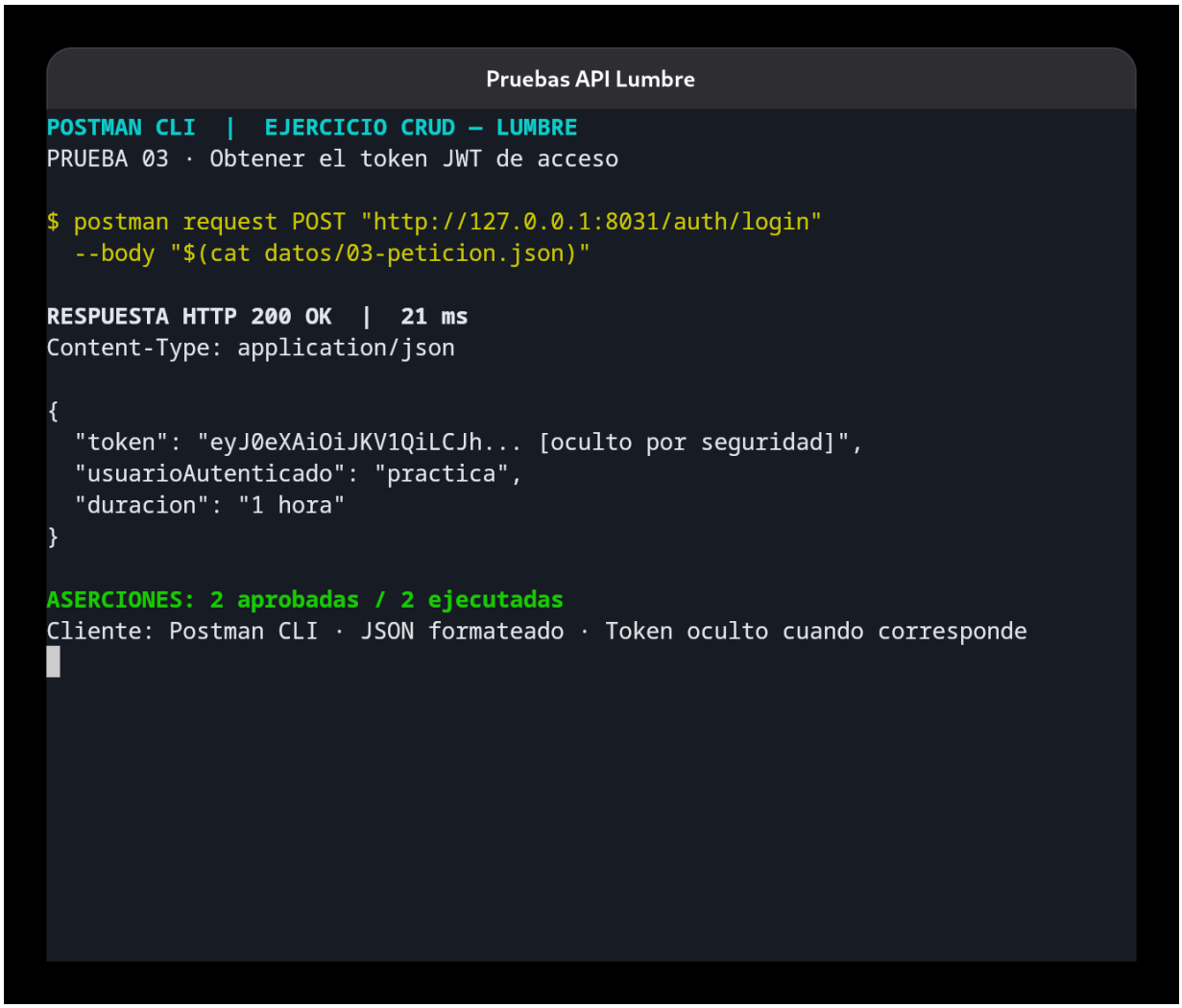


Figura 03. HTTP 200; 21 ms; 2 aserciones aprobadas.

La autenticación devolvió 200 OK. La captura oculta parte del token, pero conserva usuarioAutenticado y la duración informada por la API.

7.9. PRUEBA 04: RECHAZAR UNA PETICIÓN SIN TOKEN.

1. Intenté consultar GET /libros/obtenerVarios sin enviar la cabecera Authorization.

2. El router ejecutó verificarToken antes de llegar al controlador que consulta los libros.

3. Esperé un estado 401 y comprobé que la respuesta estuviera en formato JSON.

Resultados:

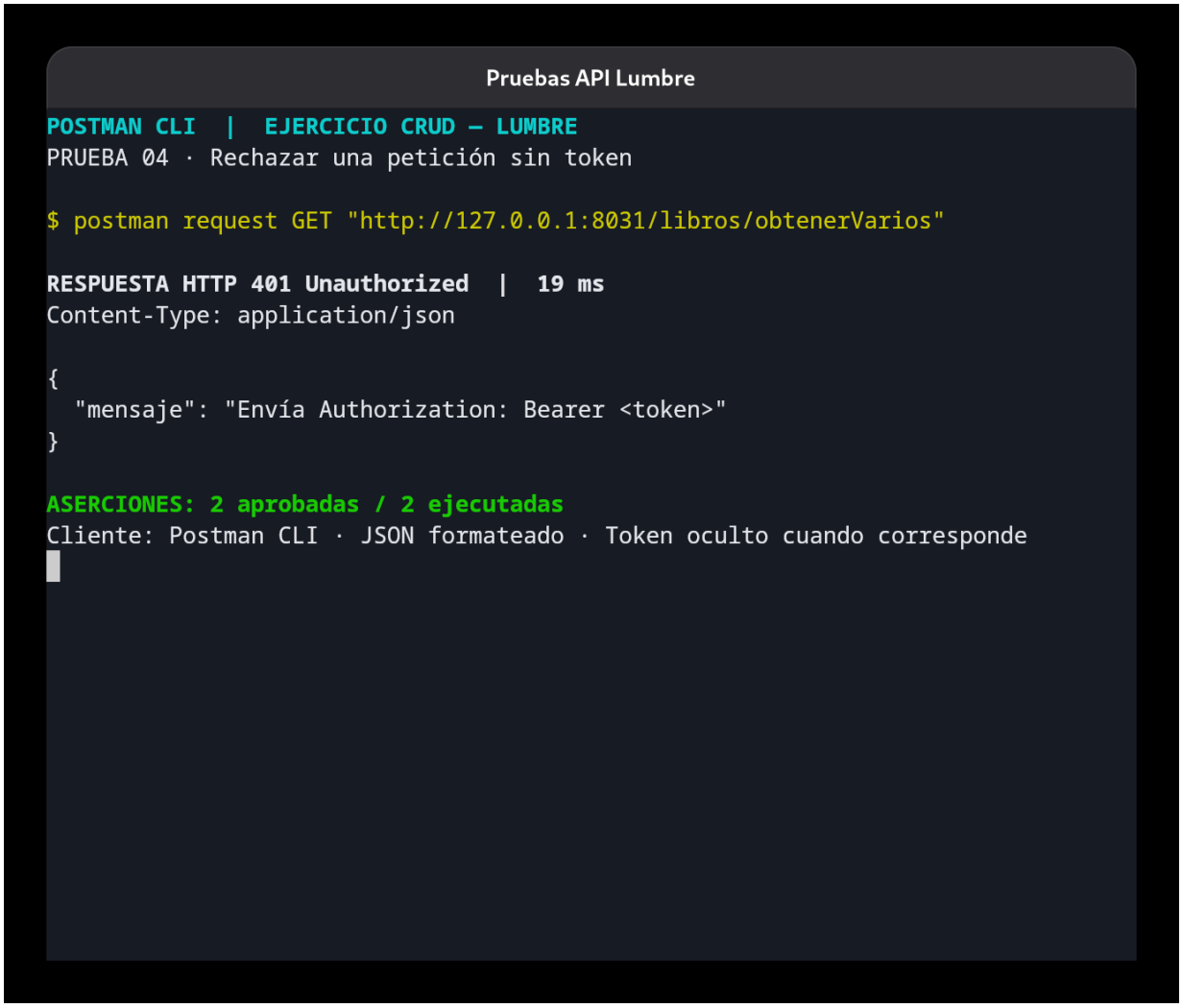


Figura 04. HTTP 401; 19 ms; 2 aserciones aprobadas.

La API respondió 401 e indicó que se debe enviar Authorization: Bearer <token>. El middleware bloqueó la consulta sin devolver el catálogo.

7.10. PRUEBA 05: RECHAZAR UN TOKEN VENCIDO.

1. El entorno de pruebas preparó un JWT firmado con su misma clave temporal y con exp establecido en una fecha pasada.

2. Envié ese token como Bearer en GET /generos/obtenerVarios.

3. Verifiqué que el middleware identificara la expiración y devolviera 403.

Resultados:

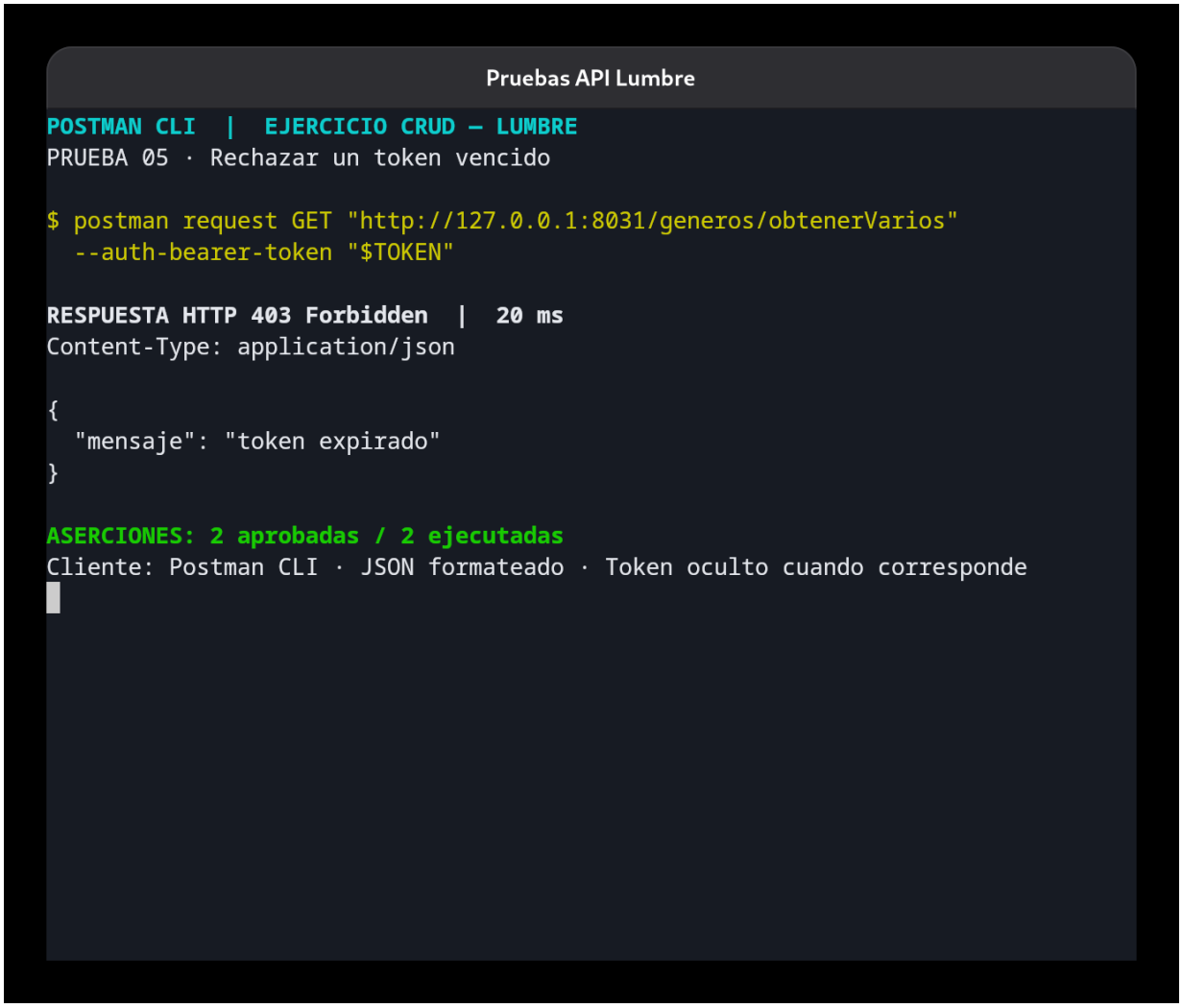


Figura 05. HTTP 403; 20 ms; 2 aserciones aprobadas.

Se recibió 403 Forbidden y el mensaje token expirado. Así se distinguió un token vencido de una petición que simplemente no envía token.

7.11. PRUEBA 06: CREAR UN GÉNERO LITERARIO.

1. Con el token válido, envié POST a /generos/falta con nombre Tecnología y la descripción Libros sobre programación y sistemas.

2. El servicio validó el nombre, verificó que no estuviera registrado y guardó un documento Genero.

3. Comprobé el estado 201 y conservé el _id de la respuesta como idGenero para las consultas y modificaciones posteriores.

Resultados:

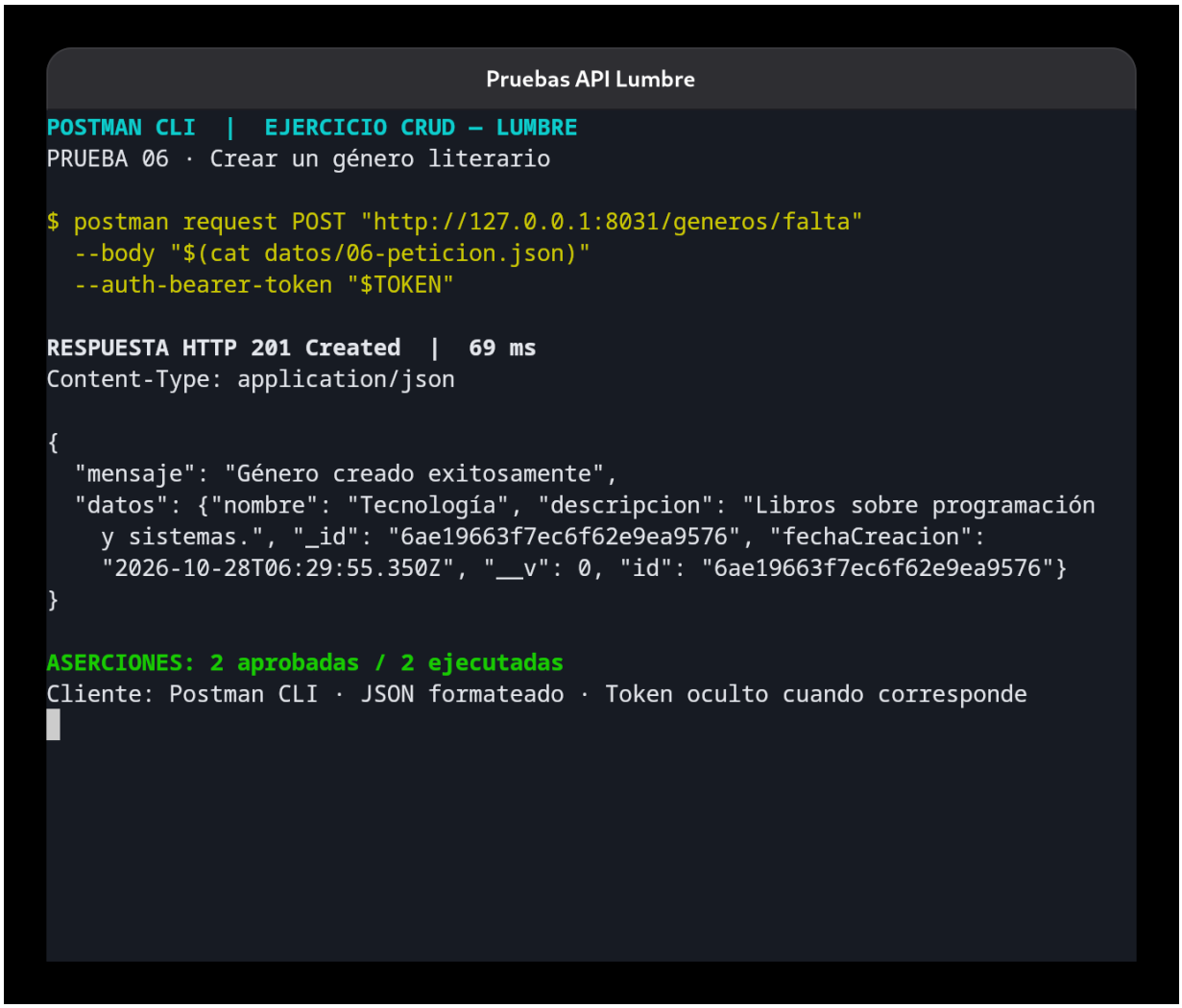


Figura 06. HTTP 201; 69 ms; 2 aserciones aprobadas.

Se creó el género en MongoDB y se devolvió 201 Created. La respuesta contiene los datos almacenados y el identificador generado.

7.12. PRUEBA 07: CONSULTAR TODOS LOS GÉNEROS.

1. Envié GET a /generos/obtenerVarios con el token de acceso.

2. El servicio ejecutó la consulta de géneros e incluyó la relación virtual libros.

3. Comprobé que datos fuera una lista con un registro, porque la base temporal inició vacía y se había creado un solo género.

Resultados:

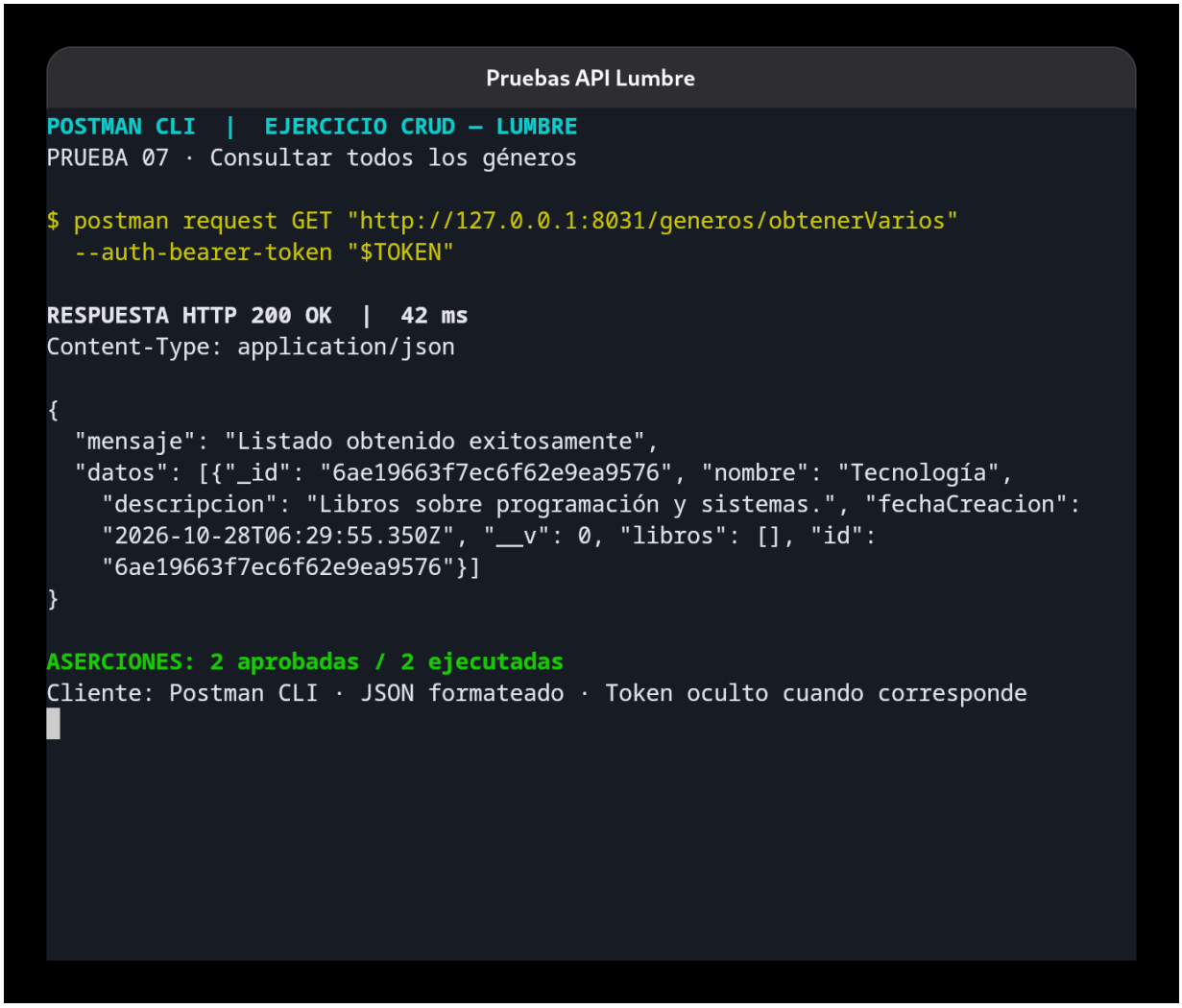


Figura 07. HTTP 200; 42 ms; 2 aserciones aprobadas.

La respuesta 200 OK contiene Tecnología. En este momento libros está vacío porque todavía no se había registrado ningún título asociado.

7.13. PRUEBA 08: CONSULTAR UN GÉNERO POR ID.

1. Sustituí {idGenero} por el _id recibido en el alta y envié GET a /generos/obtenerUno/{idGenero}.

2. El servicio validó el formato ObjectId y buscó el documento correspondiente.

3. Comprobé el estado 200 y el nombre Tecnología en los datos devueltos.

Resultados:

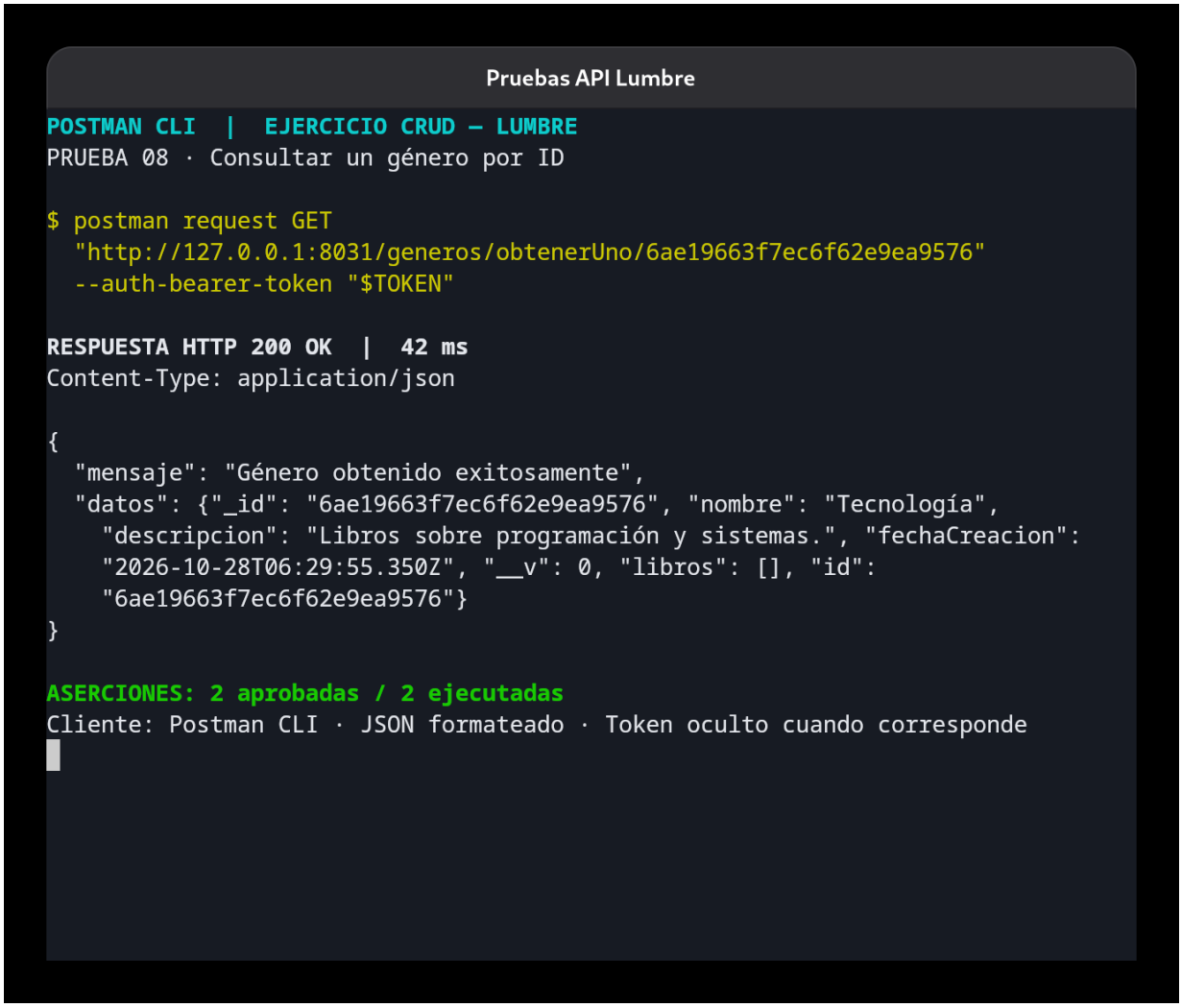


Figura 08. HTTP 200; 42 ms; 2 aserciones aprobadas.

Se recuperó un documento concreto. Esta consulta usa un identificador único, mientras que obtenerVarios devuelve una lista de registros.

7.14. PRUEBA 09: ACTUALIZAR UN GÉNERO.

1. Envié PUT a /generos/baja/{idGenero} con el nombre Informática y la descripción Programación, bases de datos y redes.

2. El servicio aceptó los campos permitidos y aplicó la actualización con validaciones de Mongoose.

3. Comprobé el estado 200 y que el nombre devuelto cambiara a Informática, conservando el mismo _id.

Resultados:

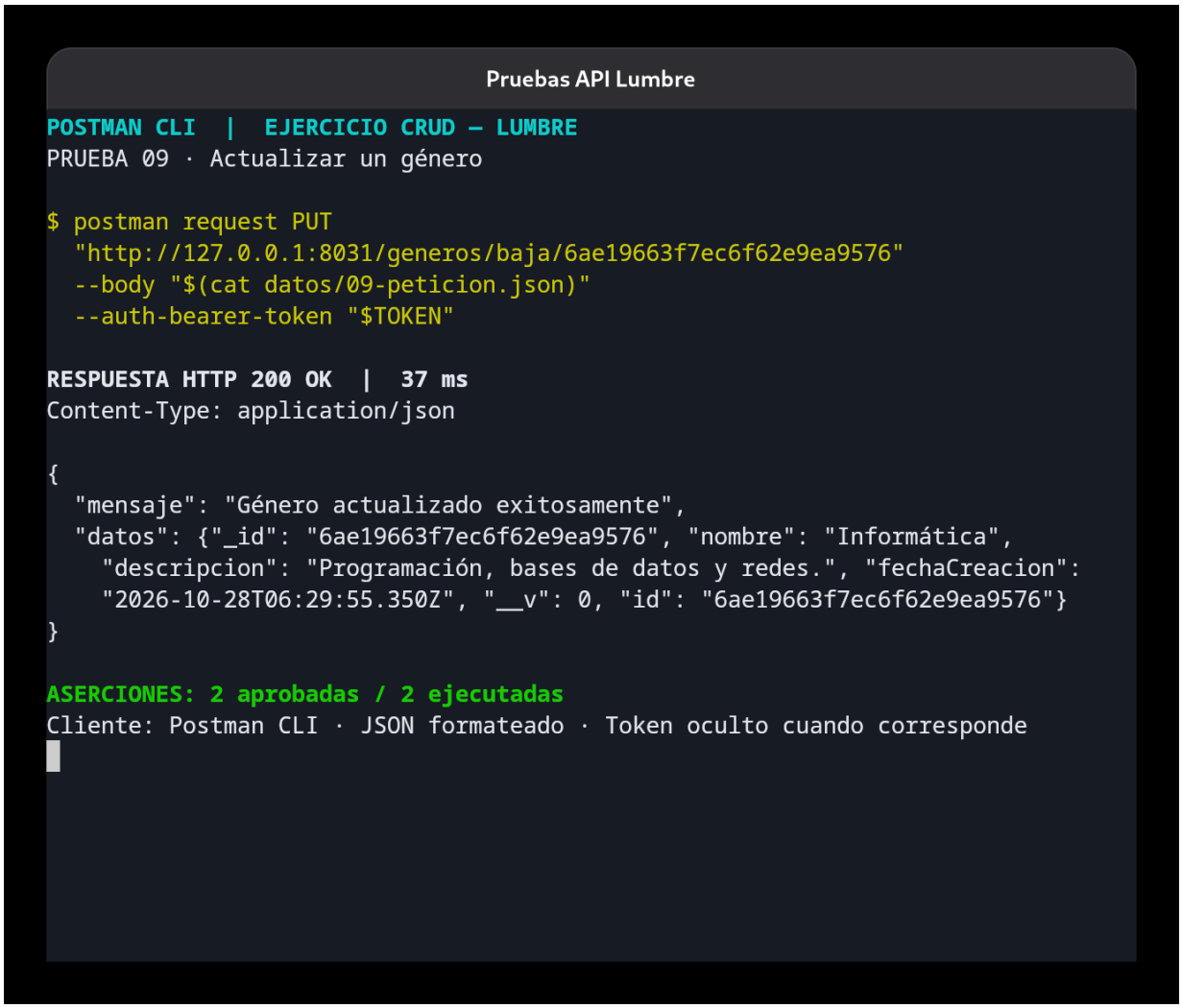


Figura 09. HTTP 200; 37 ms; 2 aserciones aprobadas.

El género se actualizó correctamente. Aunque la ruta heredada contiene baja, aquí el método PUT modifica el documento; la eliminación se realiza en /eliminar/:id.

7.15. PRUEBA 10: CREAR UN LIBRO VINCULADO AL GÉNERO.

1. Envié POST a /libros/falta con el título Fundamentos de APIs REST, autor Ana Torres, estante A-01, tres ejemplares y el idGenero obtenido.

2. El servicio comprobó los campos obligatorios y que el género existiera antes de guardar el libro.

3. Verifiqué el estado 201 y el género Informática en la respuesta; guardé el nuevo _id como idLibro.

Resultados:

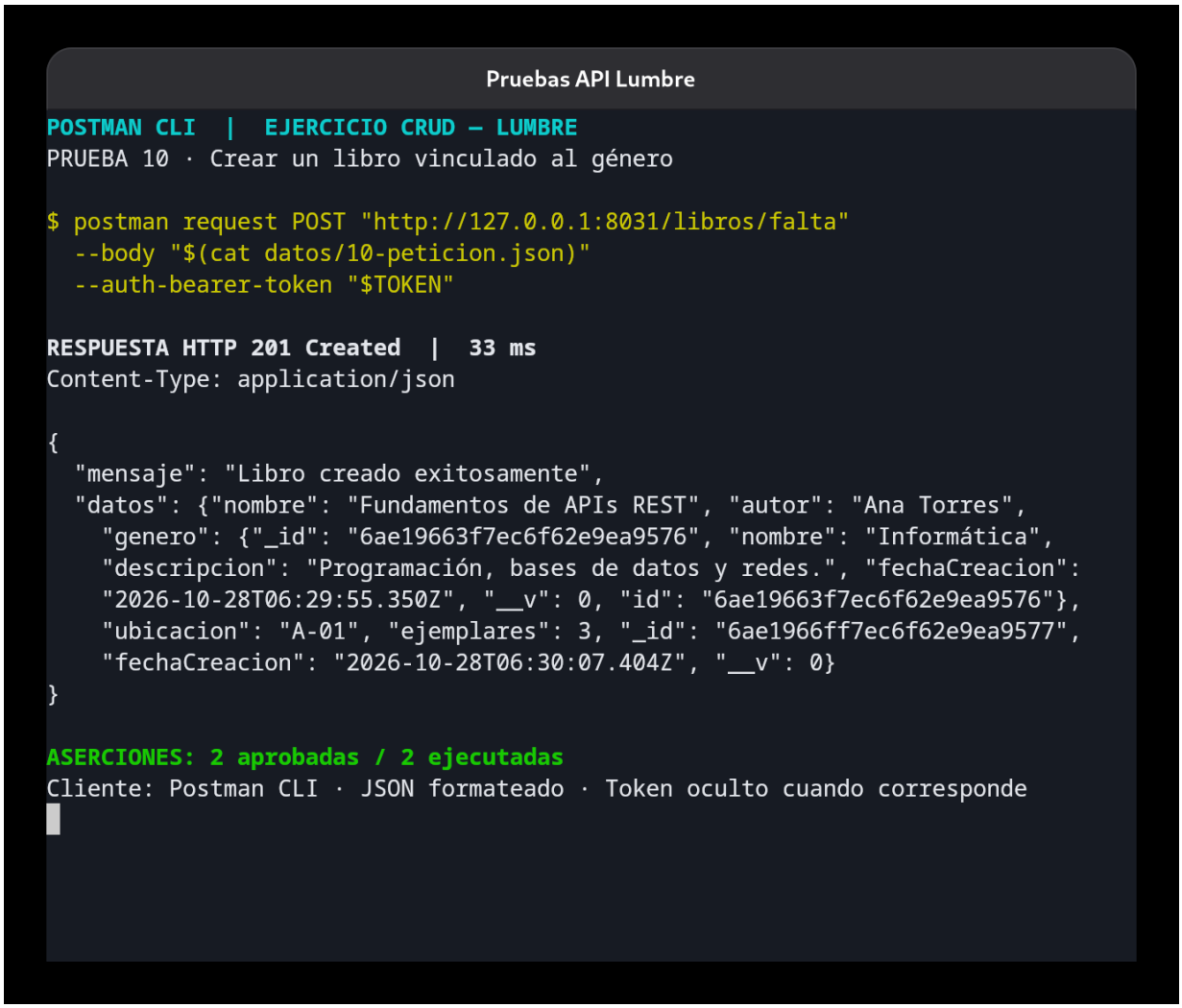


Figura 10. HTTP 201; 33 ms; 2 aserciones aprobadas.

El libro quedó almacenado y vinculado al género. La respuesta incorpora el documento del género mediante populate, lo que permite revisar la relación creada.

7.16. PRUEBA 11: CONSULTAR EL CATÁLOGO DE LIBROS.

1. Envié GET a /libros/obtenerVarios utilizando el token válido.

2. El servicio consultó Libro.find() y pobló la referencia genero de cada resultado.

3. Comprobé que la lista incluyera exactamente un libro y que su información coincidiera con el alta anterior.

Resultados:

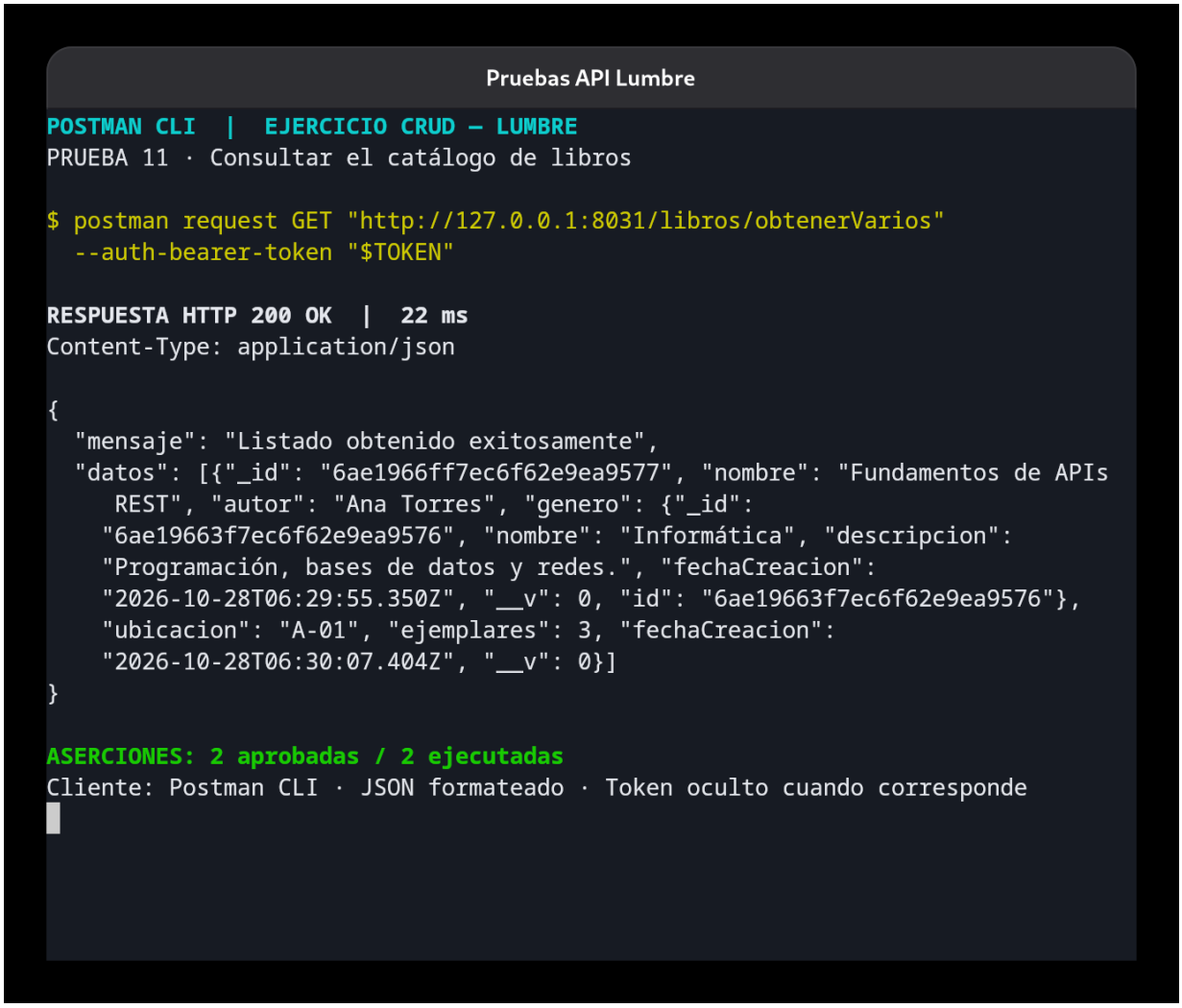


Figura 11. HTTP 200; 22 ms; 2 aserciones aprobadas.

La API devolvió 200 OK con el libro de prueba. La consulta posterior al alta confirma que el registro puede recuperarse desde MongoDB.

7.17. PRUEBA 12: CONSULTAR UN LIBRO POR ID.

1. Envié GET a /libros/obtenerUno/{idLibro}, sustituyendo el parámetro con el identificador devuelto al crear el libro.

2. El servicio validó el ObjectId y buscó el documento con su género relacionado.

3. Verifiqué el estado 200 y que el campo autor contuviera Ana Torres.

Resultados:

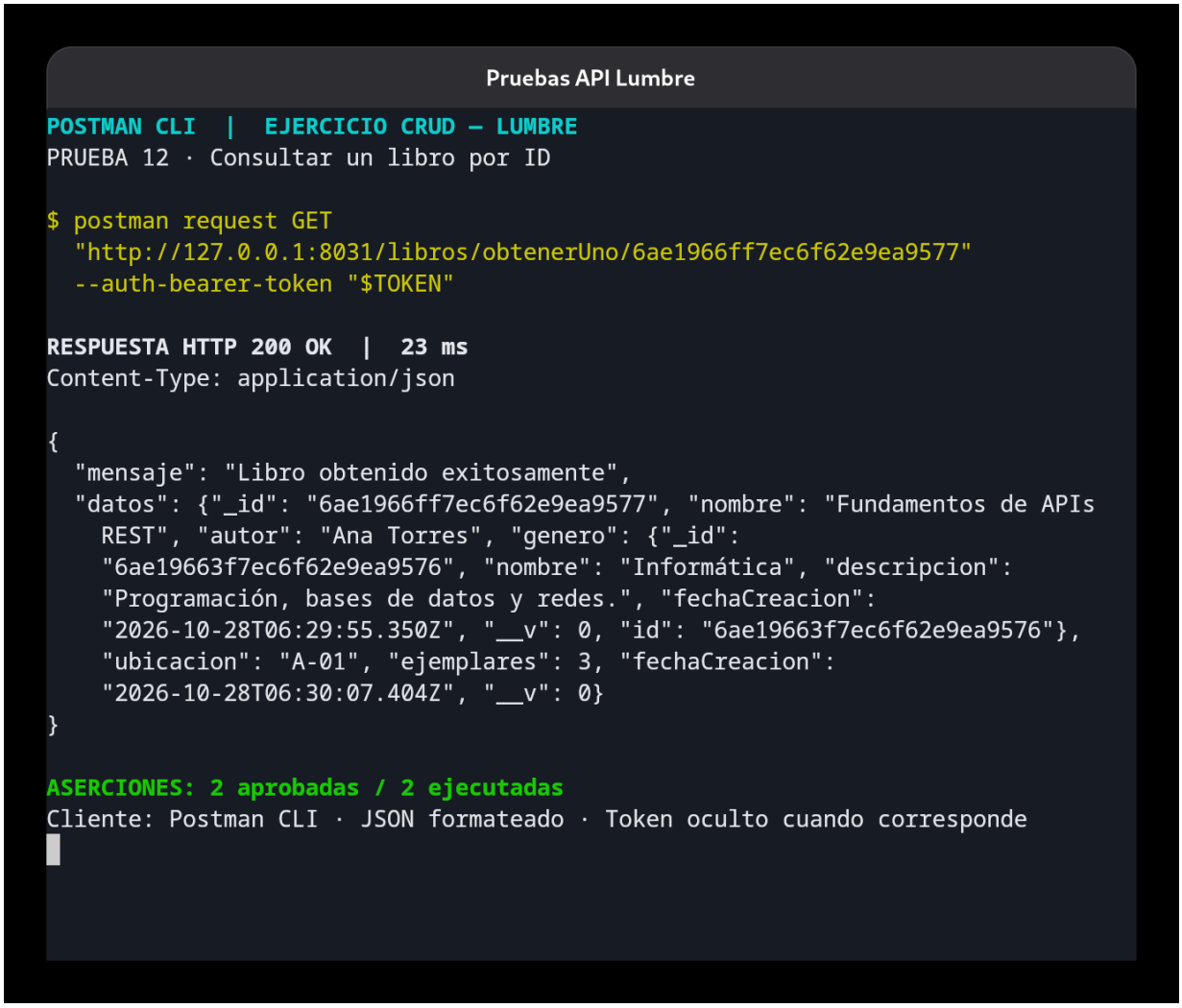


Figura 12. HTTP 200; 23 ms; 2 aserciones aprobadas.

La respuesta devuelve el título, autor, género, ubicación y ejemplares del libro solicitado. El identificador permite seleccionar un registro sin depender de su nombre.

7.18. PRUEBA 13: ACTUALIZAR ESTANTE Y EJEMPLARES.

1. Envié PUT a /libros/baja/{idLibro} con ubicacion B-02 y ejemplares igual a 5.

2. El servicio revisó los campos permitidos y validó que la cantidad fuera un entero no negativo.

3. Comprobé en la respuesta tanto el nuevo estante como los cinco ejemplares; el identificador del libro se mantuvo.

Resultados:

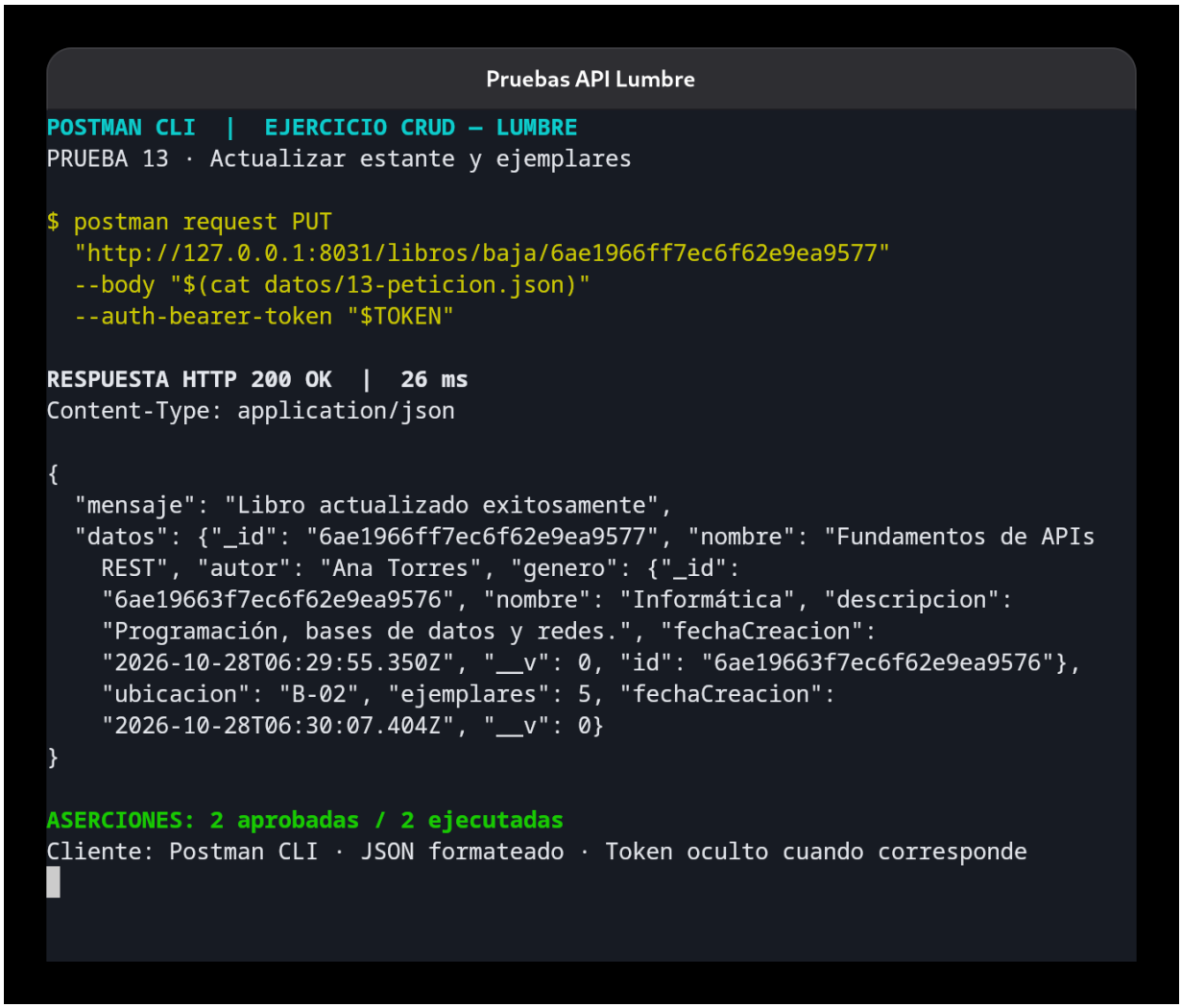


Figura 13. HTTP 200; 26 ms; 2 aserciones aprobadas.

La actualización respondió 200 OK. El título y el autor se conservaron porque el cuerpo de la petición solo modificó ubicación y ejemplares.

7.19. PRUEBA 14: PROTEGER UN GÉNERO CON LIBROS ASOCIADOS.

1. Intenté DELETE /generos/eliminar/{idGenero} mientras el libro de prueba todavía apuntaba a ese género.

2. El servicio contó los libros asociados antes de ejecutar la eliminación.

3. Comprobé el estado 400 y que el mensaje explicara que existe un libro asociado.

Resultados:

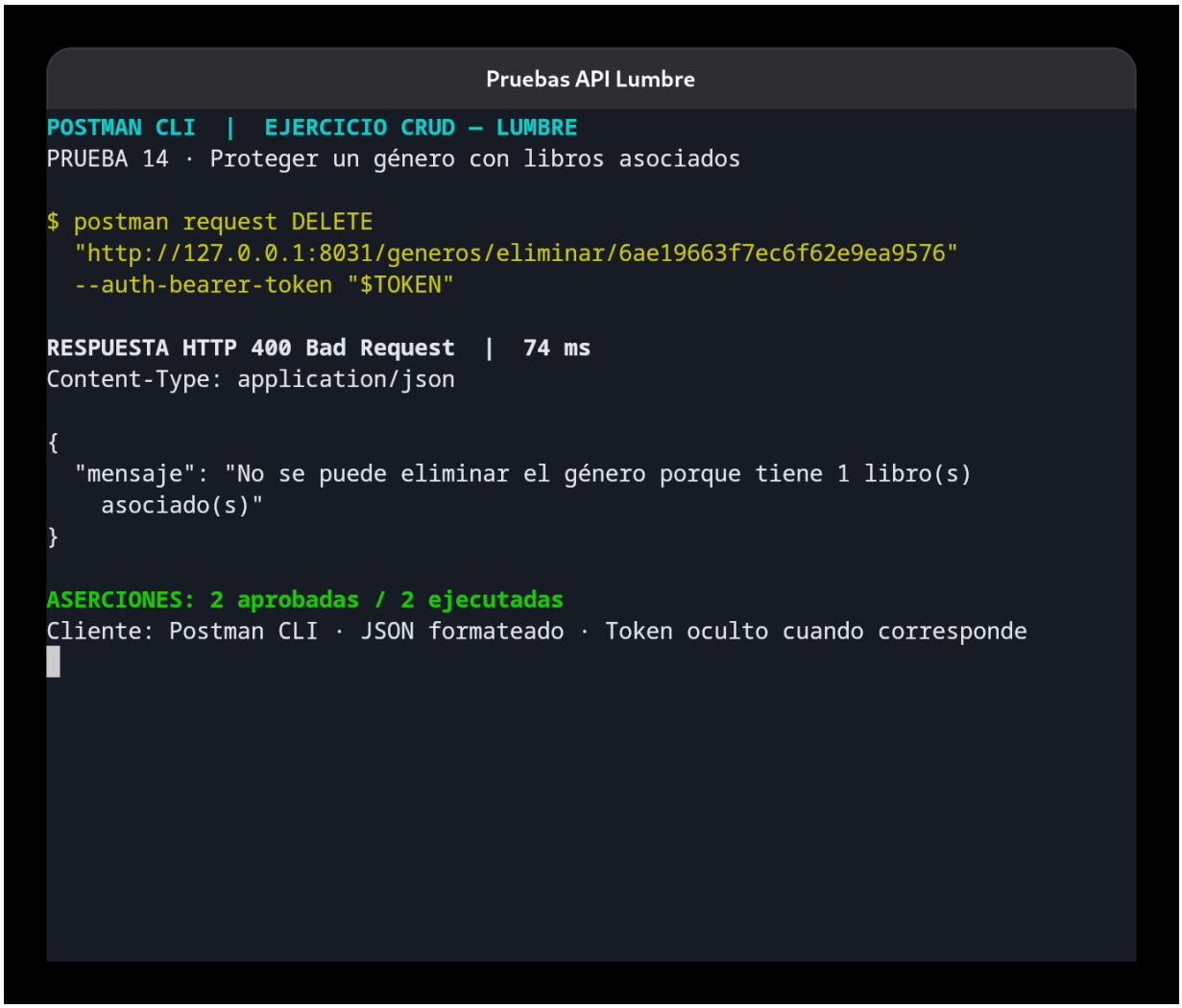


Figura 14. HTTP 400; 74 ms; 2 aserciones aprobadas.

El género no se eliminó. La validación impidió borrar un documento que aún participa en la relación; más adelante se repite la baja después de eliminar el libro.

7.20. PRUEBA 15: RECHAZAR EJEMPLARES NEGATIVOS.

1. Envié PUT a /libros/baja/{idLibro} con ejemplares igual a -2.

2. El servicio detectó que el valor era negativo y detuvo la actualización.

3. Verifiqué el estado 400 y el mensaje que exige un entero no negativo. El registro eliminado en la prueba 17 permite observar que se conservaron cinco ejemplares.

Resultados:

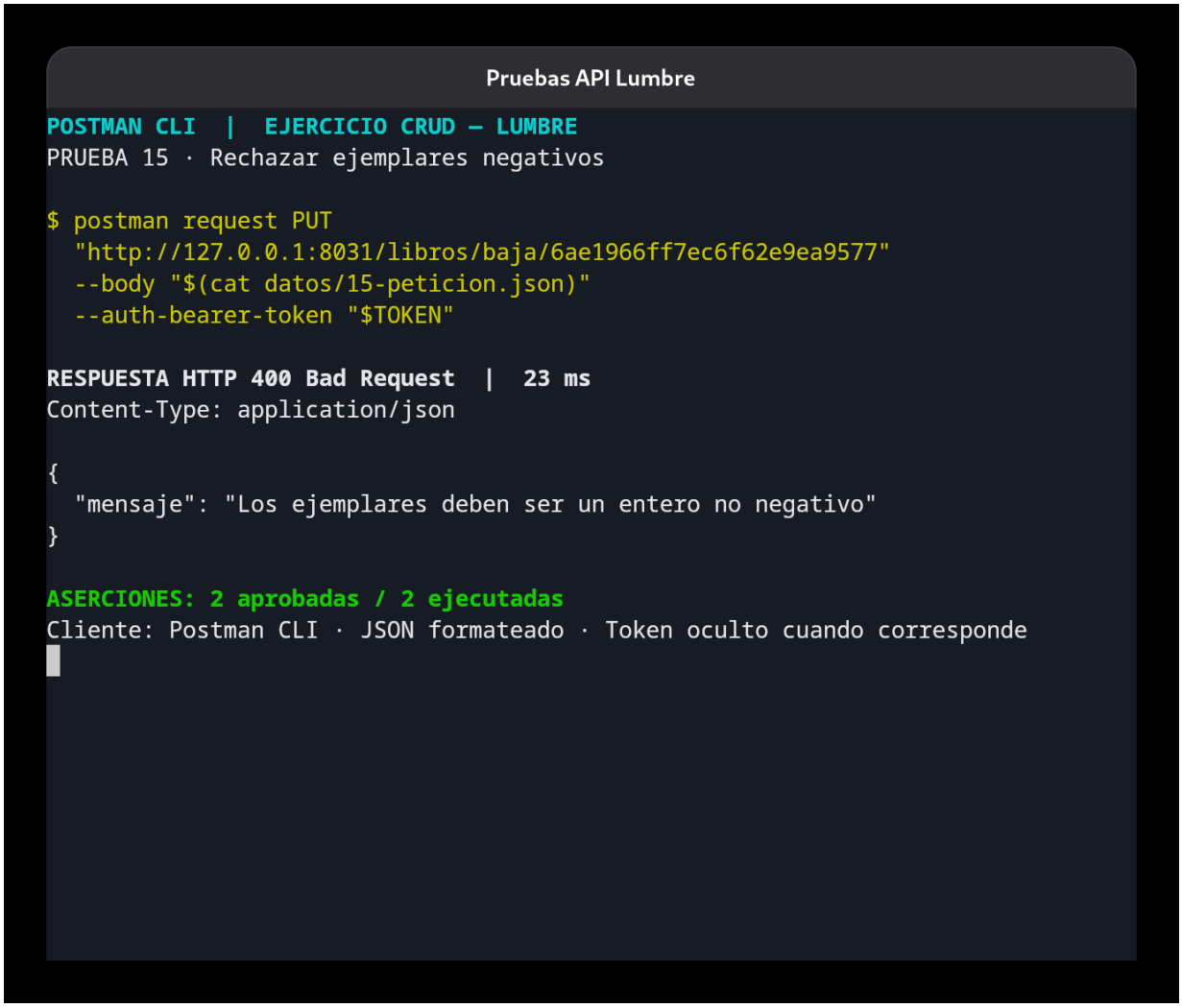


Figura 15. HTTP 400; 23 ms; 2 aserciones aprobadas.

La petición inválida no cambió la cantidad almacenada. La validación evita registrar existencias negativas dentro de la biblioteca.

7.21. PRUEBA 16: RECHAZAR UN ID CON FORMATO INVÁLIDO.

1. Envié GET a /libros/obtenerUno/id-invalido para probar un parámetro que no es un ObjectId.

2. El servicio revisó el formato antes de realizar la búsqueda del libro.

3. Comprobé que la API respondiera 400 y explicara que el identificador no era válido.

Resultados:

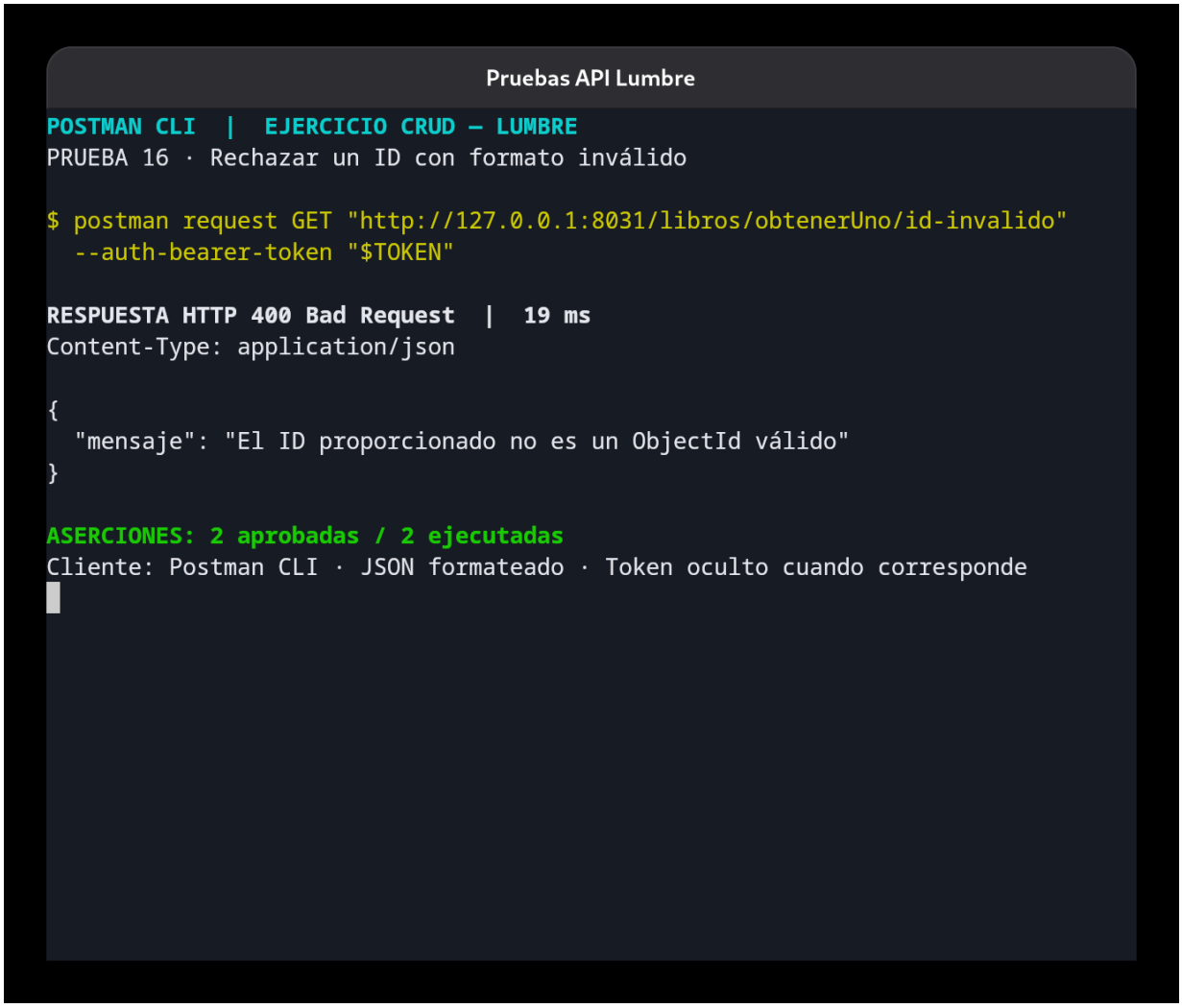


Figura 16. HTTP 400; 19 ms; 2 aserciones aprobadas.

Se obtuvo un error de entrada controlado. Este caso se diferencia del 404: aquí el formato del identificador es incorrecto, no se trata de un documento válido que dejó de existir.

7.22. PRUEBA 17: ELIMINAR UN LIBRO.

1. Envié DELETE a /libros/eliminar/{idLibro} con el token válido.

2. El servicio localizó y eliminó el documento mediante findByIdAndDelete.

3. Verifiqué el estado 200 y el título del registro devuelto. La respuesta también conserva B-02 y cinco ejemplares como últimos valores guardados.

Resultados:

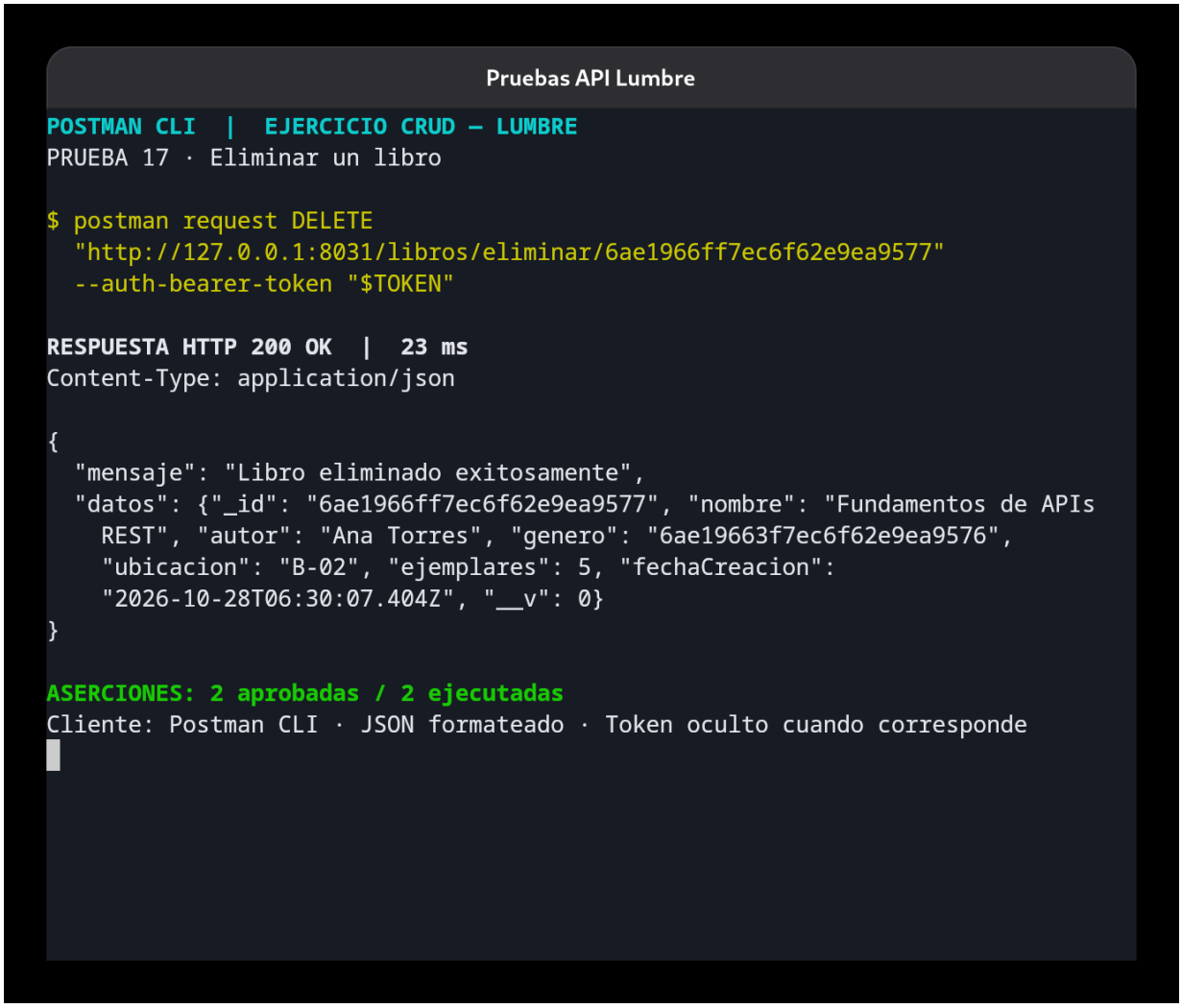


Figura 17. HTTP 200; 23 ms; 2 aserciones aprobadas.

El libro de prueba fue eliminado. La respuesta permite identificar qué documento se borró; la siguiente consulta comprueba que ya no puede recuperarse.

7.23. PRUEBA 18: COMPROBAR QUE EL LIBRO YA NO EXISTE.

1. Repetí GET /libros/obtenerUno/{idLibro} utilizando exactamente el identificador del libro eliminado.

2. El formato del ID siguió siendo válido, pero la consulta ya no encontró un documento asociado.

3. Verifiqué el estado 404 y la respuesta JSON del servidor.

Resultados:

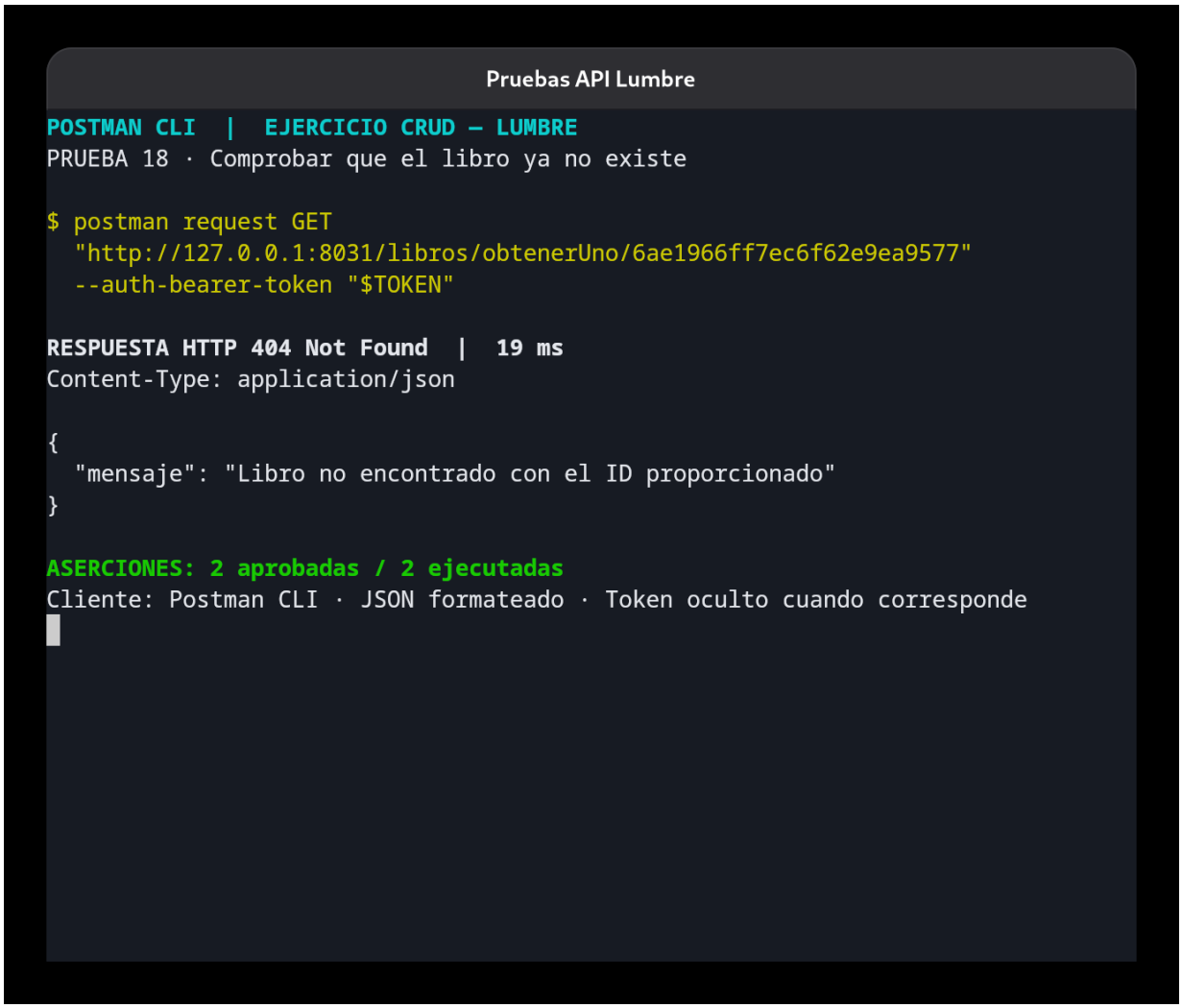


Figura 18. HTTP 404; 19 ms; 2 aserciones aprobadas.

La API devolvió 404 Not Found. Esta lectura posterior a DELETE confirma la eliminación del registro en la base temporal.

7.24. PRUEBA 19: ELIMINAR EL GÉNERO SIN LIBROS ASOCIADOS.

1. Repetí DELETE /generos/eliminar/{idGenero} después de eliminar el libro.

2. El servicio volvió a contar las referencias y esta vez no encontró libros asociados.

3. Comprobé el estado 200 y los datos del género eliminado.

Resultados:

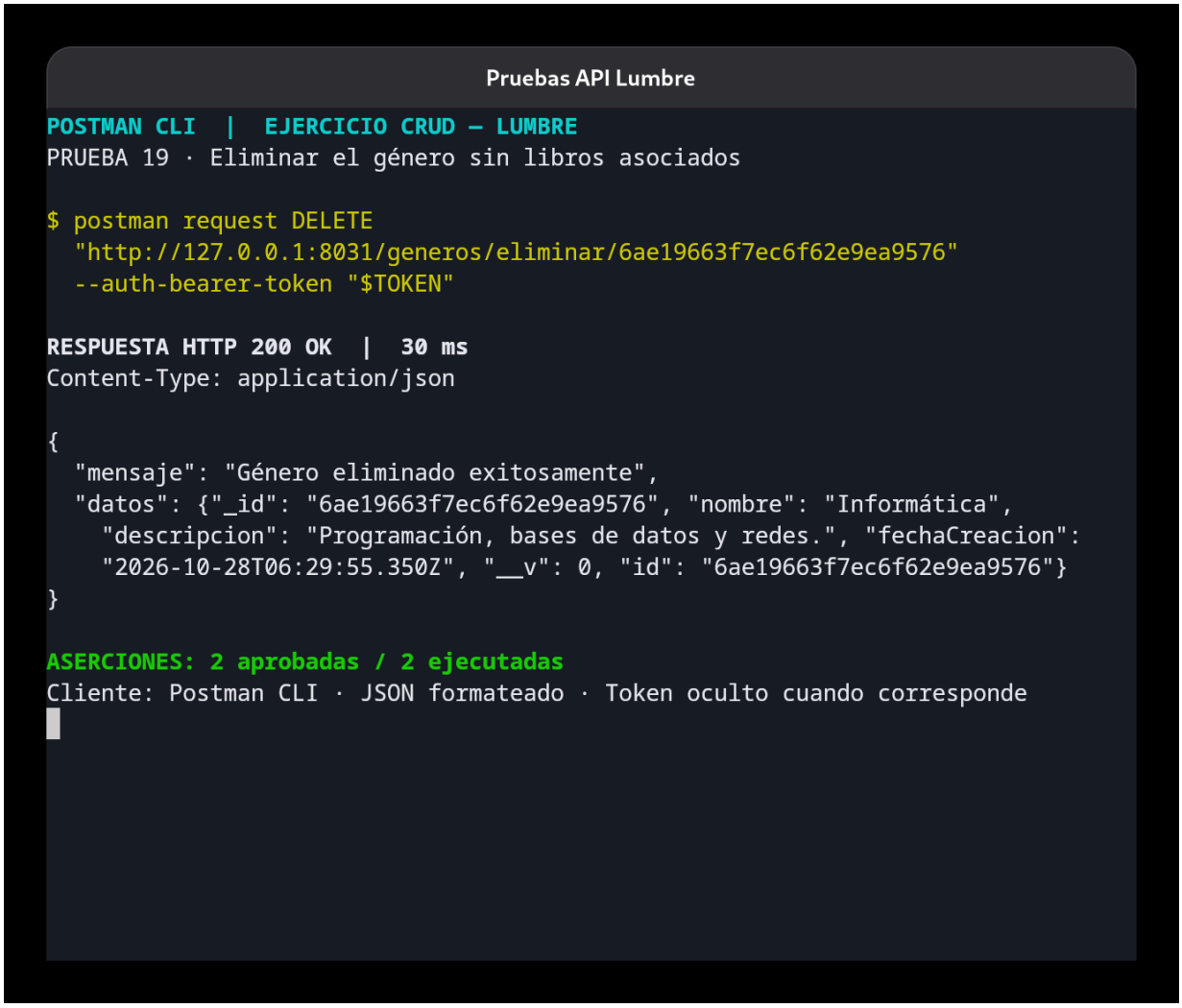


Figura 19. HTTP 200; 30 ms; 2 aserciones aprobadas.

Ahora la eliminación fue permitida. La comparación con la prueba 14 muestra que el resultado depende de si existen libros vinculados al género.

7.25. PRUEBA 20: COMPROBAR EL CATÁLOGO VACÍO.

1. Consulté GET /libros/obtenerVarios una vez concluida la eliminación.

2. Revisé la propiedad datos y comprobé mediante una aserción que su longitud fuera cero.

3. Verifiqué que la API respondiera 200, porque consultar una colección vacía es una operación válida.

Resultados:

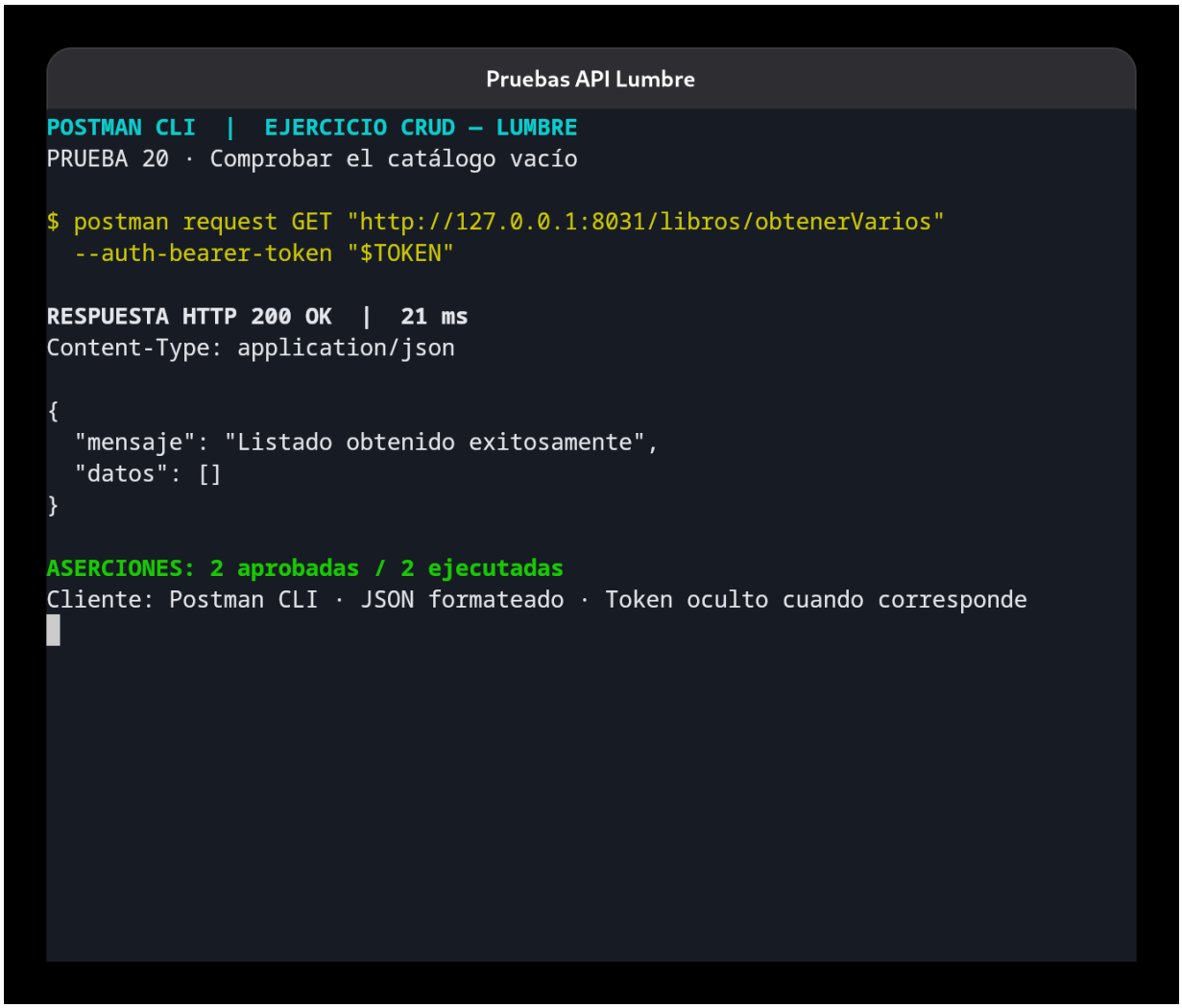


Figura 20. HTTP 200; 21 ms; 2 aserciones aprobadas.

La respuesta contiene datos: []. La ausencia de libros no es un error del servidor; es el estado esperado del catálogo al terminar este recorrido de pruebas.

7.26. PRUEBA 21: COMPROBAR LA ELIMINACIÓN DEL GÉNERO.

1. Envié GET a /generos/obtenerUno/{idGenero} usando el identificador del género eliminado.

2. El servicio no encontró el documento y generó la respuesta correspondiente.

3. Comprobé el estado 404. Con esta consulta se cerró el ciclo de crear, leer, actualizar y eliminar el género.

Resultados:

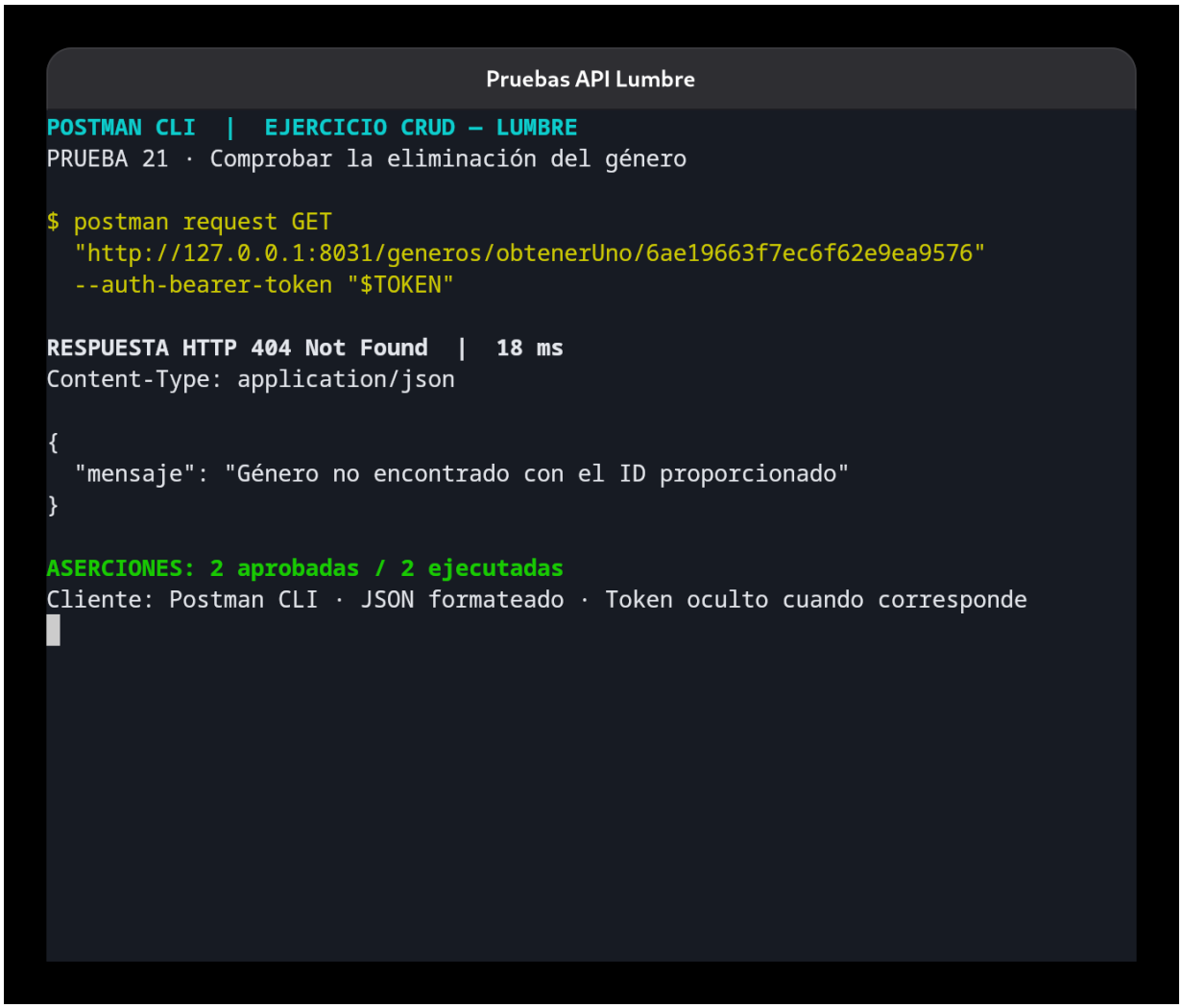


Figura 21. HTTP 404; 18 ms; 2 aserciones aprobadas.

La respuesta 404 confirma que el género dejó de estar disponible. Los dos registros creados para la práctica fueron eliminados sin modificar el catálogo habitual.

7.27. PRUEBA 22: BUSCAR OBRAS: OPEN LIBRARY SIMULADO.

1. En la instancia del puerto 8032 configuré OPENLIBRARY_API_URL para apuntar a la simulación local de Postman.

2. Envié GET /servicios/libros?q=fox. El backend ejecutó Axios contra /search.json y transformó docs al formato de la biblioteca.

3. Comprobé el estado 200 y el título Fantastic Mr. Fox que devuelve el escenario simulado.

Resultados:

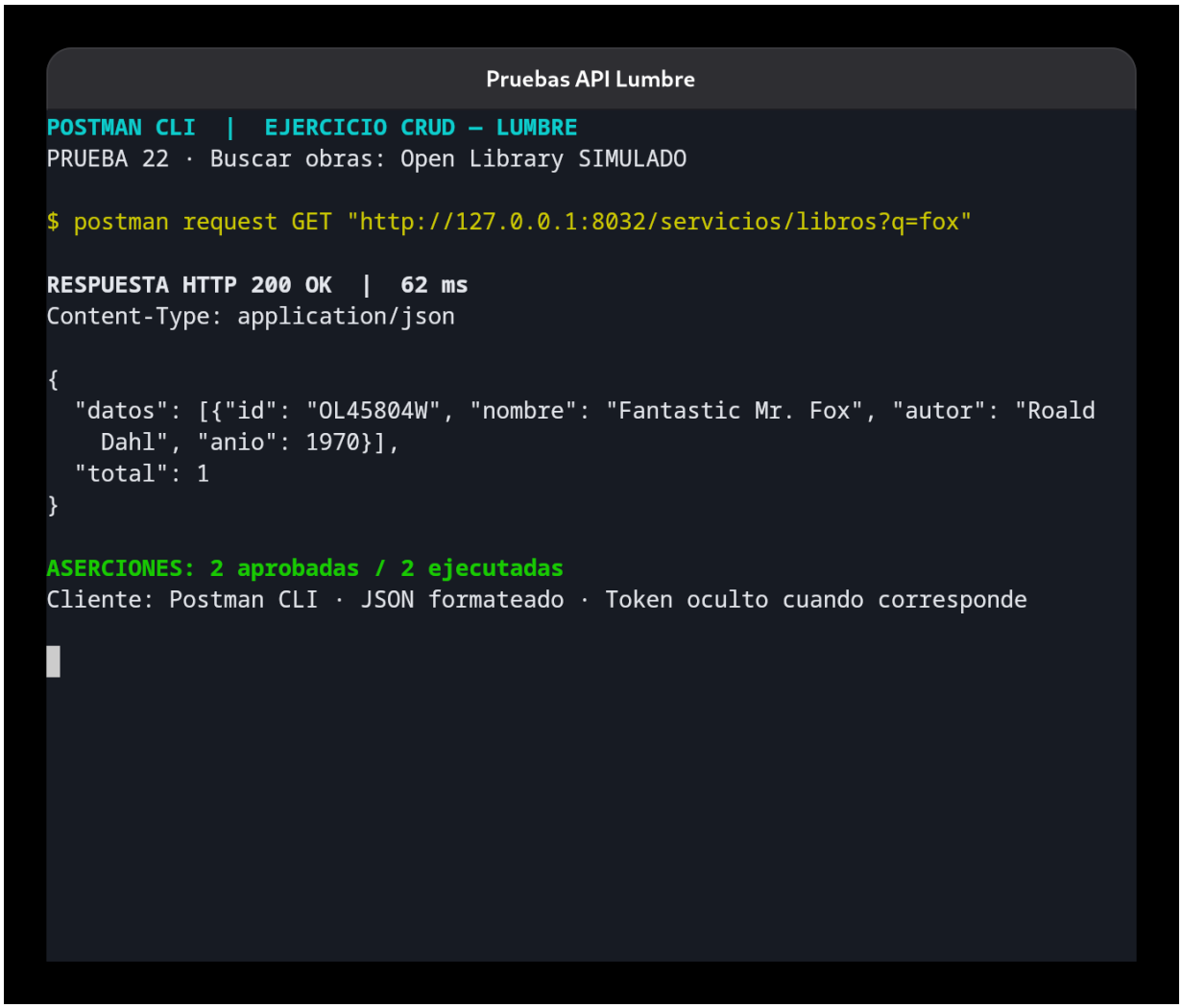


Figura 22. HTTP 200; 62 ms; 2 aserciones aprobadas.

La ruta local y el cliente Axios se ejecutaron realmente. La respuesta de Open Library fue simulada; esta prueba no demuestra disponibilidad del proveedor en Internet.

7.28. PRUEBA 23: CONSULTAR FICHA: PROVEEDOR SIMULADO.

1. Envié GET a /servicios/libros/OL45804W en la instancia conectada a la simulación.

2. Axios consultó /works/OL45804W.json. El escenario devuelve el título sin el campo description.

3. Comprobé el estado 200 y el mensaje alternativo que utiliza el servicio cuando falta una descripción.

Resultados:

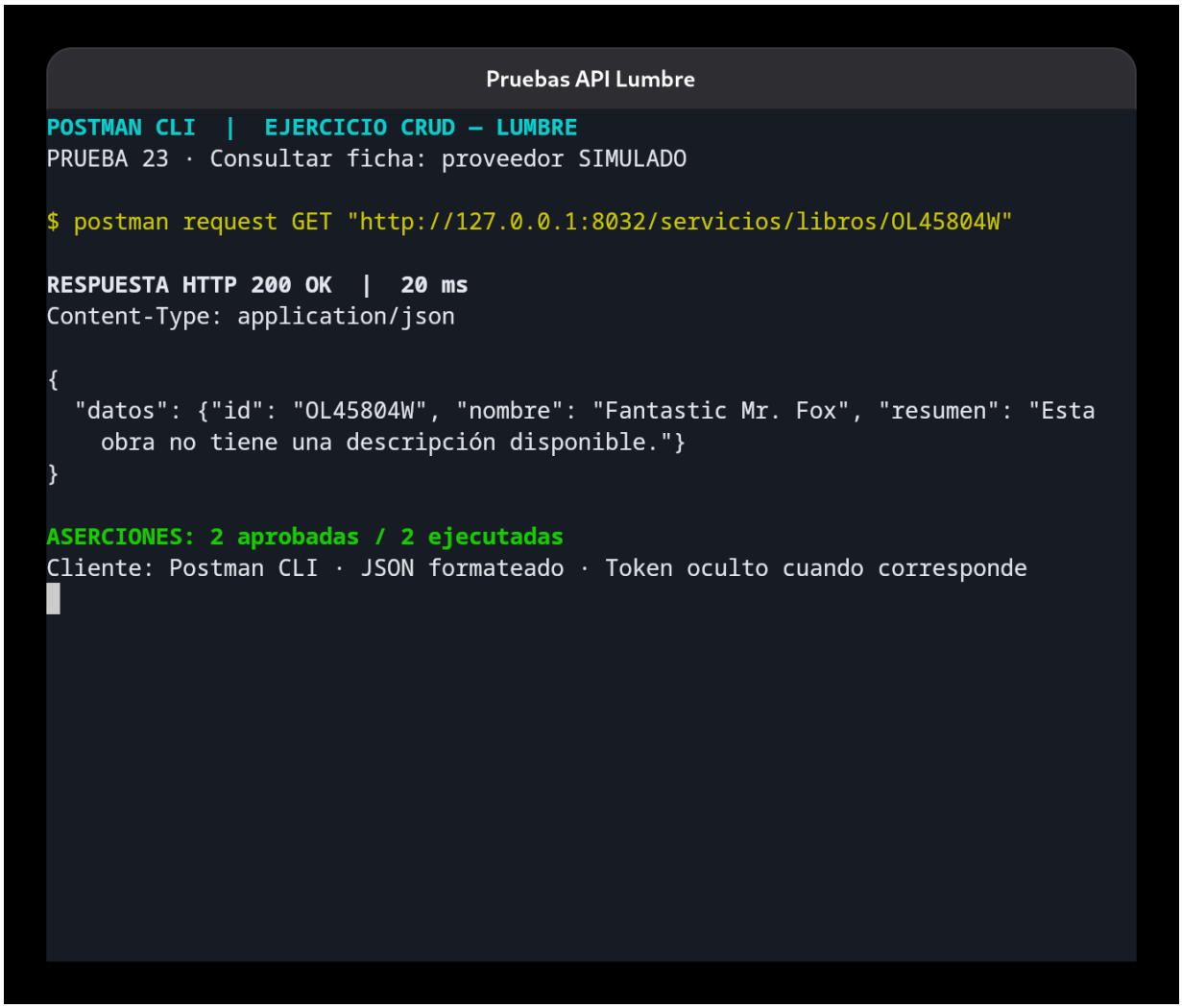


Figura 23. HTTP 200; 20 ms; 2 aserciones aprobadas.

El backend respondió sin fallar por la ausencia de un campo opcional. La prueba muestra cómo el servicio adapta la ficha externa antes de entregarla al cliente.

7.29. PRUEBA 24: OBRA INEXISTENTE: PROVEEDOR SIMULADO.

1. Envié GET a /servicios/libros/OL999999W, un identificador con formato válido que el proveedor simulado no encuentra.

2. La simulación respondió 404 a la petición de Axios.

3. Verifiqué que el router local conservara el estado 404 y devolviera el mensaje Obra no encontrada.

Resultados:

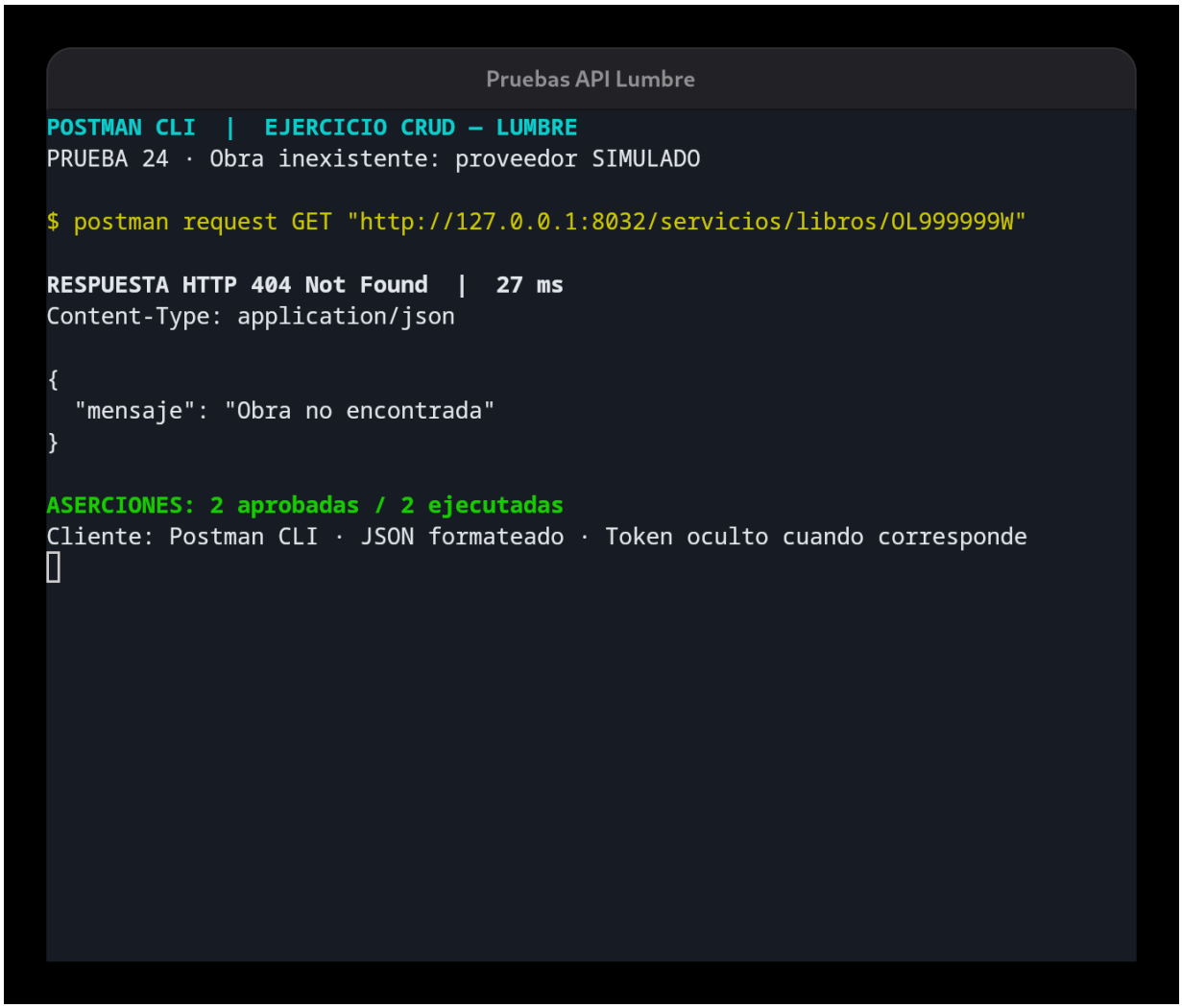


Figura 24. HTTP 404; 27 ms; 2 aserciones aprobadas.

El error del proveedor se presentó como una respuesta controlada de la API. Se comprobó el manejo de una obra inexistente sin depender del catálogo real de Open Library.

7.30. PRUEBA 25: FALLA DEL PROVEEDOR: RESPUESTA 502.

1. Envié GET /servicios/libros?q=upstream-error. Ese término activa deliberadamente una respuesta 503 en la simulación.

2. Axios recibió el error y el router de Lumbre ejecutó su bloque de manejo de excepciones.

3. Comprobé que la API local respondiera 502 y un mensaje comprensible sobre la indisponibilidad del proveedor.

Resultados:

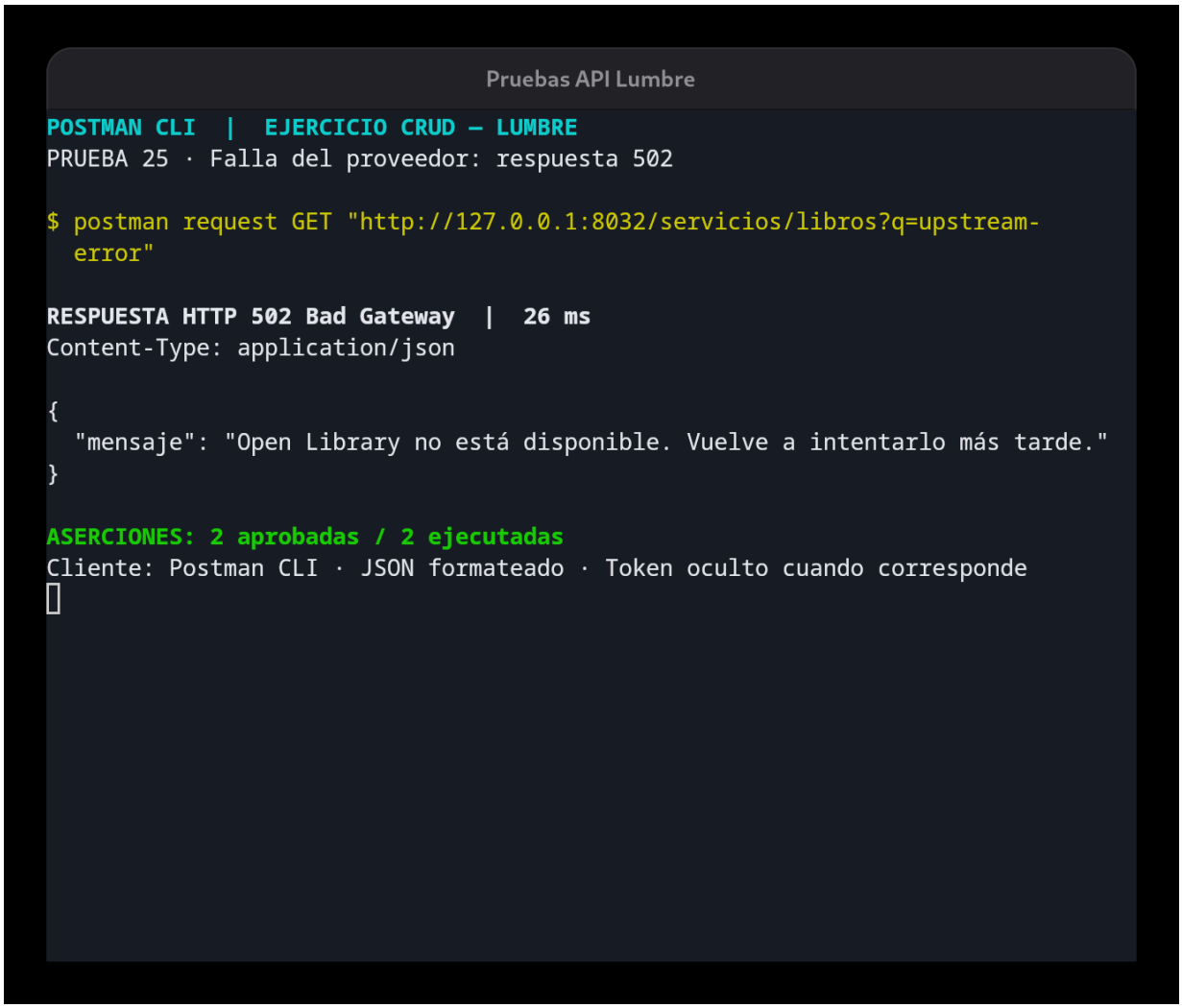


Figura 25. HTTP 502; 26 ms; 2 aserciones aprobadas.

El backend mantuvo una respuesta JSON controlada ante la falla externa. El 503 pertenece al proveedor simulado; el cliente de Lumbre recibió 502 Bad Gateway.

7.31. PRUEBA 26: RENOVAR ACCESO PARA PRUEBAS NEGATIVAS.

1. Envié POST a /auth/login. Body JSON: usuario practica y contraseña del entorno temporal.

2. La petición usó No Auth. Content-Type: application/json.

3. Contrasté HTTP 200 con el contrato y ejecuté dos aserciones en Postman.

Resultados:

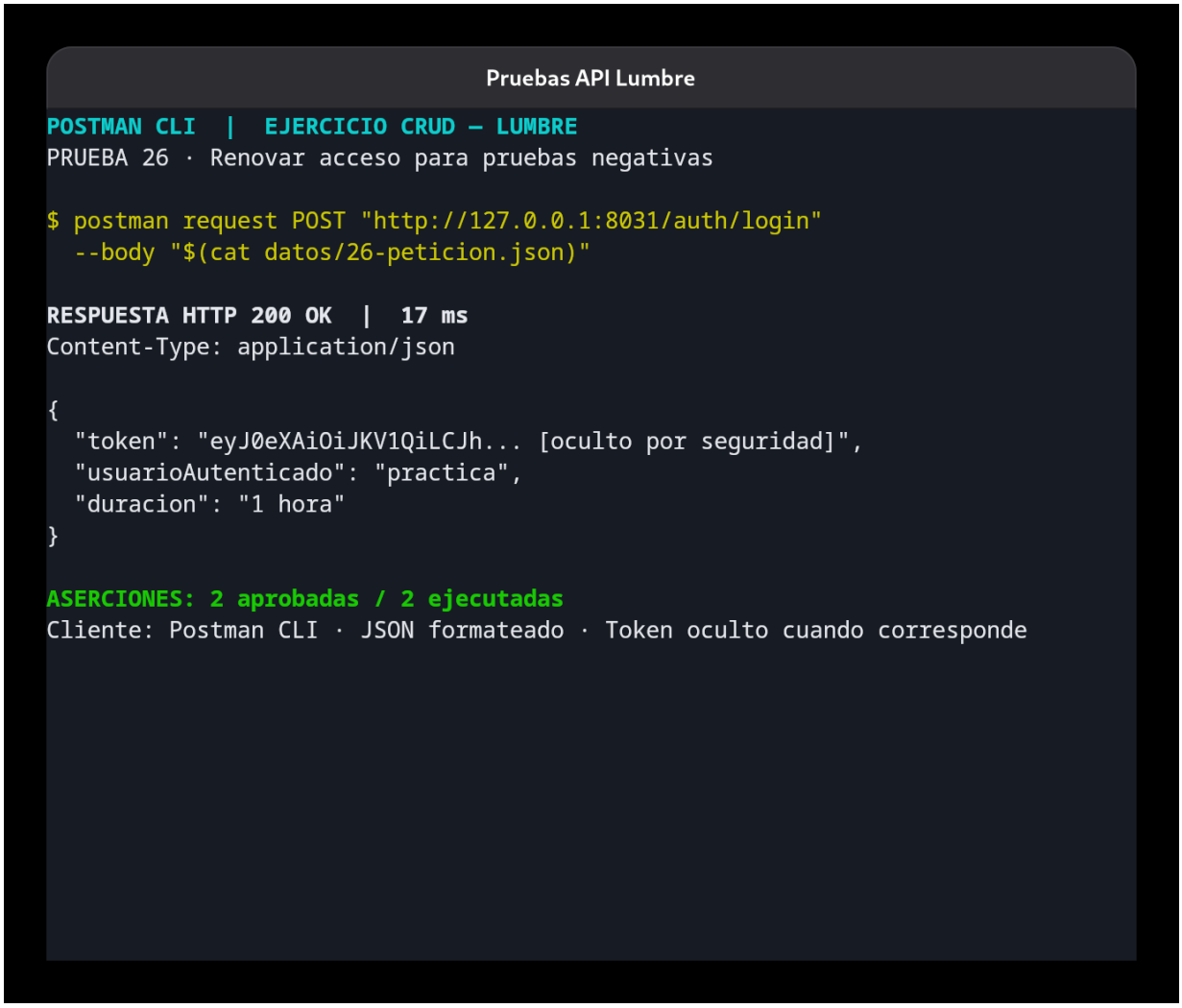


Figura 26. HTTP 200; 17 ms; 2 aserciones aprobadas.

Se obtuvo un token nuevo para continuar las pruebas negativas en la misma instancia temporal.

7.32. PRUEBA 27: LOGIN SIN CAMPOS REQUERIDOS.

1. Envié POST a /auth/login. Body JSON: {}

2. La petición usó No Auth. Content-Type: application/json.

3. Contrasté HTTP 400 con el contrato y ejecuté dos aserciones en Postman.

Resultados:

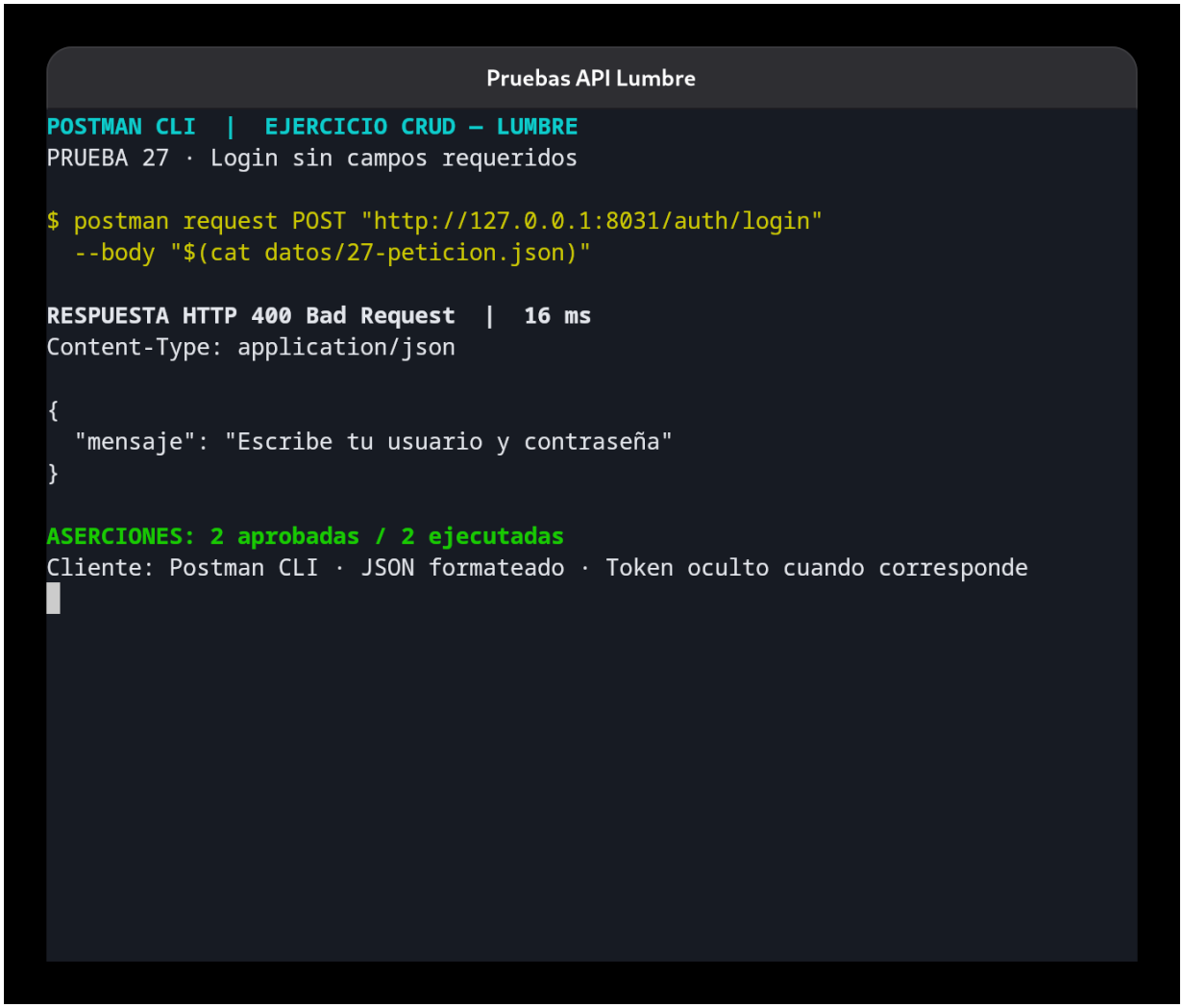


Figura 27. HTTP 400; 16 ms; 2 aserciones aprobadas.

Se rechazó un login incompleto antes de firmar tokens.

7.33. PRUEBA 28: TOKEN INVÁLIDO EN CONSULTA GENERAL.

1. Envié GET a /generos/obtenerVarios. Sin cuerpo.

2. La petición usó Bearer inválido.

3. Contrasté HTTP 401 con el contrato y ejecuté dos aserciones en Postman.

Resultados:

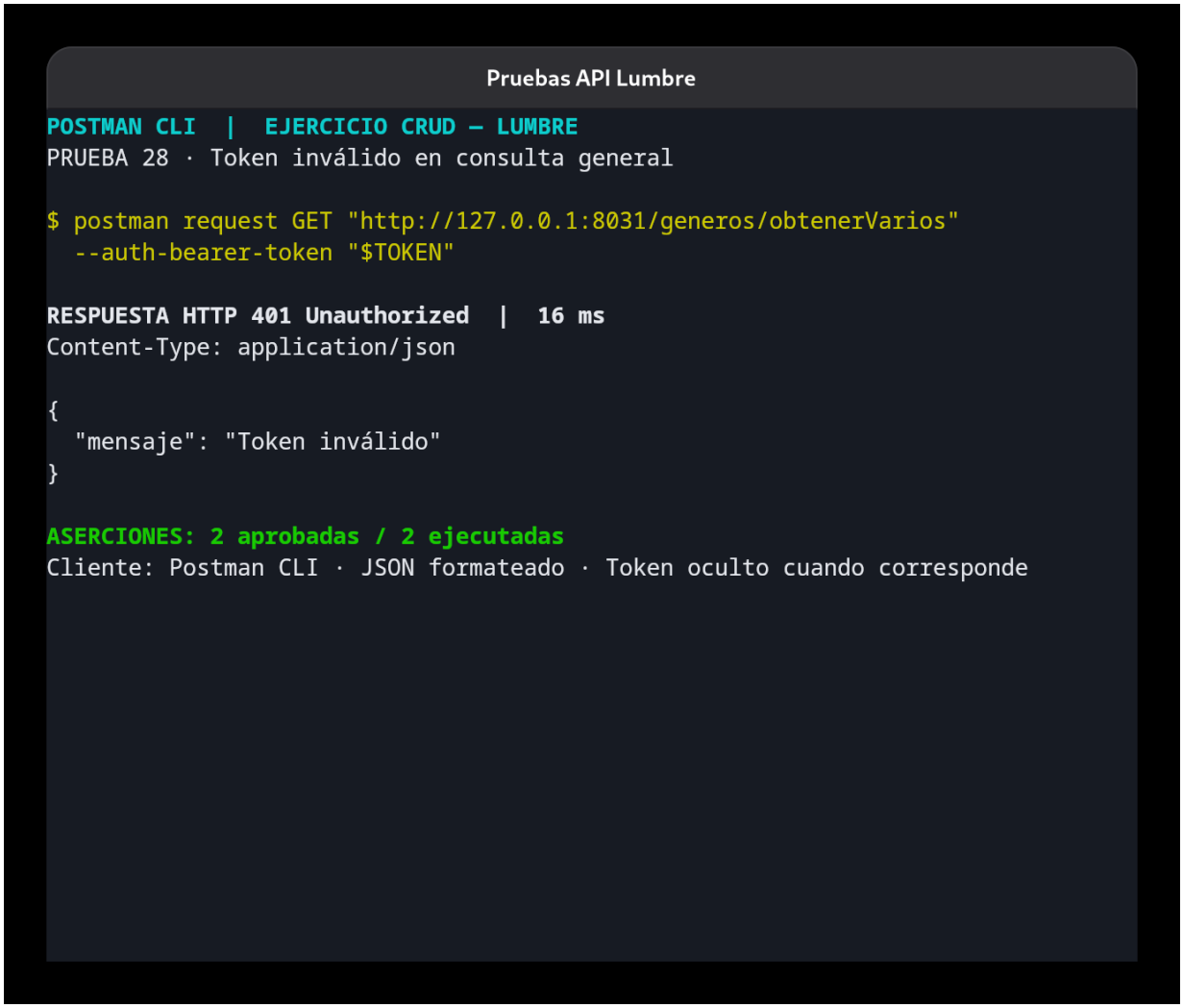


Figura 28. HTTP 401; 16 ms; 2 aserciones aprobadas.

La consulta general de géneros queda protegida igual que la de libros.

7.34. PRUEBA 29: ALTA DE GÉNERO SIN NOMBRE.

1. Envié POST a /generos/falta. Body JSON: {"descripcion": "Falta nombre"}

2. La petición usó Bearer obtenido en el login. Content-Type: application/json.

3. Contrasté HTTP 400 con el contrato y ejecuté dos aserciones en Postman.

Resultados:

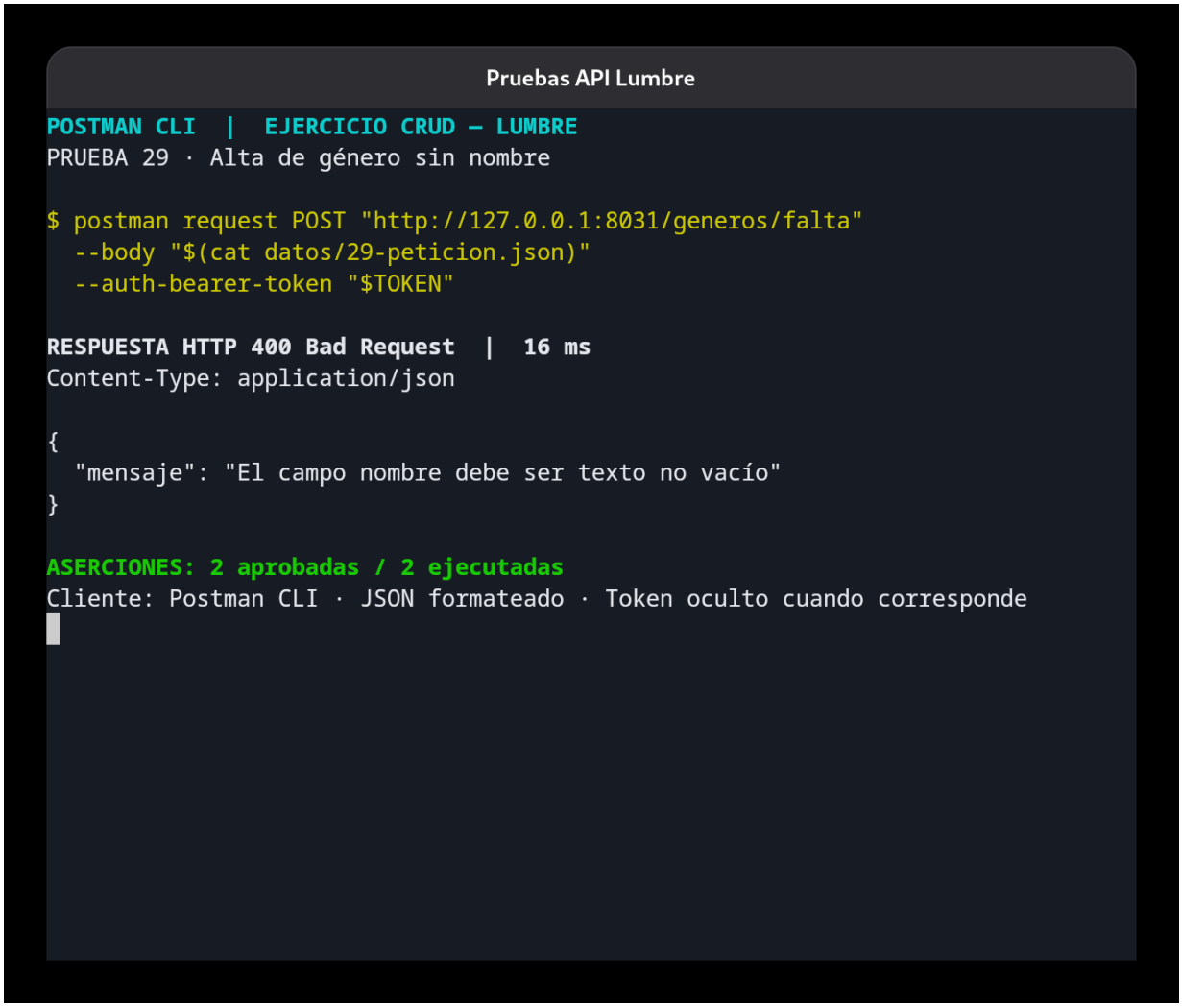


Figura 29. HTTP 400; 16 ms; 2 aserciones aprobadas.

El nombre es obligatorio en el alta de géneros.

7.35. PRUEBA 30: CREAR GÉNERO PARA VALIDAR CONFLICTOS.

1. Envié POST a /generos/falta. Body JSON: {"nombre": "Ensayo"}

2. La petición usó Bearer obtenido en el login. Content-Type: application/json.

3. Contrasté HTTP 201 con el contrato y ejecuté dos aserciones en Postman.

Resultados:

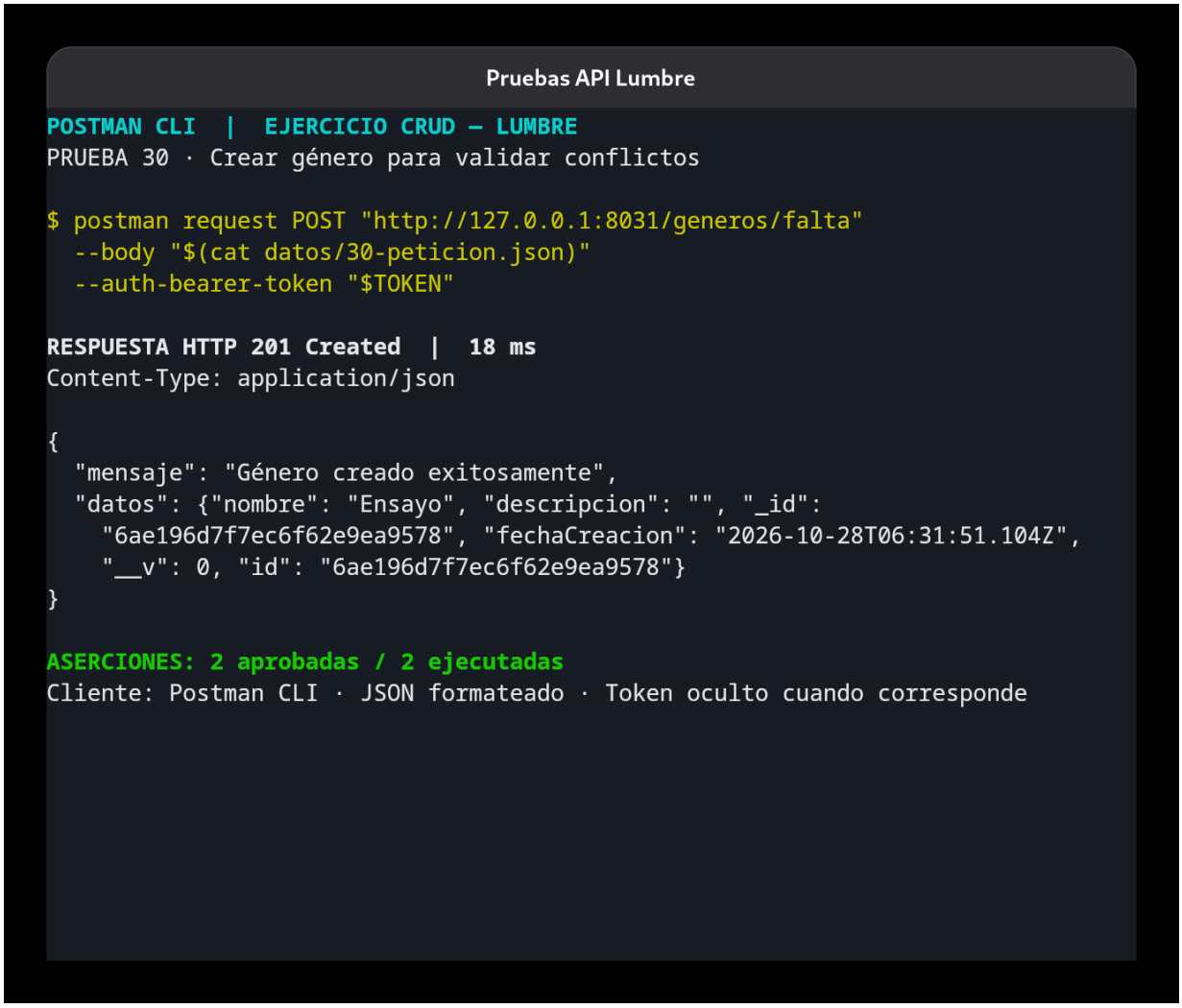


Figura 30. HTTP 201; 18 ms; 2 aserciones aprobadas.

Se preparó un género aislado para comprobar duplicados y modificaciones inválidas.

7.36. PRUEBA 31: ALTA DE GÉNERO DUPLICADO.

1. Envié POST a /generos/falta. Body JSON: {"nombre": "Ensayo"}

2. La petición usó Bearer obtenido en el login. Content-Type: application/json.

3. Contrasté HTTP 400 con el contrato y ejecuté dos aserciones en Postman.

Resultados:

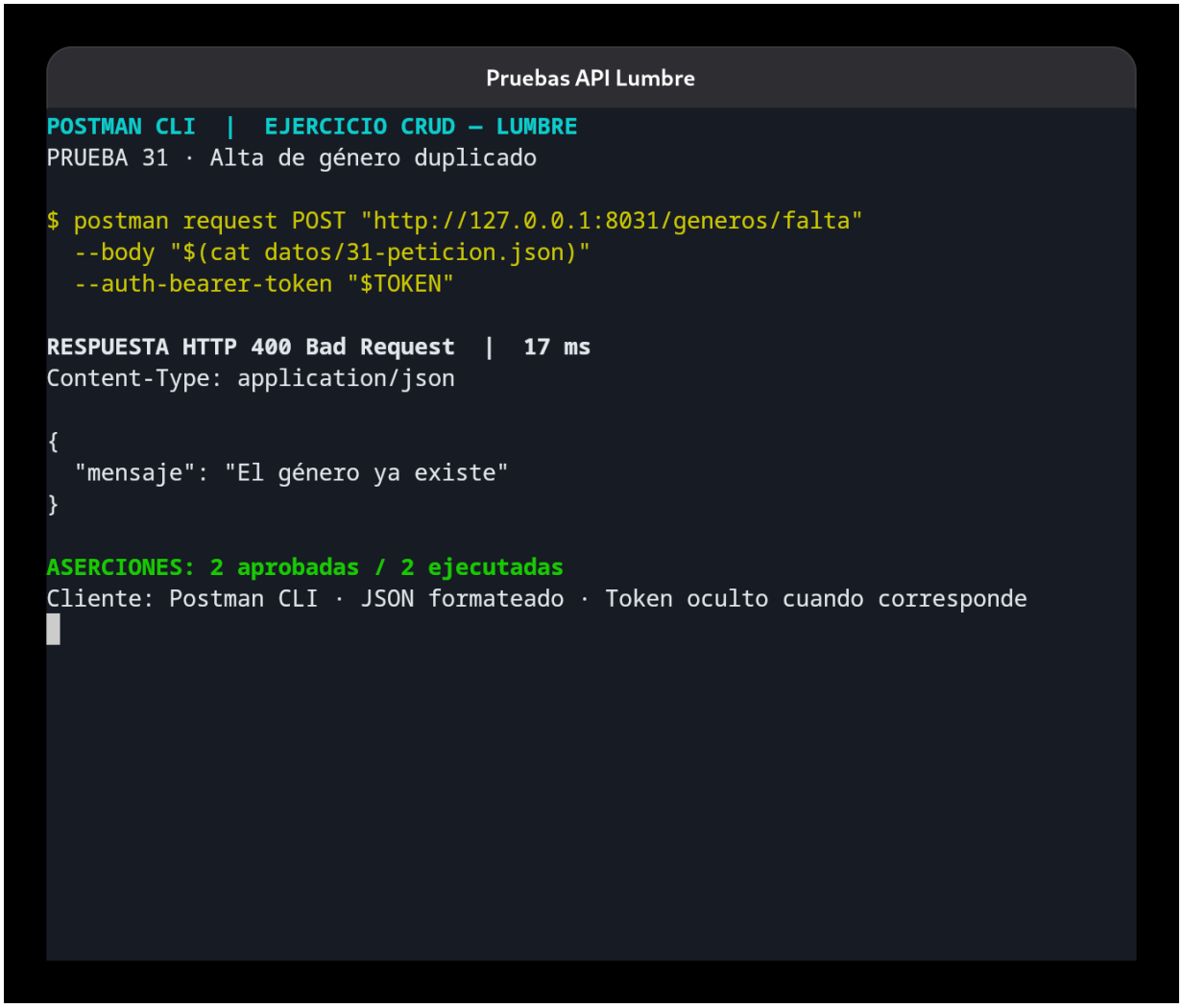


Figura 31. HTTP 400; 17 ms; 2 aserciones aprobadas.

La comprobación previa de unicidad rechaza el alta duplicada con 400.

7.37. PRUEBA 32: CAMBIO DE GÉNERO CON NOMBRE VACÍO.

1. Envié PUT a /generos/baja/6ae196d7f7ec6f62e9ea9578. Body JSON: {"nombre": ""}

2. La petición usó Bearer obtenido en el login. Content-Type: application/json.

3. Contrasté HTTP 400 con el contrato y ejecuté dos aserciones en Postman.

Resultados:

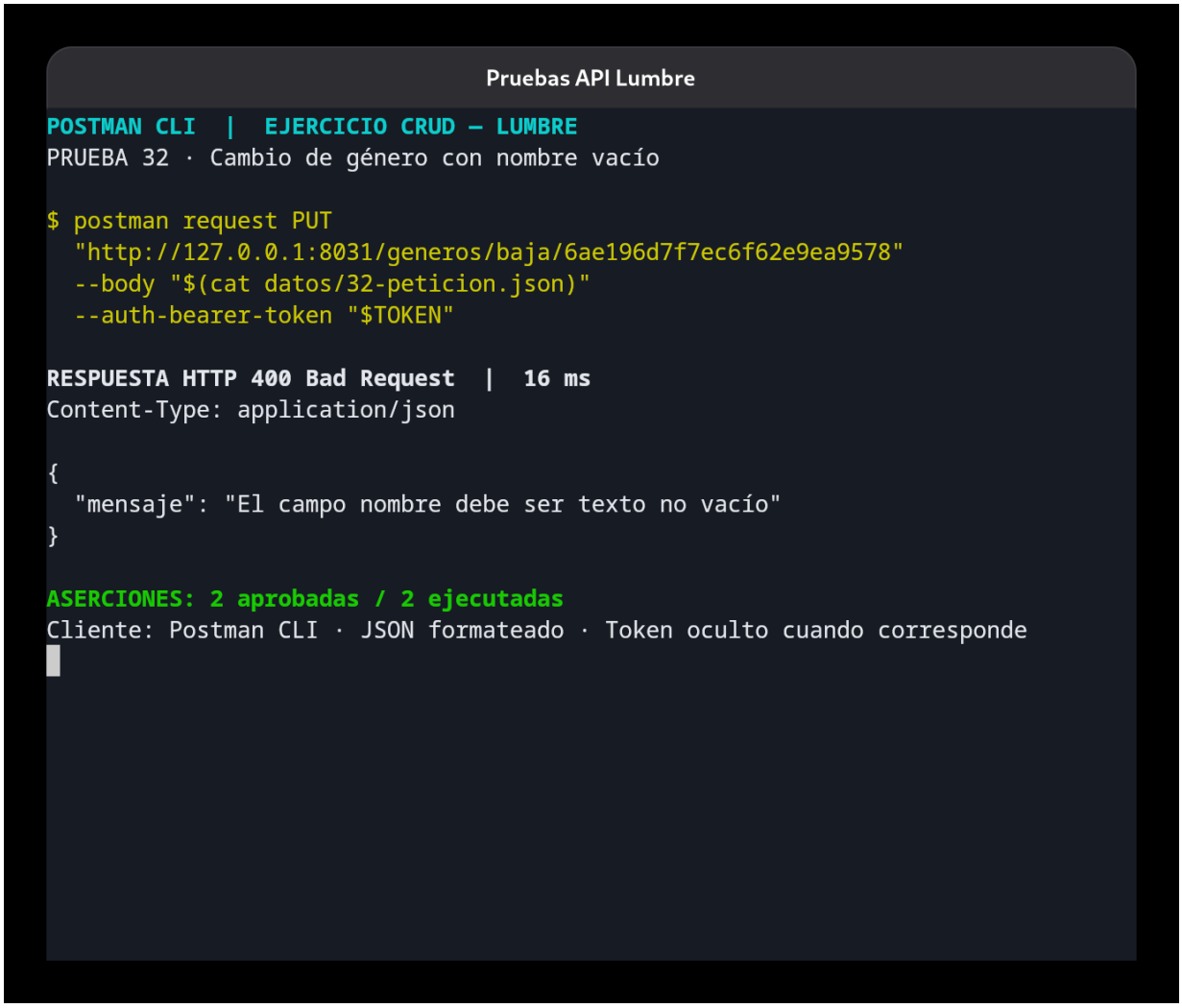


Figura 32. HTTP 400; 16 ms; 2 aserciones aprobadas.

Un cambio no permite vaciar el nombre del género.

7.38. PRUEBA 33: CREAR SEGUNDO GÉNERO.

1. Envié POST a /generos/falta. Body JSON: {"nombre": "Historia"}

2. La petición usó Bearer obtenido en el login. Content-Type: application/json.

3. Contrasté HTTP 201 con el contrato y ejecuté dos aserciones en Postman.

Resultados:

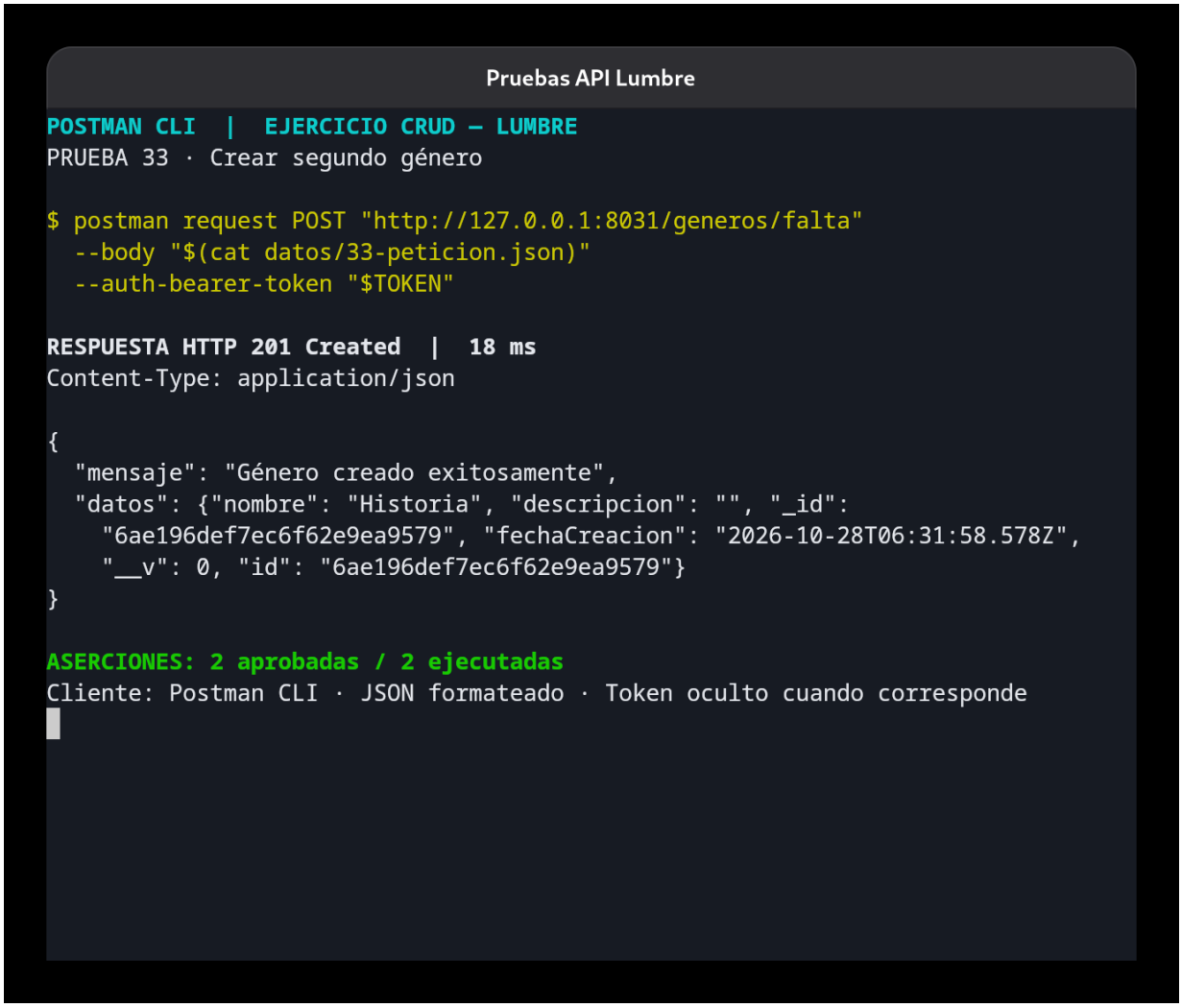


Figura 33. HTTP 201; 18 ms; 2 aserciones aprobadas.

El segundo género permite intentar renombrarlo con un nombre ya existente.

7.39. PRUEBA 34: RENOMBRADO CON NOMBRE DUPLICADO.

1. Envié PUT a /generos/baja/6ae196def7ec6f62e9ea9579. Body JSON: {"nombre": "Ensayo"}

2. La petición usó Bearer obtenido en el login. Content-Type: application/json.

3. Contrasté HTTP 409 con el contrato y ejecuté dos aserciones en Postman.

Resultados:

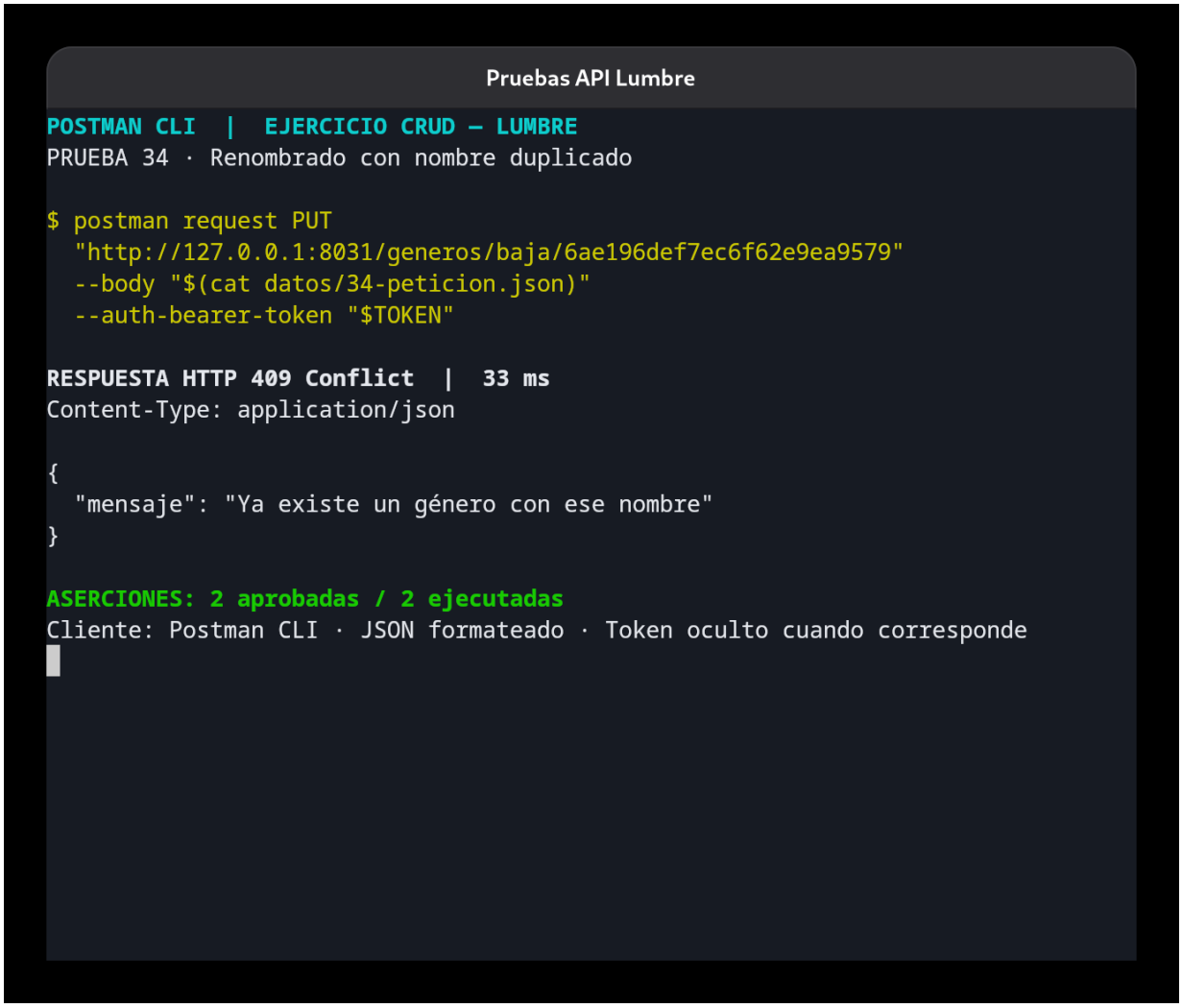


Figura 34. HTTP 409; 33 ms; 2 aserciones aprobadas.

El índice único impidió el renombrado; el error de MongoDB se convirtió en 409.

7.40. PRUEBA 35: CONSULTA DE GÉNERO CON ID INVÁLIDO.

1. Envié GET a /generos/obtenerUno/invalido. Sin cuerpo.

2. La petición usó Bearer obtenido en el login.

3. Contrasté HTTP 400 con el contrato y ejecuté dos aserciones en Postman.

Resultados:

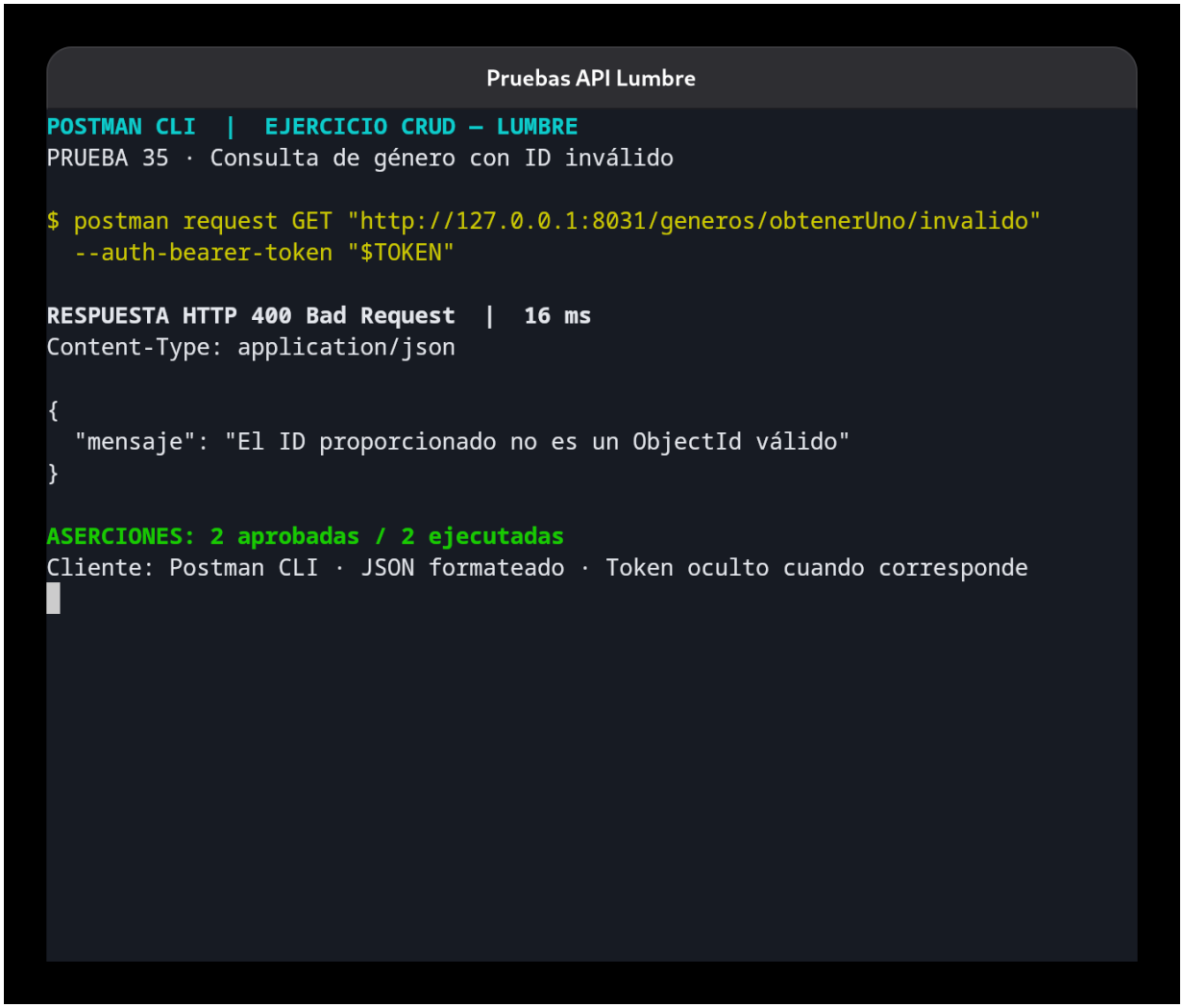


Figura 35. HTTP 400; 16 ms; 2 aserciones aprobadas.

El formato se valida antes de consultar MongoDB.

7.41. PRUEBA 36: CAMBIO DE GÉNERO INEXISTENTE.

1. Envié PUT a /generos/baja/000000000000000000000000. Body JSON: {"nombre": "No existe"}

2. La petición usó Bearer obtenido en el login. Content-Type: application/json.

3. Contrasté HTTP 404 con el contrato y ejecuté dos aserciones en Postman.

Resultados:

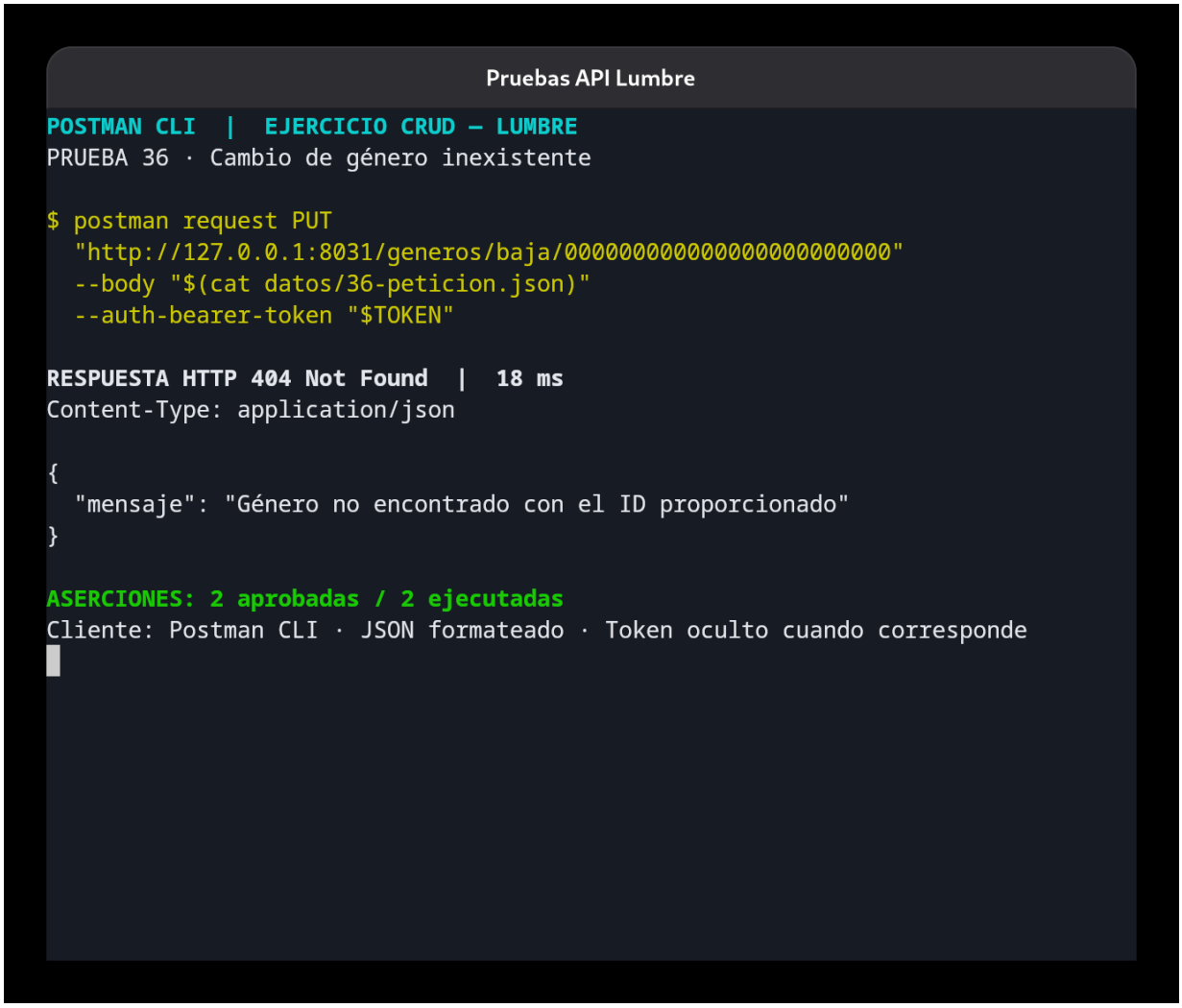


Figura 36. HTTP 404; 18 ms; 2 aserciones aprobadas.

Un ID de formato válido no implica que el documento exista.

7.42. PRUEBA 37: BAJA DE GÉNERO INEXISTENTE.

1. Envié DELETE a /generos/eliminar/000000000000000000000000. Sin cuerpo.

2. La petición usó Bearer obtenido en el login.

3. Contrasté HTTP 404 con el contrato y ejecuté dos aserciones en Postman.

Resultados:

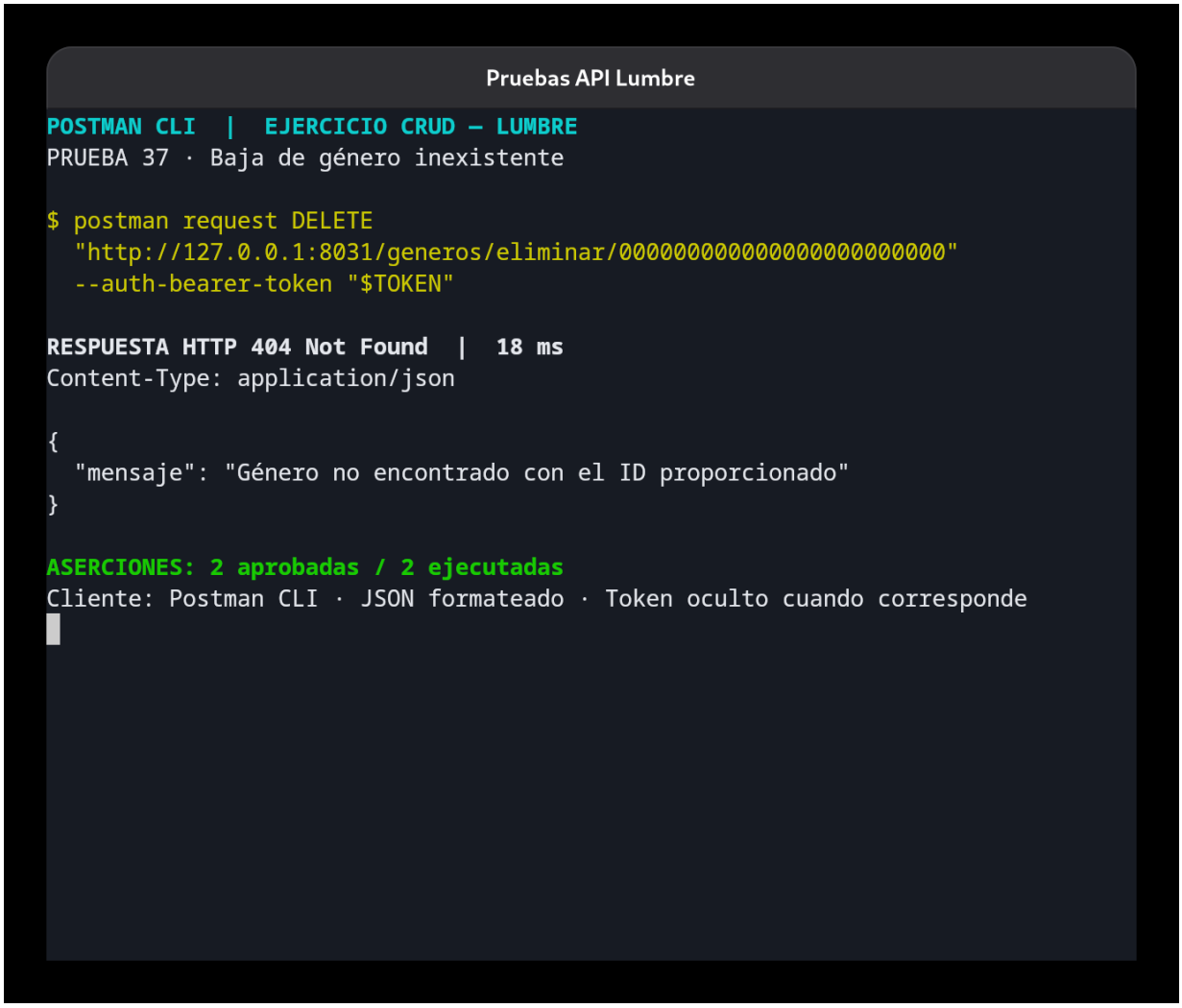


Figura 37. HTTP 404; 18 ms; 2 aserciones aprobadas.

La eliminación de un documento inexistente devuelve 404.

7.43. PRUEBA 38: ALTA DE LIBRO SIN AUTOR.

1. Envié POST a /libros/falta. Body JSON: {"nombre": "Ensayos", "genero": "6ae196d7f7ec6f62e9ea9578", "ubicacion": "C-01", "ejemplares": 1}

2. La petición usó Bearer obtenido en el login. Content-Type: application/json.

3. Contrasté HTTP 400 con el contrato y ejecuté dos aserciones en Postman.

Resultados:

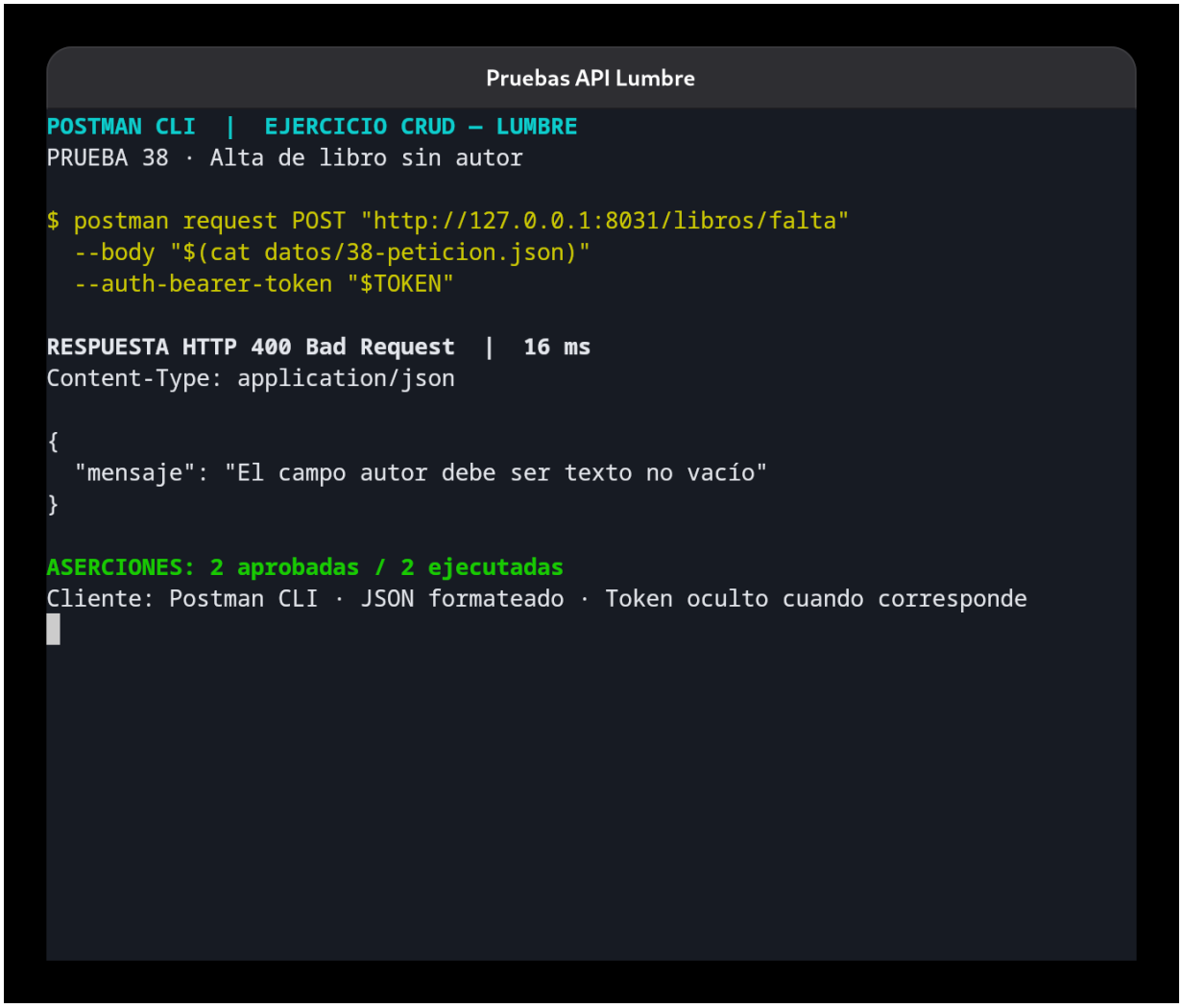


Figura 38. HTTP 400; 16 ms; 2 aserciones aprobadas.

El autor es obligatorio al crear un libro.

7.44. PRUEBA 39: ALTA DE LIBRO CON GÉNERO INEXISTENTE.

1. Envié POST a /libros/falta. Body JSON: {"nombre": "Ensayos", "autor": "Autor de prueba", "genero": "000000000000000000000000", "ubicacion": "C-01", "ejemplares": 1}

2. La petición usó Bearer obtenido en el login. Content-Type: application/json.

3. Contrasté HTTP 400 con el contrato y ejecuté dos aserciones en Postman.

Resultados:

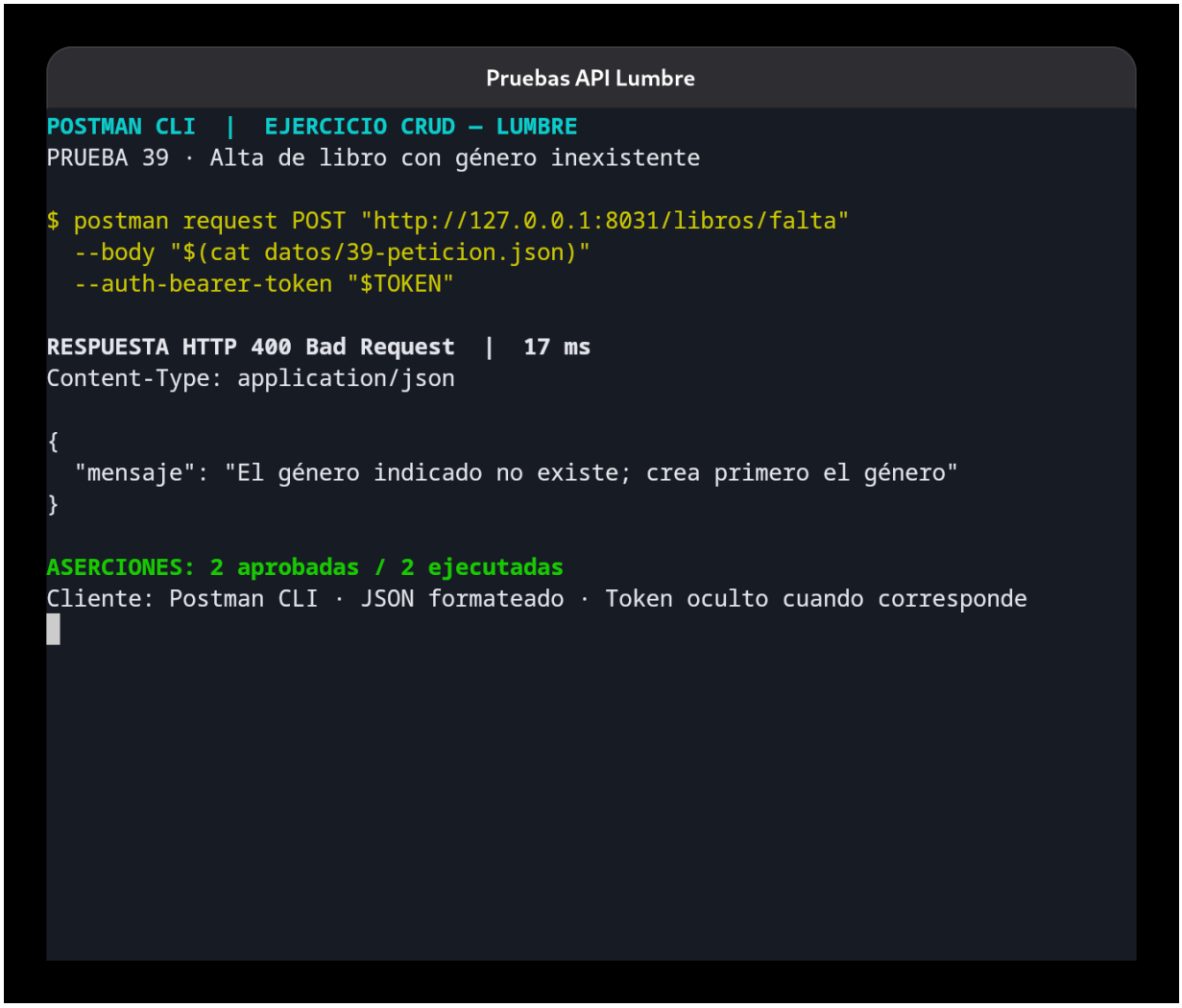


Figura 39. HTTP 400; 17 ms; 2 aserciones aprobadas.

Se impide crear una referencia a un género ausente.

7.45. PRUEBA 40: ALTA DE LIBRO CON EJEMPLARES FRACCIONARIOS.

1. Envié POST a /libros/falta. Body JSON: {"nombre": "Ensayos", "autor": "Autor de prueba", "genero": "6ae196d7f7ec6f62e9ea9578", "ubicacion": "C-01", "ejemplares": 1.5}

2. La petición usó Bearer obtenido en el login. Content-Type: application/json.

3. Contrasté HTTP 400 con el contrato y ejecuté dos aserciones en Postman.

Resultados:

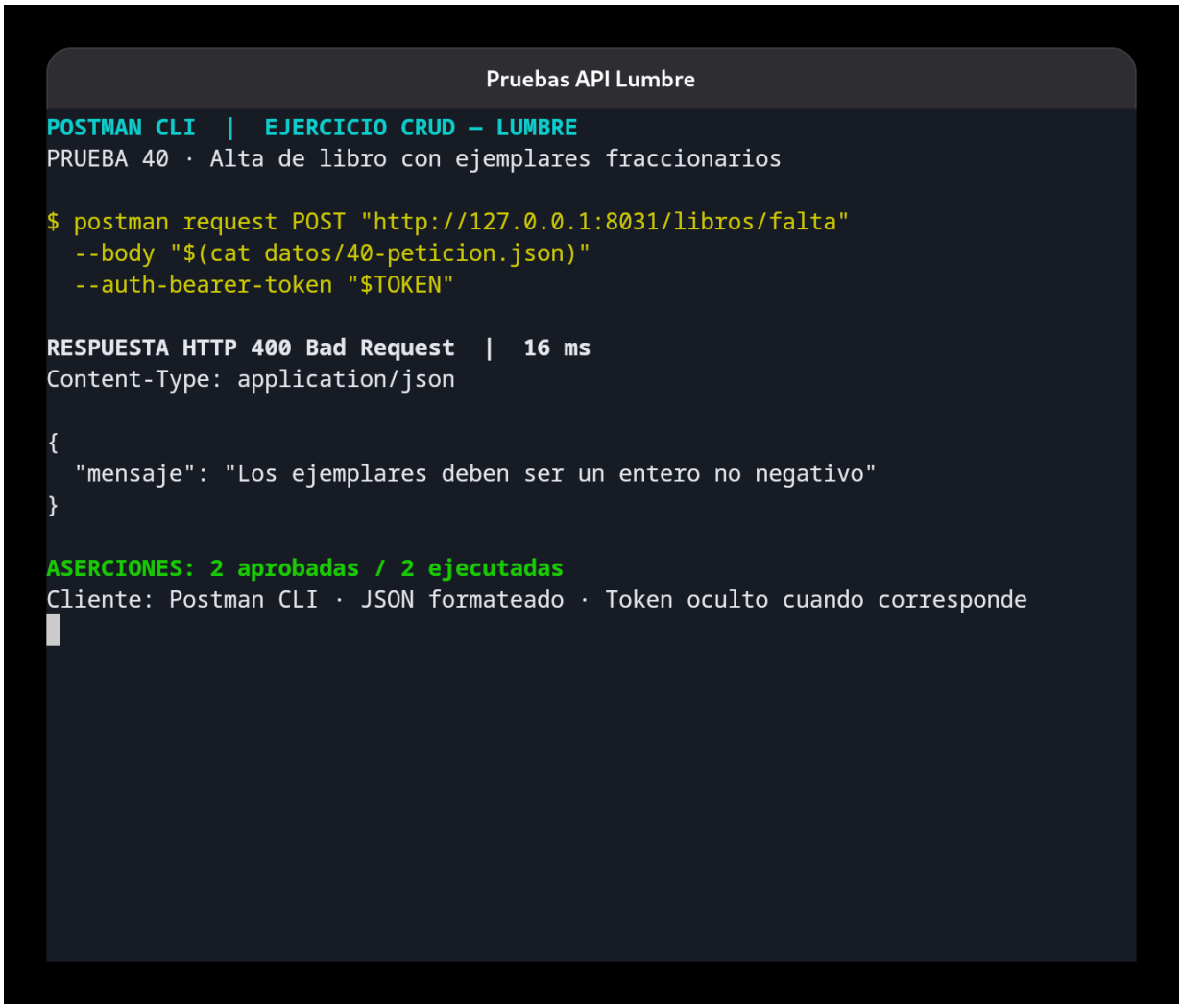


Figura 40. HTTP 400; 16 ms; 2 aserciones aprobadas.

La cantidad debe ser entera; un valor fraccionario no es aceptable.

7.46. PRUEBA 41: CAMBIO DE LIBRO INEXISTENTE.

1. Envié PUT a /libros/baja/000000000000000000000000. Body JSON: {"ejemplares": 2}

2. La petición usó Bearer obtenido en el login. Content-Type: application/json.

3. Contrasté HTTP 404 con el contrato y ejecuté dos aserciones en Postman.

Resultados:

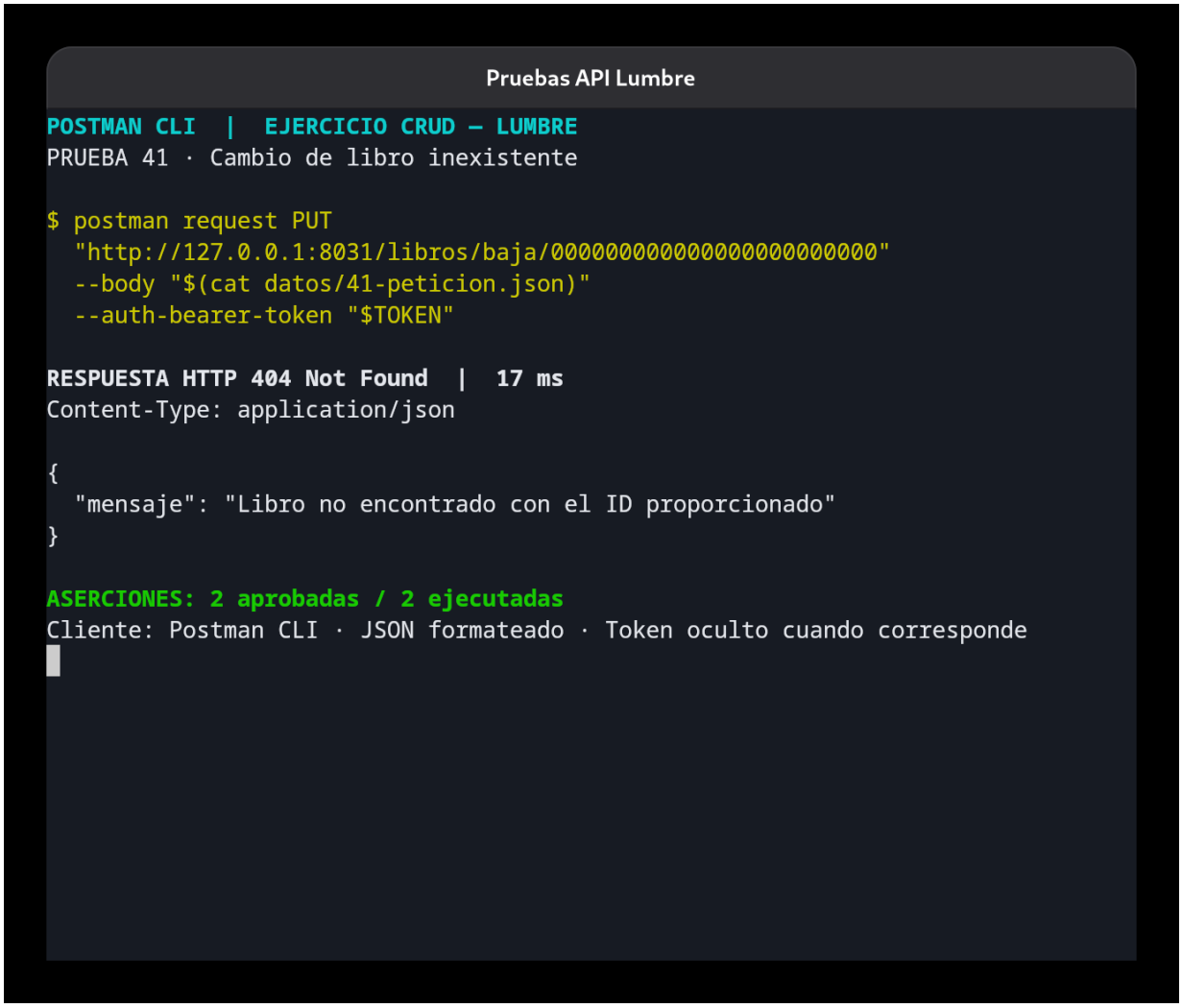


Figura 41. HTTP 404; 17 ms; 2 aserciones aprobadas.

La modificación no crea implícitamente un documento inexistente.

7.47. PRUEBA 42: CAMBIO DE LIBRO CON CAMPOS NO PERMITIDOS.

1. Envié PUT a /libros/baja/000000000000000000000000. Body JSON: {"propietario": "otro"}

2. La petición usó Bearer obtenido en el login. Content-Type: application/json.

3. Contrasté HTTP 400 con el contrato y ejecuté dos aserciones en Postman.

Resultados:

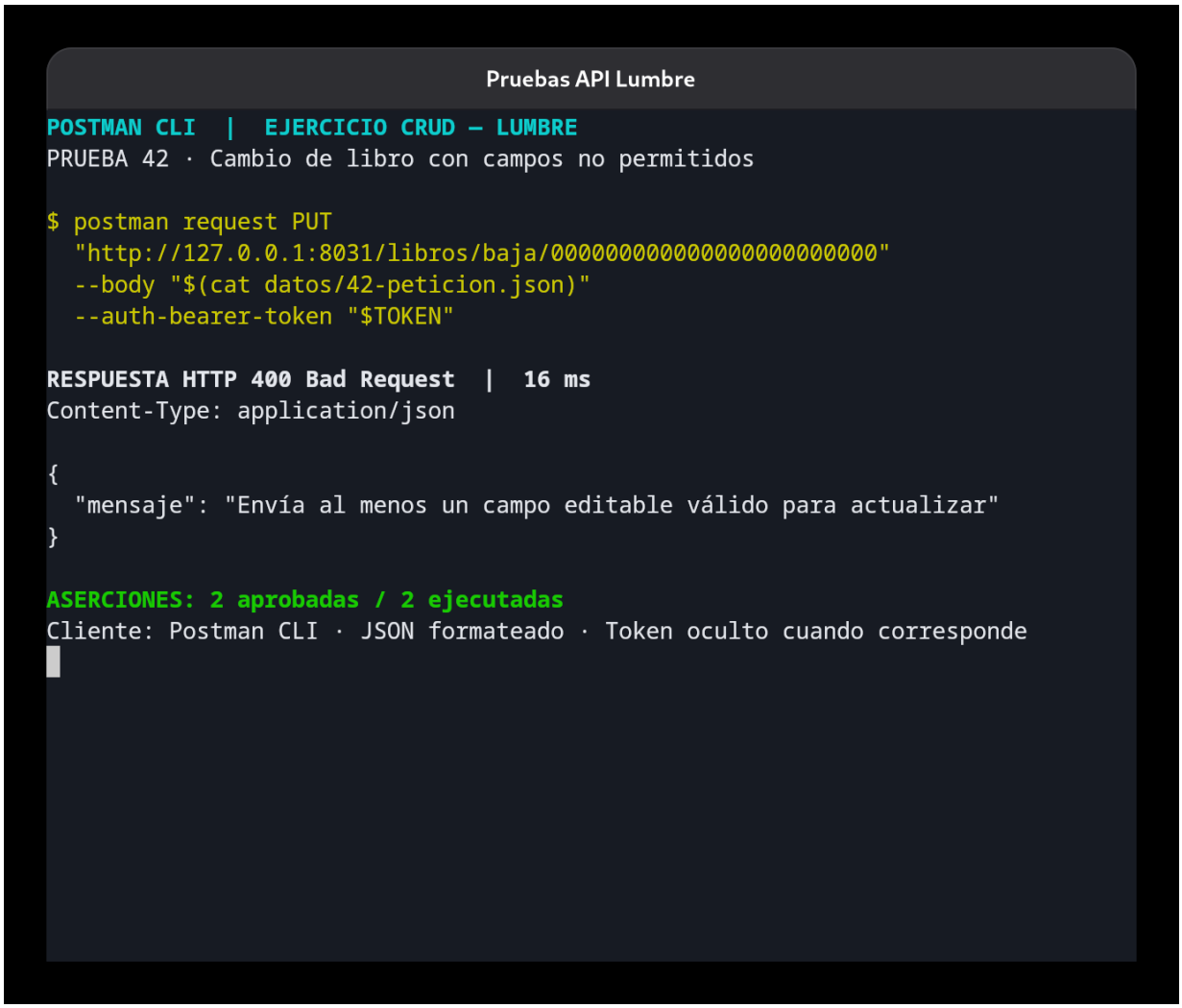


Figura 42. HTTP 400; 16 ms; 2 aserciones aprobadas.

La lista de campos permitidos impide modificaciones ajenas al esquema editable.

7.48. PRUEBA 43: BAJA DE LIBRO INEXISTENTE.

1. Envié DELETE a /libros/eliminar/000000000000000000000000. Sin cuerpo.

2. La petición usó Bearer obtenido en el login.

3. Contrasté HTTP 404 con el contrato y ejecuté dos aserciones en Postman.

Resultados:

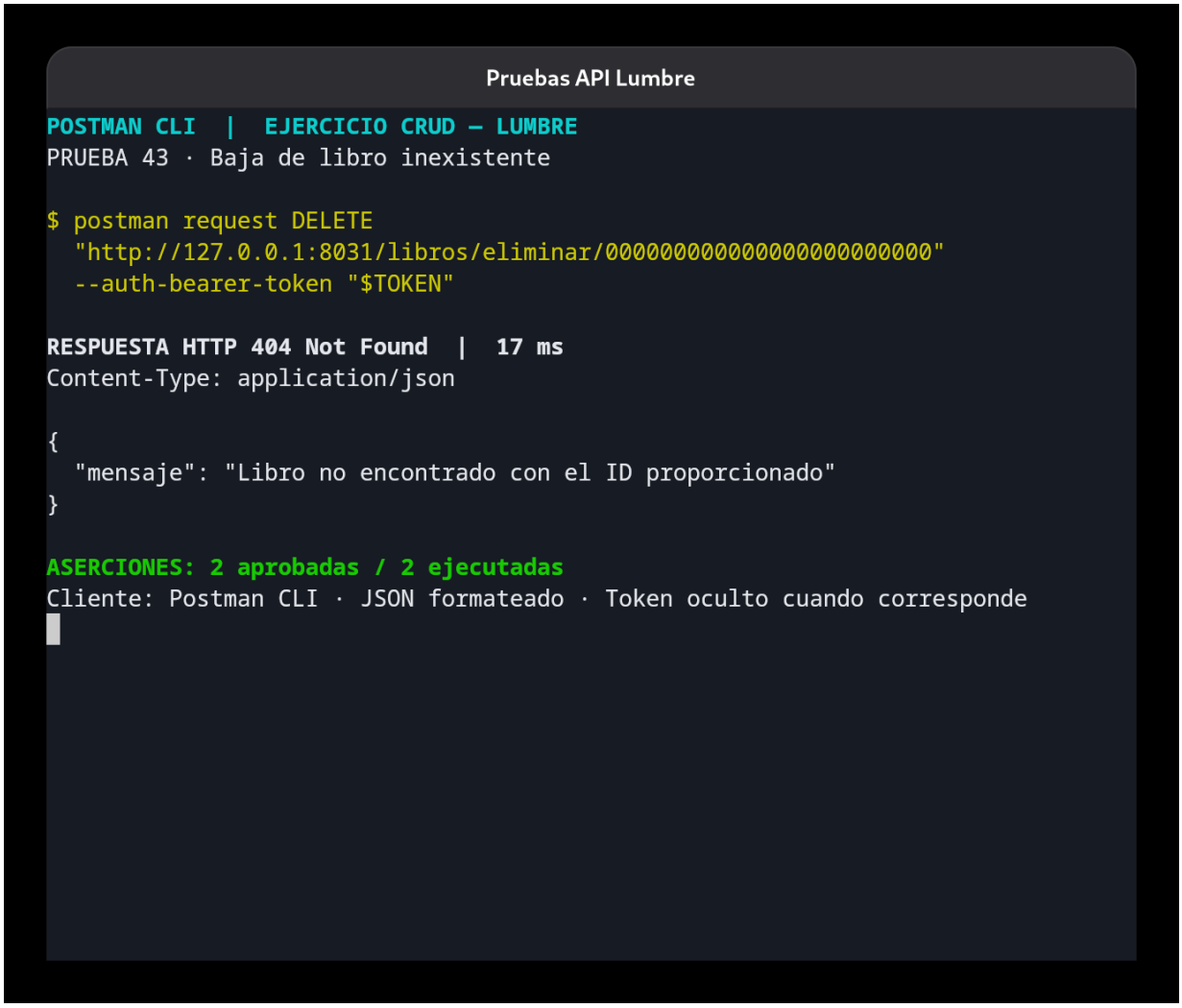


Figura 43. HTTP 404; 17 ms; 2 aserciones aprobadas.

La baja de un libro inexistente es un error de recurso no encontrado.

7.49. PRUEBA 44: BAJA DE LIBRO CON ID INVÁLIDO.

1. Envié DELETE a /libros/eliminar/invalido. Sin cuerpo.

2. La petición usó Bearer obtenido en el login.

3. Contrasté HTTP 400 con el contrato y ejecuté dos aserciones en Postman.

Resultados:

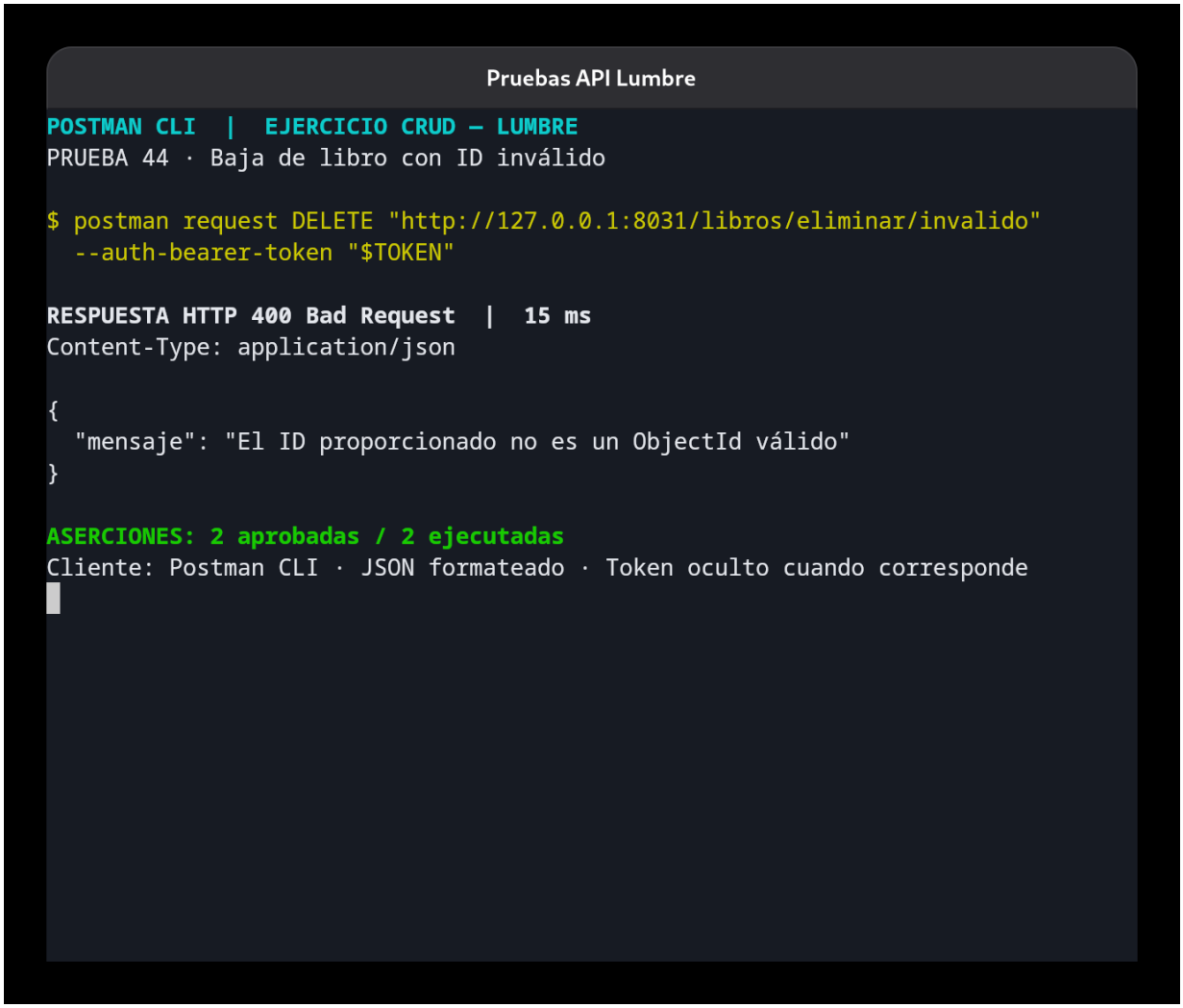


Figura 44. HTTP 400; 15 ms; 2 aserciones aprobadas.

Un ID inválido se distingue de un ID válido inexistente.

7.50. PRUEBA 45: ELIMINAR GÉNERO DE CONTROL.

1. Envié DELETE a /generos/eliminar/6ae196def7ec6f62e9ea9579. Sin cuerpo.

2. La petición usó Bearer obtenido en el login.

3. Contrasté HTTP 200 con el contrato y ejecuté dos aserciones en Postman.

Resultados:

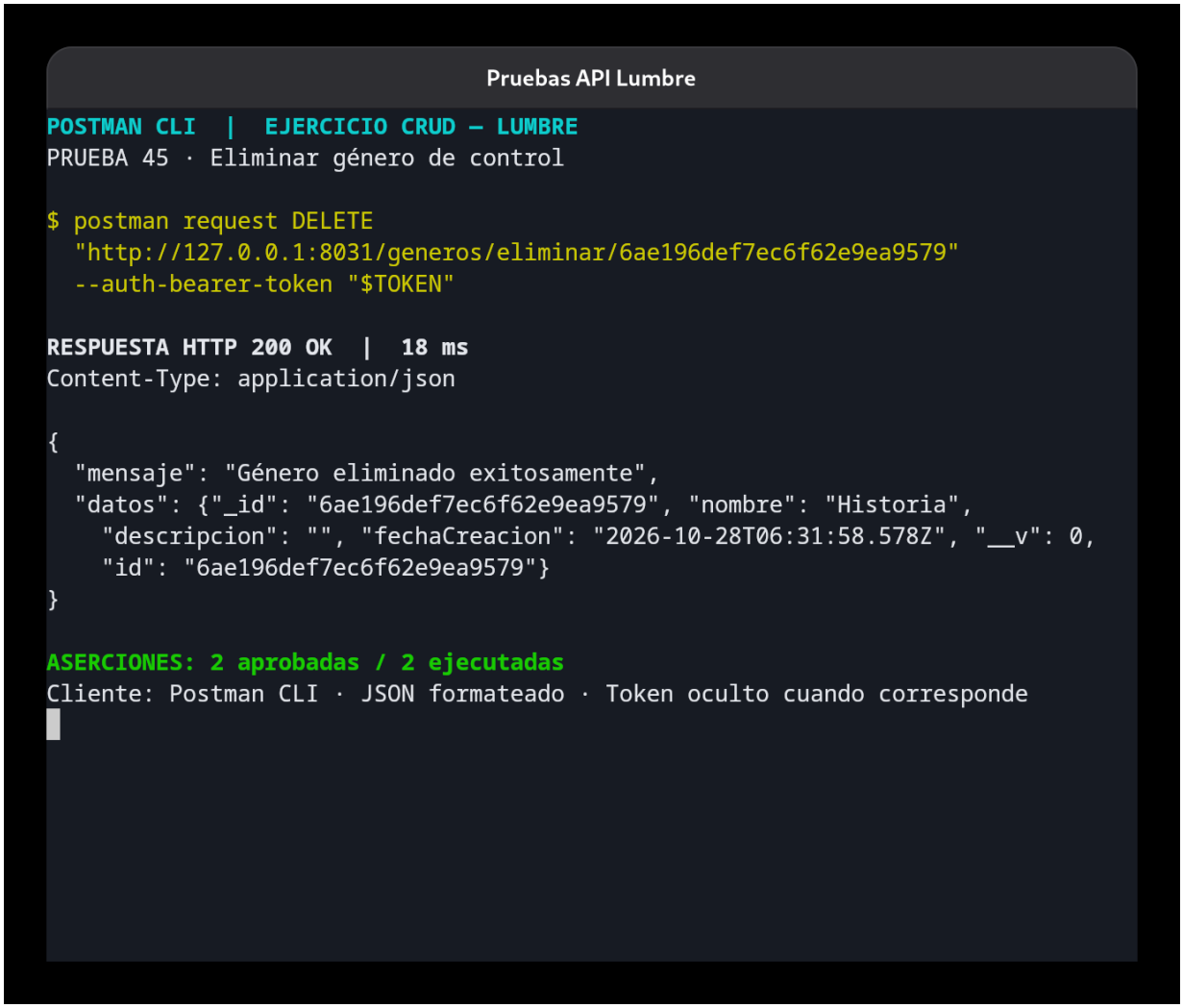


Figura 45. HTTP 200; 18 ms; 2 aserciones aprobadas.

Se eliminó el segundo género de prueba después del intento de conflicto.

7.51. PRUEBA 46: ELIMINAR GÉNERO DE PRUEBAS NEGATIVAS.

1. Envié DELETE a /generos/eliminar/6ae196d7f7ec6f62e9ea9578. Sin cuerpo.

2. La petición usó Bearer obtenido en el login.

3. Contrasté HTTP 200 con el contrato y ejecuté dos aserciones en Postman.

Resultados:

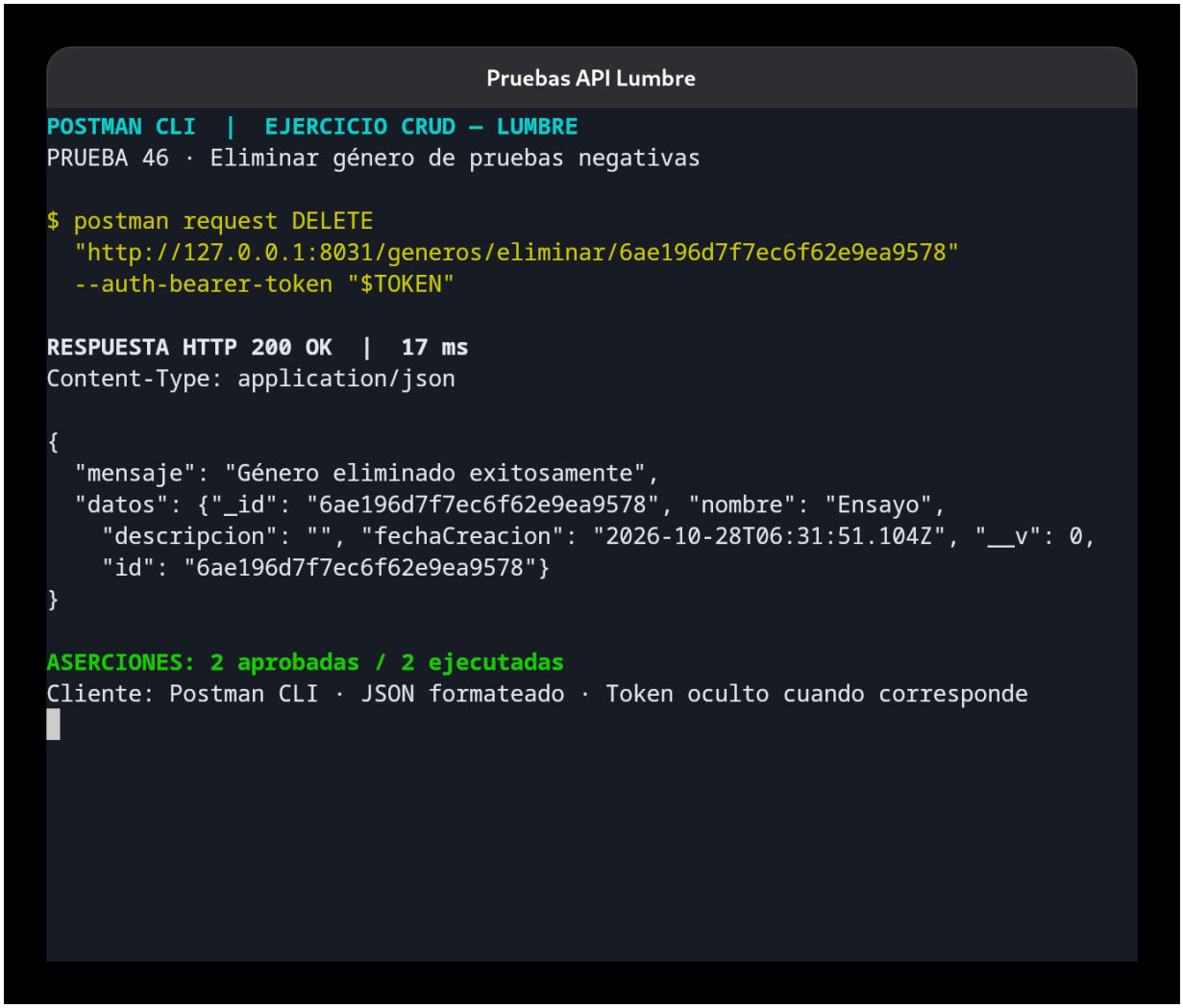


Figura 46. HTTP 200; 17 ms; 2 aserciones aprobadas.

Se eliminó el género usado en los casos negativos.

7.52. PRUEBA 47: CONFIRMAR GÉNEROS VACÍOS.

1. Envié GET a /generos/obtenerVarios. Sin cuerpo.

2. La petición usó Bearer obtenido en el login.

3. Contrasté HTTP 200 con el contrato y ejecuté dos aserciones en Postman.

Resultados:

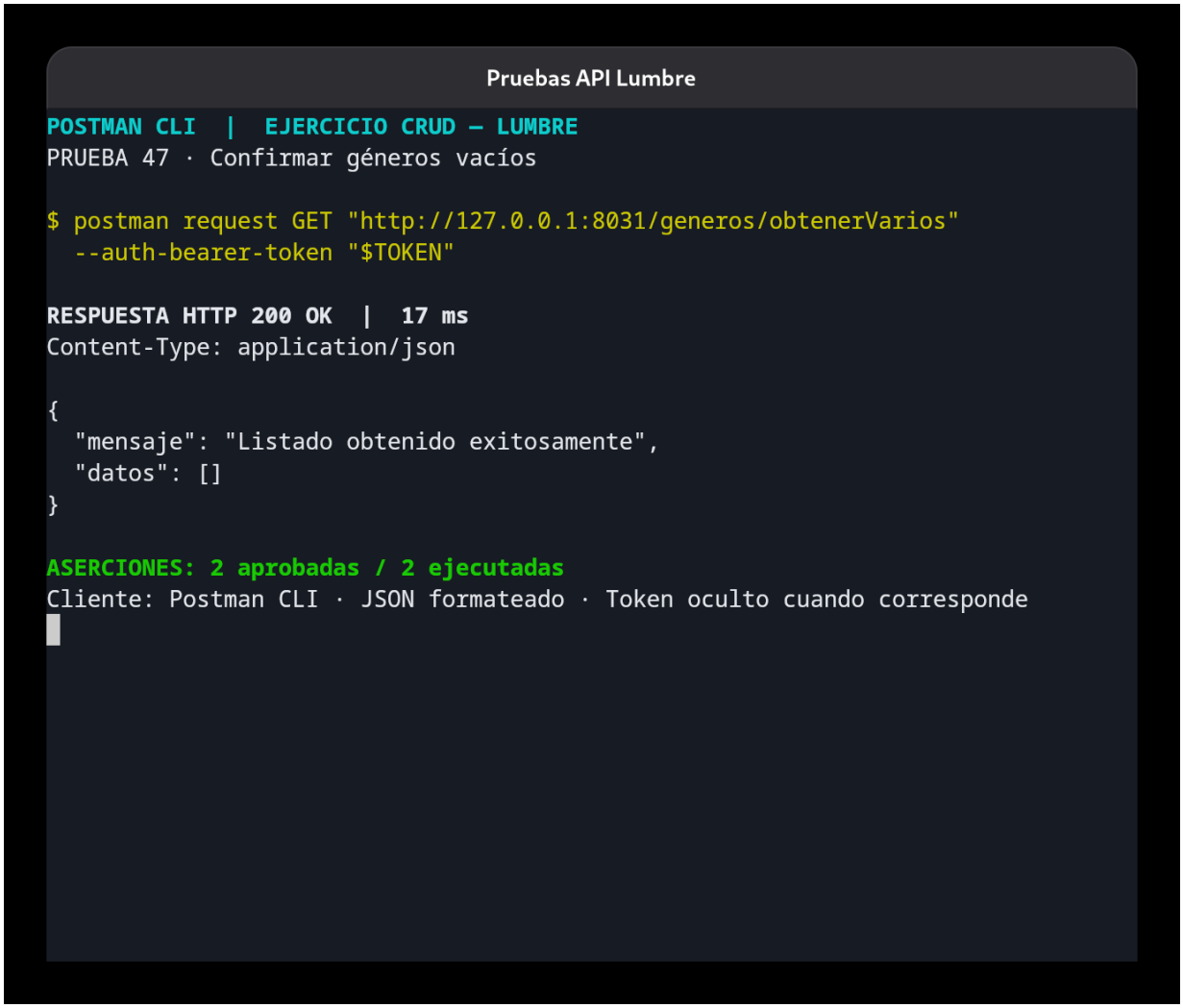


Figura 47. HTTP 200; 17 ms; 2 aserciones aprobadas.

La consulta confirma que no quedaron géneros de la ejecución.

7.53. PRUEBA 48: BÚSQUEDA EXTERNA CON PARÁMETRO VACÍO.

1. Envié GET a /servicios/libros?q=. Sin cuerpo.

2. La petición usó No Auth.

3. Contrasté HTTP 400 con el contrato y ejecuté dos aserciones en Postman.

Resultados:

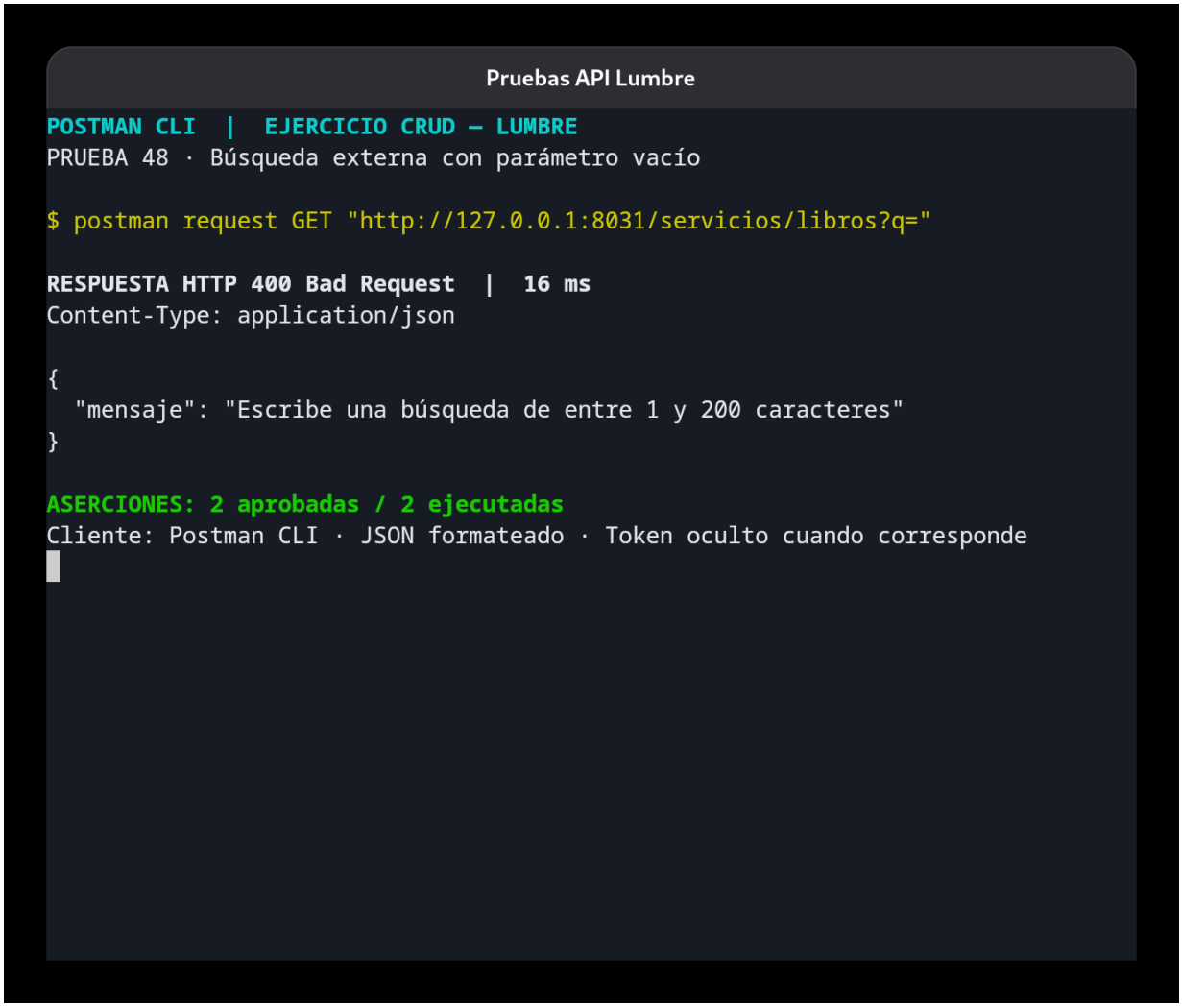


Figura 48. HTTP 400; 16 ms; 2 aserciones aprobadas.

La validación local rechaza una búsqueda vacía sin llamar al proveedor.

7.54. PRUEBA 49: DETALLE EXTERNO CON ID INVÁLIDO.

1. Envié GET a /servicios/libros/id-invalido. Sin cuerpo.

2. La petición usó No Auth.

3. Contrasté HTTP 400 con el contrato y ejecuté dos aserciones en Postman.

Resultados:

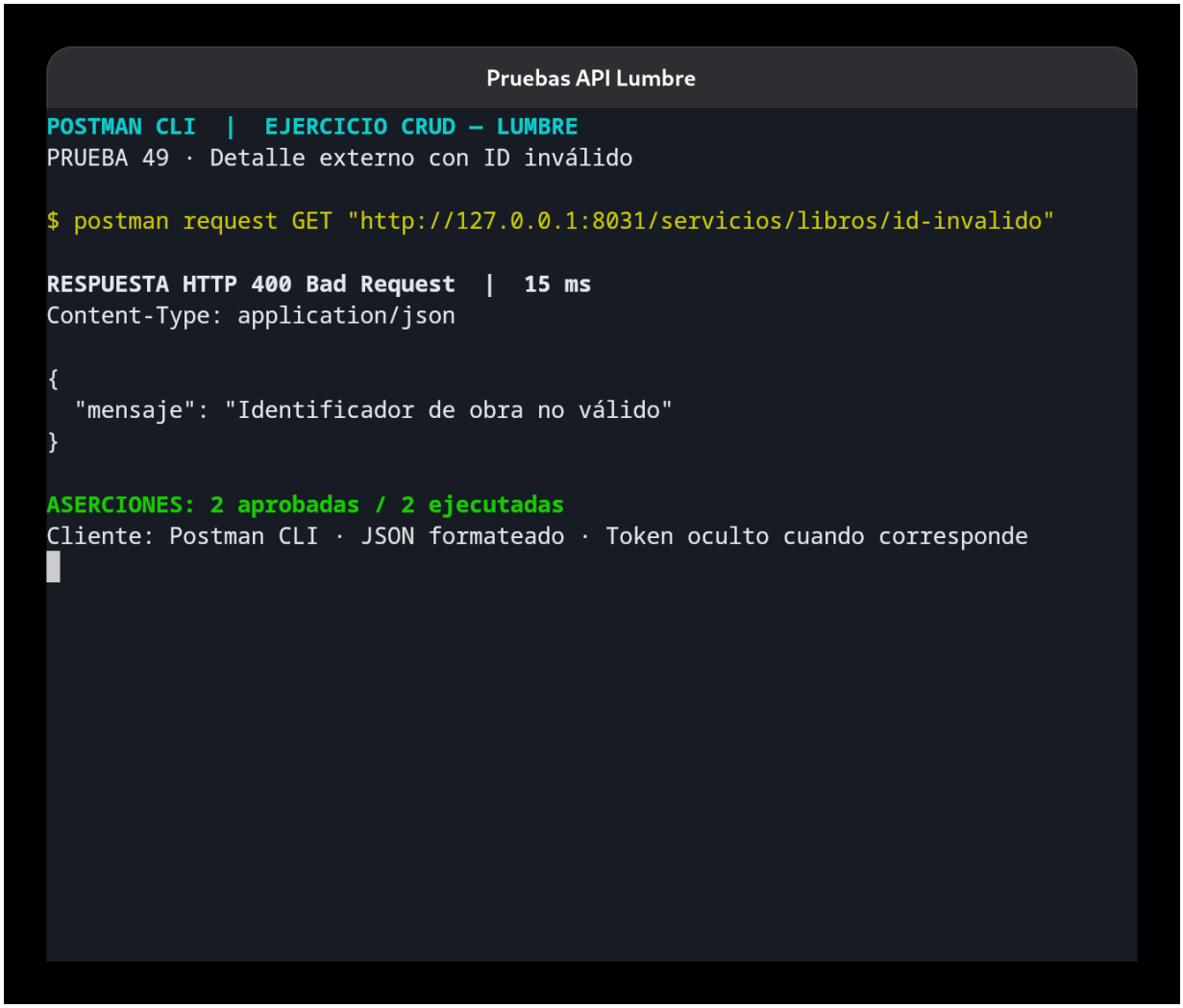


Figura 49. HTTP 400; 15 ms; 2 aserciones aprobadas.

El patrón de ID de obra se valida antes de Axios.

7.55. PRUEBA 50: BUSCAR PEDRO PÁRAMO EN OPEN LIBRARY REAL.

1. Envié GET a /servicios/libros?q=Pedro%20Paramo. Sin cuerpo.

2. La petición usó No Auth.

3. Contrasté HTTP 200 con el contrato y ejecuté dos aserciones en Postman.

Resultados:

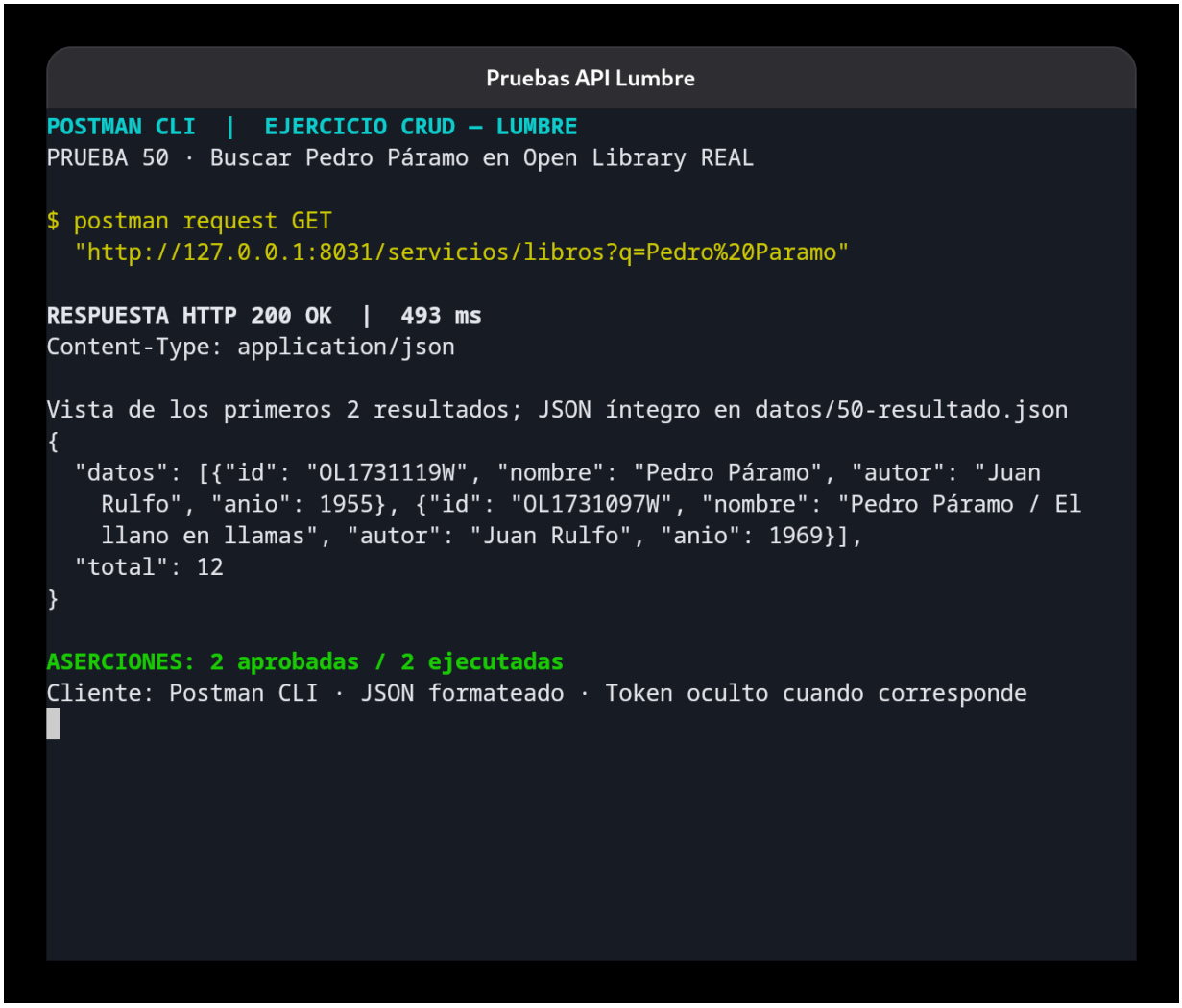


Figura 50. HTTP 200; 493 ms; 2 aserciones aprobadas.

El backend consultó Open Library público y adaptó los datos bibliográficos. La respuesta corresponde a la ejecución real, no al mock.

7.56. PRUEBA 51: CONSULTAR OBRA EN OPEN LIBRARY REAL.

1. Envié GET a /servicios/libros/OL1731119W. Sin cuerpo.

2. La petición usó No Auth.

3. Contrasté HTTP 200 con el contrato y ejecuté dos aserciones en Postman.

Resultados:

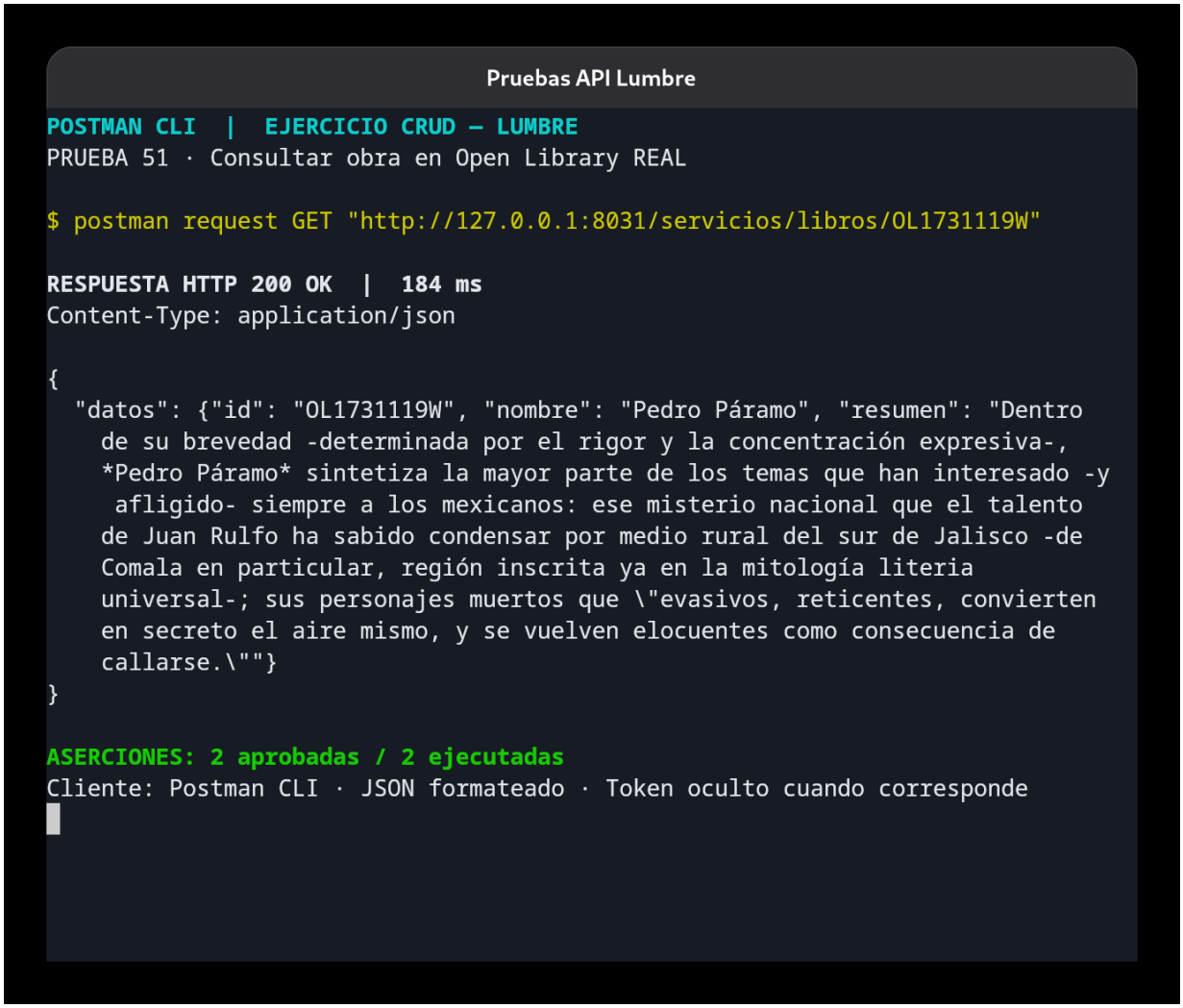


Figura 51. HTTP 200; 184 ms; 2 aserciones aprobadas.

El backend obtuvo una ficha real y normalizó título y descripción para el contrato local.

8. RESULTADOS Y EVIDENCIAS.

Se completaron 51 peticiones y 102 aserciones sin fallos en las ejecuciones individuales. Los errores HTTP de esta tabla fueron enviados intencionalmente y coinciden con el estado esperado.

N.º	Método	Prueba	HTTP	Test
1	GET	Consultar los recursos de la API	200	2/2
2	POST	Rechazar una contraseña incorrecta	401	2/2
3	POST	Obtener el token JWT de acceso	200	2/2
4	GET	Rechazar una petición sin token	401	2/2
5	GET	Rechazar un token vencido	403	2/2
6	POST	Crear un género literario	201	2/2
7	GET	Consultar todos los géneros	200	2/2
8	GET	Consultar un género por ID	200	2/2
9	PUT	Actualizar un género	200	2/2
10	POST	Crear un libro vinculado al género	201	2/2
11	GET	Consultar el catálogo de libros	200	2/2
12	GET	Consultar un libro por ID	200	2/2
13	PUT	Actualizar estante y ejemplares	200	2/2
14	DELETE	Proteger un género con libros asociados	400	2/2
15	PUT	Rechazar ejemplares negativos	400	2/2
16	GET	Rechazar un ID con formato inválido	400	2/2
17	DELETE	Eliminar un libro	200	2/2

Entorno: Express y MongoDB reales. Solo 22–25 utilizan Open Library simulado. Las capturas P1–P4 de Postman Desktop complementan el recorrido automatizado, sin contarse como nuevas aserciones de la colección.

8. RESULTADOS Y EVIDENCIAS (CONTINUACIÓN).

Se completaron 51 peticiones y 102 aserciones sin fallos en las ejecuciones individuales. Los errores HTTP de esta tabla fueron enviados intencionalmente y coinciden con el estado esperado.

N.º	Método	Prueba	HTTP	Test
18	GET	Comprobar que el libro ya no existe	404	2/2
19	DELETE	Eliminar el género sin libros asociados	200	2/2
20	GET	Comprobar el catálogo vacío	200	2/2
21	GET	Comprobar la eliminación del género	404	2/2
22	GET	Buscar obras: Open Library SIMULADO	200	2/2
23	GET	Consultar ficha: proveedor SIMULADO	200	2/2
24	GET	Obra inexistente: proveedor SIMULADO	404	2/2
25	GET	Falla del proveedor: respuesta 502	502	2/2
26	POST	Renovar acceso para pruebas negativas	200	2/2
27	POST	Login sin campos requeridos	400	2/2
28	GET	Token inválido en consulta general	401	2/2
29	POST	Alta de género sin nombre	400	2/2
30	POST	Crear género para validar conflictos	201	2/2
31	POST	Alta de género duplicado	400	2/2
32	PUT	Cambio de género con nombre vacío	400	2/2
33	POST	Crear segundo género	201	2/2
34	PUT	Renombrado con nombre duplicado	409	2/2

Entorno: Express y MongoDB reales. Solo 22–25 utilizan Open Library simulado. Las capturas P1–P4 de Postman Desktop complementan el recorrido automatizado, sin contarse como nuevas aserciones de la colección.

8. RESULTADOS Y EVIDENCIAS (CONTINUACIÓN).

Se completaron 51 peticiones y 102 aserciones sin fallos en las ejecuciones individuales. Los errores HTTP de esta tabla fueron enviados intencionalmente y coinciden con el estado esperado.

N.º	Método	Prueba	HTTP	Test
35	GET	Consulta de género con ID inválido	400	2/2
36	PUT	Cambio de género inexistente	404	2/2
37	DELETE	Baja de género inexistente	404	2/2
38	POST	Alta de libro sin autor	400	2/2
39	POST	Alta de libro con género inexistente	400	2/2
40	POST	Alta de libro con ejemplares fraccionarios	400	2/2
41	PUT	Cambio de libro inexistente	404	2/2
42	PUT	Cambio de libro con campos no permitidos	400	2/2
43	DELETE	Baja de libro inexistente	404	2/2
44	DELETE	Baja de libro con ID inválido	400	2/2
45	DELETE	Eliminar género de control	200	2/2
46	DELETE	Eliminar género de pruebas negativas	200	2/2
47	GET	Confirmar géneros vacíos	200	2/2
48	GET	Búsqueda externa con parámetro vacío	400	2/2
49	GET	Detalle externo con ID inválido	400	2/2
50	GET	Buscar Pedro Páramo en Open Library REAL	200	2/2
51	GET	Consultar obra en Open Library REAL	200	2/2

Entorno: Express y MongoDB reales. Solo 22–25 utilizan Open Library simulado. Las capturas P1–P4 de Postman Desktop complementan el recorrido automatizado, sin contarse como nuevas aserciones de la colección.

8.1. TRAZABILIDAD Y VALIDACIÓN FINAL.

Elemento	Definición / responsabilidad
Criterio 1 · SDD	docs/ESPECIFICACION-SDD.md y apartados 1–1.2; se declara el alcance de la iteración sin alterar la cronología.
Criterio 2 · CRUD	Apartado 2; pruebas 6–21 y 29–47; colección con contratos, cuerpos, respuestas y scripts.
Criterio 3 · Capas	Apartado 3; routers/control/service/modelo/middleware con responsabilidades separadas.
Criterio 4 · API externa	Apartado 4; búsqueda y detalle reales 50–51; simulación 22–25; validación 48–49.
Criterio 5 · JWT	Apartado 5; login 2–5 y 26–28; pruebas Node de claims, algoritmo y Bearer.
Criterio 6 · Código	Apartado 6; configuración, esquema, conexión, dependencias, funciones y decisiones.
Criterio 7 · Postman	Apartado 7; cuatro capturas Desktop, 51 CLI, colección importable y resultados JSON.
Criterio 8 · Entrega	Este reporte, código fuente, especificación, capturas, colección y guía de reproducción.

npm test con OPENLIBRARY_TEST_URL activo comprobó cuatro pruebas automatizadas aprobadas, ninguna omitida. Se añadieron aserciones JWT a la prueba de autenticación. El archivo datos/pruebas-node.txt conserva la salida de la verificación final.

datos/coleccion-postman.txt conserva la ejecución completa de la colección. SHA256SUMS.txt permite comprobar la integridad de las capturas. La persistencia se verificó consultando después de guardar, modificar y eliminar.

8.2. CONCLUSIONES Y LÍMITES.

La práctica permitió verificar una API REST de biblioteca con dos colecciones relacionadas. Las altas produjeron documentos consultables; las modificaciones conservaron identificadores y cambiaron datos; las bajas se confirmaron con 404 y listados vacíos. La regla de género ocupado se comprobó antes y después de eliminar el libro relacionado.

La iteración SDD hizo explícitos los contratos antes de reorganizar el código. Separar HTTP de las reglas de negocio permitió controlar validaciones y errores sin duplicarlas en los controladores. Las pruebas se derivaron de los requerimientos y contrastaron tanto estados exitosos como rechazos esperados.

JWT permitió demostrar la diferencia entre autenticar credenciales y verificar solicitudes posteriores. La firma protege la integridad, no oculta el payload. La verificación de algoritmo, claims y vencimiento impidió usar tokens incompletos o alterados para consultar el CRUD.

El consumo de Open Library se comprobó con el proveedor público y también con escenarios simulados reproducibles. Las respuestas externas se adaptaron a un contrato local y los errores se tradujeron de forma controlada. Las simulaciones prueban el manejo del código, mientras que las peticiones reales prueban conectividad e integración en el momento registrado.

Límites declarados: administrador único, uso local, listados sin paginación y comprobación de relaciones sin transacciones concurrentes. No se hizo prueba de carga ni despliegue público. La portada conserva la fecha solicitada y la evidencia su fecha real. La especificación inicial de esta iteración no acredita SDD previo a la primera versión del proyecto.

Para defender la implementación se puede seguir un alta desde router hasta MongoDB, explicar por qué un género ocupado no se elimina, mostrar la verificación JWT y repetir la colección. Las decisiones, resultados y archivos necesarios quedan incluidos en la entrega.

8.3. REPRODUCCIÓN Y FUENTES.

1. Abrir Ejercicio CRUD, instalar con npm ci y configurar .env con la plantilla. Iniciar MongoDB con docker compose up -d.

2. Seguir reporte/LEEME.txt para levantar las dos instancias temporales y la simulación. Ejecutar la colección en orden con tokenVencido temporal.

3. Ejecutar OPENLIBRARY_TEST_URL=http://127.0.0.1:4901/openlibrary npm test para la verificación del código.

4. Detener las instancias temporales con Ctrl+C. Conservar el PDF, código, colección, JSON y capturas al entregar.

Fuentes técnicas consultadas.

Open Library — Search API: https://openlibrary.org/dev/docs/api/search. Open Library — Works API: https://openlibrary.org/dev/docs/api/books. Estas fuentes definen las solicitudes y campos del proveedor externo.

Postman — Send API requests and get response data: https://learning.postman.com/docs/use/send-requests/requests/. Describe solicitudes, respuestas, autorización y reutilización de variables. La versión observada de Desktop es 12.32.3 y CLI 1.74.0.

El comportamiento específico de Lumbre se documentó a partir de app.js, modelos, controladores, servicios, middleware y sus pruebas locales. Las versiones instaladas se reproducen con package-lock.json. No se infiere el resultado de una prueba únicamente por la documentación de una librería.

El PDF es la versión con paginación verificada. El Word contiene el mismo contenido editable; un editor o fuentes diferentes pueden redistribuir sus páginas. La carpeta reporte contiene originales de las evidencias y scripts para regenerar documentos.